

Laporan M2 Tugas Besar 1

IF2010 Pemrograman Berorientasi Objek 2025/2026

Nimonspoli

Kode Kelompok : PCH

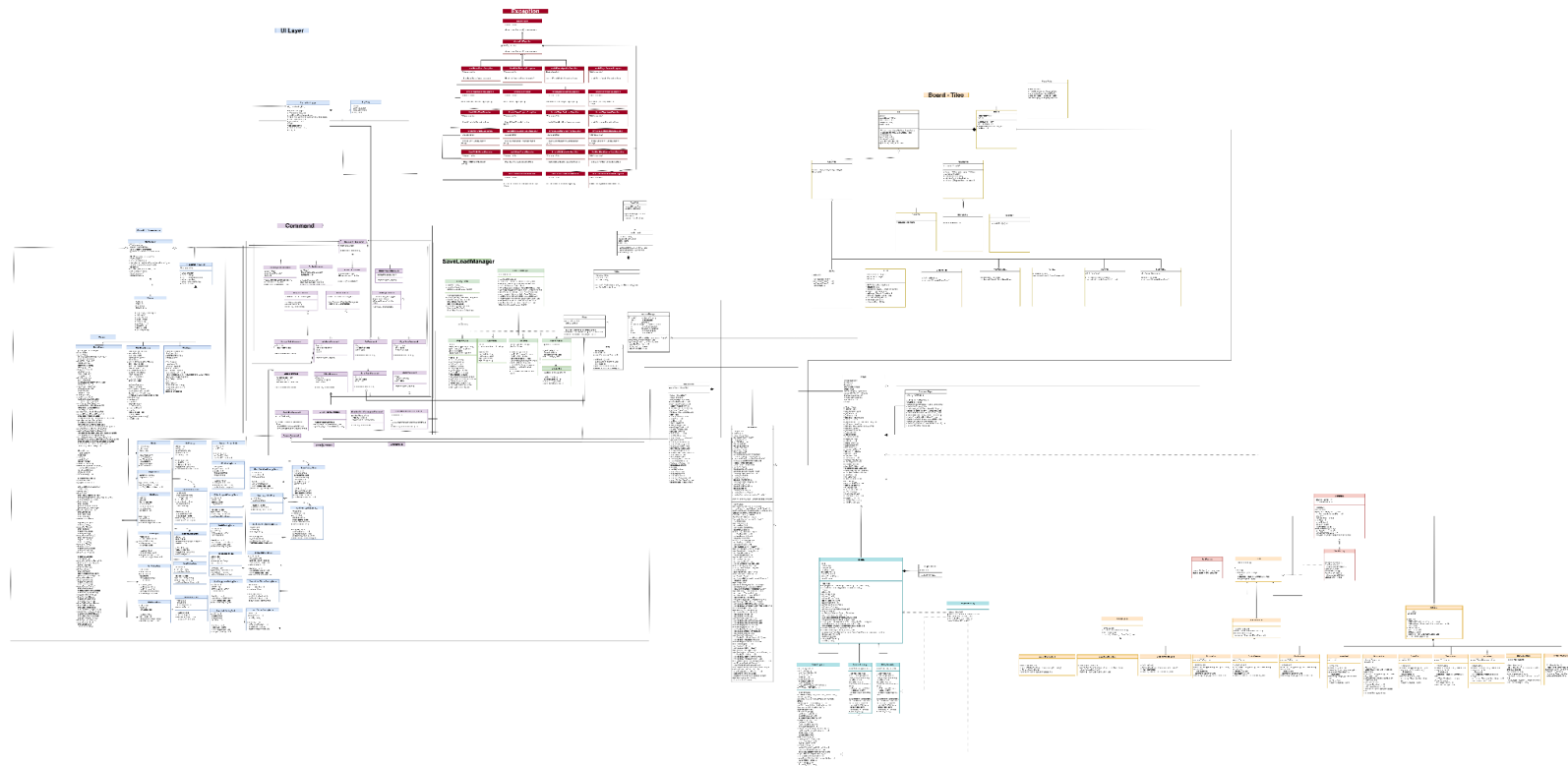
Nama Kelompok : placeholder

1. 13524047 / Muhammad Jordan Ferimeison
2. 13524049 / Arina Azka
3. 13524075 / Hakam Avicena Mustain
4. 13524077 / Yavie Azka Putra Araly
5. 13524079 / Angelina Andra Alanna



PCH - placeholder

1. Diagram Kelas



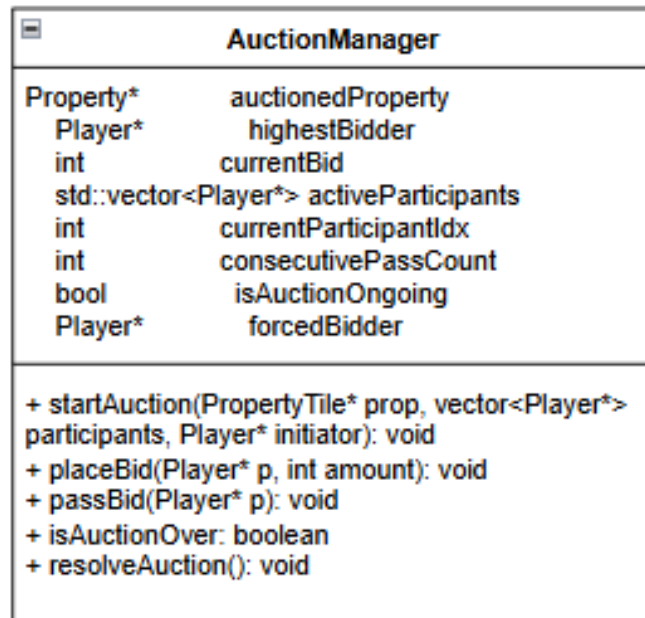
Gambar 1.1 Diagram Kelas (Placeholder)

Link UML Diagram dapat diakses melalui link berikut.

https://drive.google.com/file/d/1mZjQFs4lrc1otYZfvWGOYSBPdEhT9R6J/view?usp=drive_link

2. Deskripsi Kelas

2.1. Kelas AuctionManager



Kelas AuctionManager memisahkan logika pelanggan dari alur giliran utama permainan.

List Atribut:

- PropertyTile* currentProperty: menyimpan property lelang yang sedang berjalan
- Int currentHighestBid: menyimpan harga tertinggi property lelang yang sedang berjalan
- vector<Player*> activeBidders
- Int passCount
- Player* HighestBidder

List Method:

- Void placeBid(): melakukan validasi tawaran
- Void passBid(): mengurangi jumlah partisipan aktif
- Void resolveAuction: mendistribusikan properti kepada highestBidder saat isAuctionOver()
- Boolean isAuctionOver: menandakan apakah lelang masih berjalan atau sudah berakhir

Kelas ini dipanggil secara eksklusif oleh LelangCommand sebagai jembatan input pemain, dan memiliki relasi *asociation* dengan PropertyTile dan sekumpulan objek Player.

2.2. Kelas Dadu

Dice
- daduVal1:int - daduVal2: int - playerConcecutiveDoubles: int
+ rollRandom(): void + setManual(int v1, int v2): void + getTotal(): int + getConsecutiveDoubles(): int + resetConsecutiveDouvles(): void

Kelas ini bertanggung jawab untuk menangani logika probabilitas dan pergerakan acak di dalam permainan.

List Atribut:

- Int daduVal1

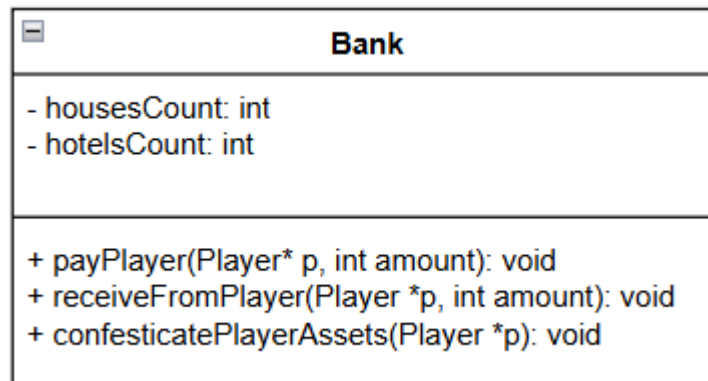
- Int daduVal2

List Method:

- void rollRandom: menghasilkan angka acak
- void setManual: menghasilkan angka untuk kebutuhan testing spesifik

Kelas ini memiliki relasi dengan LemparDaduCommand dan AturDaduCommand yang akan memanggil method-method tersebut untuk menentukan jumlah langkah bidak pemain.

2.3. Kelas Bank



Kelas Bank bertindak sebagai pengelola sirkulasi keuangan utama permainan yang memiliki akses ke dana tak terbatas.

List Atribut:

- Int housesCount
- Int hotelsCount

List Method:

- `payPlayer()`: memberikan uang kepada player, misal saat gadai.
- `confiscatePlayerAssets()`: menyita dan melelang aset saat pemain bangkrut

Kelas ini memiliki relasi dependency dengan hampir seluruh kelas command transaksi, seperti `BeliCommand`, `BayarPajakCommand`, dan `GadaiCommand` yang mendelegasikan eksekusi pemotongan/penambahan saldo kepadanya

2.4. Kelas Tile

Tile
<pre># id: int # colorDisplay: string # tileType: tileType # tileName: string # code: string</pre>
<pre>+ Tile(int, string, string, TileType, ColorCode) + onLanded(Player&, GameState&): void + getIndex(): int + getCode(): string + getName(): string + getTileType(): TileType + getDisplayColor(): ColorCode</pre>

Kelas Tile dibuat secarai petak secara umum dilanjutkan dengan petak aksi dan petak properti setelah itu masing-masing tipe petaknya.

List Atribut:

- Int id: sebagai pembeda dan identifier setiap tile
- String warna: pewarnaan tile untuk display
- String nama: nama tile
- String kode
- String tipe tile

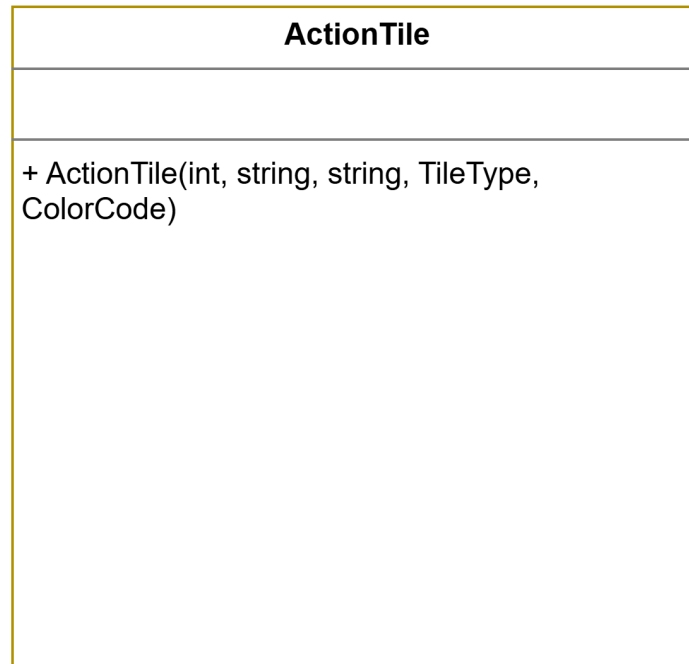
List method:

- onLanded(player&, GameState&): aksi yang diimplementasikan pada setiap player yang berkunjung ke tile tersebut
- getID()
- getWarna(): mengembalikan warna untuk display
- getNama()
- getKode()
- getTipe()

Hubungan dengan kelas lain:

- Kelas tile berhubungan inheritance dengan action tile dan property tile karena kelas tile berhubungan general-spesifik untuk action tile dan property tile.

2.4.1. Action Tile



Kelas Action Tile dibuat secara inheritance dari petak secara umum dilanjutkan dengan masing-masing tipe petak aksi.

Hubungan dengan kelas lain:

- Kelas Action Tile berhubungan inheritance dengan festival tile, go tile, jail tile, go to jail tile, free parking tile, tax tile, dan card tile karena kelas action tile berhubungan general-spesifik untuk tile-tile tersebut.

2.4.1.1. Festival Tile

FestivalTile
- log:TransactionLogger
+FestivalTile(index) + onLanded(Player&, GameState&)

Kelas Festival Tile dibuat secara inheritance dari action tile sebagai salah satu jenis dari action tile.

List method:

- onLanded(Player&, GameState&): aksi yang dapat melipatgandakan harga sewa dari properti yang dimiliki.

2.4.1.2. GoTile

GoTile
- salary: int
+ GoTile(index, salary) + onLanded(Player&): void + onPassed(Player&): void + getSalary(): int

Kelas Go Tile dibuat secara inheritance dari action tile sebagai salah satu jenis dari action tile.

List Atribut:

- Int salary

List method:

- onLanded(Player&, GameState&): aksi yang dapat menambahkan state uang player sesuai salarynya.
- onPassed(Player&, GameState&): aksi yang dapat menambahkan state uang player sesuai salarynya.

2.4.1.3. Jail Tile

JailTile
- inmates: vector<player> - visitor: vector<player> - jailfine: int
+ JailTile(int index, int jailfine) + getJailFine():int + onLanded(Player&, GameState&):void + sendToJail(Player&):void + tryEscape(Player&, Dice&):bool + payFine(Player&, Bank&): void + useJailCard(Player&):void + release(Player&): void + isInmate(Player&): bool

Kelas Jail Tile dibuat secara inheritance dari action tile sebagai salah satu jenis dari action tile.

List Atribut:

- Int fine
- Vector of player inmates
- Vector of player visitor

List method:

- onLanded(Player&, GameState&): dianggap visitor
- isInmate(Player&) mengembalikan player adalaht tahanan
- releaseInmate(Player&): melepas player dari tahanan
- payFine(Player&, Bank&): tahanan membayar denda untuk keluar dari penjara
- getFine()
- useJailCard(Player&): menggunakan kartu bebas penjara
- tryEscape(Player&, Dice&): tahanan mencoba keluar penjara dengan lemparan dadu
- sendToJail(Player&): memasukkan player ke dalam tahanan

2.4.1.4. Go To Jail Tile

GoToJailTile
+ GoToJailTile(index) + onLanded(Player&, GameState&)

Kelas Go To Jail Tile dibuat secara inheritance dari action tile sebagai salah satu jenis dari action tile.

List method:

- onLanded(Player&, GameState&): mengirimkan player ke jail

2.4.1.5. Free Parking Tile



Kelas Free Parking Tile dibuat secara inheritance dari action tile sebagai salah satu jenis dari action tile.

List method:

- onLanded(Player&, GameState&): Tidak terjadi apa-apa

2.4.1.6. Tax Tile



Kelas Tax Tile dibuat secara inheritance dari action tile sebagai salah satu jenis dari action tile.

List method:

- onLanded(Player&, GameState&): player membayar tax sesuai dengan jenis tax tile nya

2.4.1.7. Card Tile

CardTile
- deck: CardDeck*
+ CardTile(int index, CardDeck*) + onLanded(Player&, GameState&): void

Kelas Card Tile dibuat secara inheritance dari action tile sebagai salah satu jenis dari action tile.

List atribut:

- Deck pointer to CardDeck

List method:

- onLanded(Player&, GameState&): Player mengambil kartu yang ada pada deck

Hubungan dengan kelas lain:

- Sesuai dengan tanggung jawab nya adalah memberikan aksi kepada player untuk mengambil kartu pada deck

2.4.2. Property Tile

PropertyTile
property: Property*
+ PropertyTile(int, string, string, TileType, ColorCode, Property*) + getProperty(): Property* + calculateRent(int diceTotal):int + onLanded(Player&, Gamestate&): void

Kelas Property Tile dibuat secara inheritance dari petak secara umum dilanjutkan dengan masing-masing tipe petak aksi.

List atribut:

- Property*

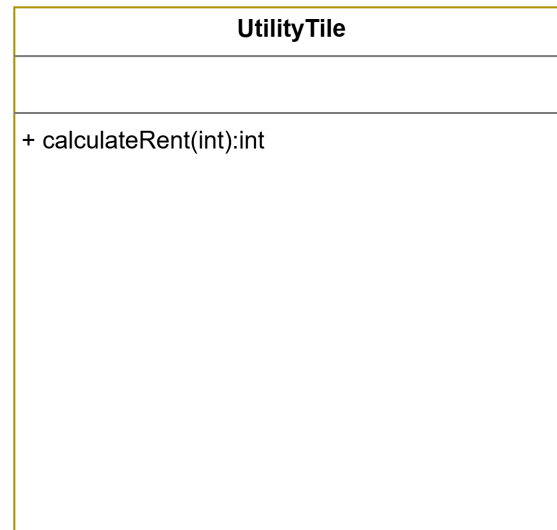
List Metode:

- getProperty()
- calculateRent(int diceTotal):
- onLanded(Player&, GameState&)

Hubungan dengan kelas lain:

- Kelas Property Tile berhubungan inheritance dengan utility tile, railroad tile, dan street tile karena kelas property tile berhubungan general-spesifik untuk tile-tile tersebut.

2.4.2.1. Utility Tile

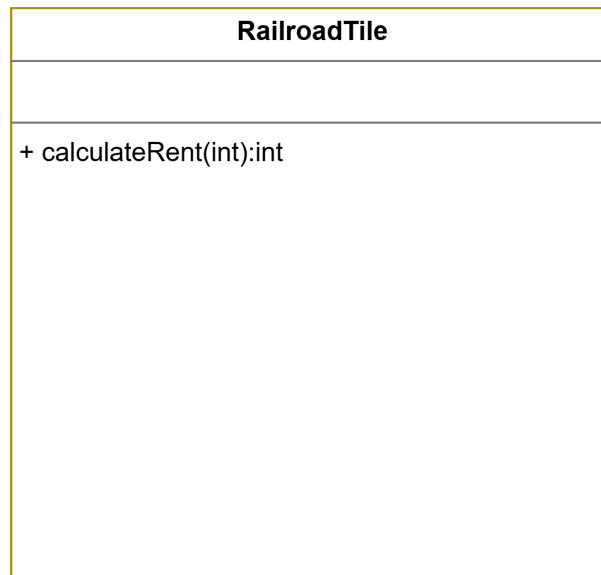


Kelas Utility Tile dibuat secara inheritance dari property tile sebagai salah satu jenis dari action tile.

List method:

- calculateRent(int diceTotal)

2.4.2.2. Railroad Tile

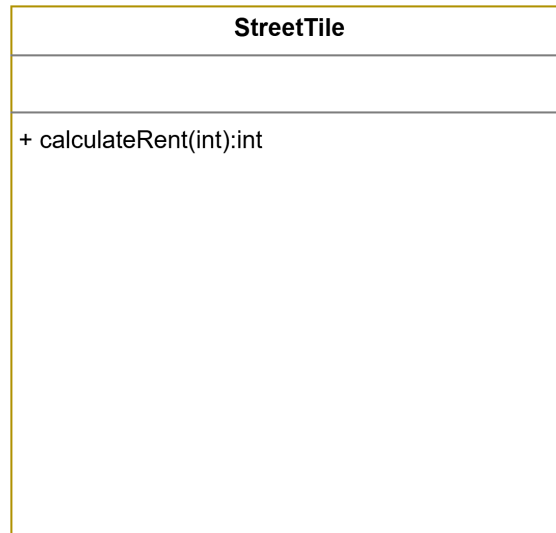


Kelas Railroad Tile dibuat secara inheritance dari property tile sebagai salah satu jenis dari action tile.

List method:

- calculateRent(int diceTotal)

2.4.2.3. Street Tile



Kelas Street Tile dibuat secara inheritance dari property tile sebagai salah satu jenis dari action tile.

List method:

- calculateRent(int diceTotal)

2.5. Kelas Board

Board
- Tiles: vector<Tile*> - size: int
+ Board(vector<Tile*>) + getTile(index): Tile* + getNextTile(int cur, int step): int + getSize(): int

Kelas board sebagai pembungkus dari kumpulan objek-objek petak di bawahnya. Atribut dari kelas ini hanya vector of pointer to tile.

List Atribut:

- Vector of pointer to tile: kumpulan tile untuk berukuran size
- Int size of contained : ukuran papan

List Method:

- getTile(int): memanggil anggota tile dalam atribut vector of pointer to tile
- getNextTile(int idx, int step): memanggil tile selanjutnya sesuai langkah pada dadu
- getSize(): mengembalikan ukuran papan

Hubungan dengan kelas lain:

Kelas board berhubungan secara komposisi dengan kelas tile karena tile tidak memiliki konteks bila tidak dimasukkan dalam board dan hubungan board has-a tile.

2.6. Kelas Board Factory

BoardFactory
<pre> + BoardFactory() + createBoard(vector<PropertyData>, vector<ActionData>, SpecialConfig, CardDeck<Card>*, CardDeck<Card>*, vector<unique_ptr<Property>>): Board* </pre>

Kelas board factory sebagai pembangun board dari pembaca file konfigurasi dan mentranslasikan menjadi kelas yang dapat diprogramkan sebagai sebuah struktur data. Kelas ini membutuhkan kelas-kelas lainnya dari config loader yang membaca dari file config terdapat PropertyData dari property.txt, ActionData dari aksi.txt, SpecialConfig dari special.txt, CardDeck untuk kesempatan, CardDeck untuk dana umum, dan isi properti dari Property Data.

List Method:

- `createBoard(vector<PropertyData>, vector<ActionData>, SpecialConfig, CardDeck<Card>*, CardDeck<Card>*, vector<unique_ptr<Property>>)`: mengembalikan pointer to board yang sudah beirisi tile-tile berdasarkan file konfigurasi

Hubungan dengan kelas lain:

- Kelas board factory berhubungan secara realisasi terhadap kelas board karena kelas board factory ini yang membangun atau merealisasikan kelas board

2.7. Kelas Card

Card
- description: string
+ Card() + Card(description: string) + ~Card() + execute(p: Player, gs: GameState): void + getDescription(): string

Card adalah kelas dasar abstrak untuk seluruh kartu dalam permainan. Kelas ini menyimpan atribut `description` sebagai deskripsi efek kartu. Method `getDescription()` digunakan untuk mengambil deskripsi tersebut, sedangkan `execute()` menjadi method utama yang merepresentasikan aksi kartu ketika digunakan atau ditarik oleh pemain. Karena `execute()` bersifat virtual murni pada header, setiap kelas turunan wajib mendefinisikan efek kartunya masing-masing.

Relasi kelas:

- Card menjadi parent class dari `ChanceCard`, `GeneralFundCard`, dan kemungkinan juga `SkillCard`.
- Card berelasi dengan `Player` karena efek kartu diterapkan kepada pemain tertentu.

- Card berelasi dengan GameState karena eksekusi kartu membutuhkan akses ke kondisi permainan.
- CardDeck<Card> menyimpan objek kartu secara polymorphic menggunakan pointer ke Card.

2.7.1. Kelas ChanceCard

ChanceCard
+ ChanceCard() + ChanceCard(description: string) + ~ChanceCard() + execute(p: Player, gs: GameState): void

ChanceCard adalah kelas abstrak turunan dari Card yang merepresentasikan kartu Kesempatan. Kelas ini tidak menambahkan atribut baru, tetapi berfungsi sebagai kategori khusus untuk kartu-kartu kesempatan. Konstruktor ChanceCard meneruskan deskripsi kartu ke kelas induk Card. Method execute() tetap dibuat abstrak agar setiap kartu kesempatan memiliki implementasi efek yang berbeda sesuai jenis kartunya.

Relasi kelas:

- ChanceCard adalah child class dari Card.
- ChanceCard menjadi parent class untuk kartu kesempatan seperti GoToJailCard, MoveBackThreeCard, dan NearestStationCard.
- ChanceCard dapat disimpan dalam CardDeck<Card>.
- DeckFactory membuat deck kesempatan yang berisi objek turunan dari ChanceCard.

2.7.1.1. Kelas NearestStationCard

NearestStationCard
<pre> + NearestStationCard() + NearestStationCard(type: string, description: string) + ~NearestStationCard() + execute(p: Player, gs: GameState): void </pre>

NearestStationCard adalah kelas turunan dari ChanceCard yang merepresentasikan kartu untuk memindahkan pemain ke stasiun terdekat. Kelas ini tidak memiliki atribut tambahan karena target stasiun dihitung langsung saat kartu dieksekusi. Method execute() mengambil posisi pemain saat ini, membaca seluruh tile pada board, lalu mencari tile bertipe "RAILROAD" dengan jarak maju paling kecil dari posisi pemain. Jika stasiun ditemukan, kartu meminta GameMaster memindahkan pemain sebanyak jumlah langkah menuju stasiun tersebut.

Relasi kelas:

- NearestStationCard adalah child class dari ChanceCard.
- NearestStationCard mewarisi description dari Card.
- NearestStationCard menggunakan GameState untuk mengakses Board dan GameMaster.
- NearestStationCard menggunakan Board untuk membaca ukuran board dan tile.
- NearestStationCard berelasi dengan Tile karena mengecek tipe tile "RAILROAD".
- NearestStationCard menggunakan GameMaster untuk menjalankan perpindahan pemain.
- DeckFactory memasukkan NearestStationCard ke dalam deck Kesempatan.

2.7.1.2. Kelas MovebackThree

MoveBackThreeCard
<pre> + MoveToBackThreeCard() + MoveToBackThreeCard(type: string, description: string) + ~MoveToBackThreeCard() + execute(p: Player, gs: GameState): void </pre>

MoveToBackThreeCard adalah kelas turunan dari ChanceCard yang merepresentasikan kartu untuk memundurkan pemain sebanyak tiga petak. Kelas ini tidak memiliki atribut tambahan karena jumlah langkahnya tetap, yaitu -3. Method execute() mengambil GameMaster dari GameState, lalu memanggil movePlayer(&p, -3) untuk memindahkan pemain mundur tiga langkah. Dengan menggunakan GameMaster, perpindahan pemain tetap mengikuti mekanisme utama permainan, bukan hanya mengubah posisi secara manual.

Relasi kelas:

- MoveToBackThreeCard adalah child class dari ChanceCard.
- MoveToBackThreeCard mewarisi deskripsi kartu dari Card.
- MoveToBackThreeCard menggunakan GameState untuk mengambil GameMaster.
- MoveToBackThreeCard menggunakan GameMaster untuk memindahkan pemain.
- MoveToBackThreeCard berelasi dengan Player karena efeknya mengubah posisi pemain.
- DeckFactory memasukkan MoveToBackThreeCard ke dalam deck Kesempatan.

2.7.1.3. Kelas GoToJailCard

GoToJailChanceCard
<pre> + GoToJailCard() + GoToJailCard(type: string, description: string) + ~GoToJailCard() + execute(p: Player, gs: GameState): void </pre>

GoToJailCard adalah kelas turunan dari ChanceCard yang merepresentasikan kartu untuk memindahkan pemain langsung ke penjara. Kelas ini tidak memiliki atribut tambahan karena efeknya bersifat langsung. Method `execute()` mencari indeks tile penjara melalui board dengan kode "PEN", lalu mengubah posisi pemain ke indeks tersebut dan memanggil `goToJail()` pada objek pemain. Dengan demikian, kartu ini bertanggung jawab untuk mengubah posisi sekaligus status pemain menjadi berada di penjara.

Relasi kelas:

- GoToJailCard adalah child class dari ChanceCard.
- GoToJailCard mewarisi `description` dari Card.
- GoToJailCard menggunakan GameState untuk mengakses Board.
- GoToJailCard menggunakan Board untuk mencari indeks petak penjara.
- GoToJailCard berelasi dengan Player karena mengubah posisi dan status pemain.
- DeckFactory memasukkan GoToJailCard ke dalam deck Kesempatan.

2.7.2. Kelas GeneralFundCard

GeneralFundCard
+ GeneralFundCard() + GeneralFundCard(description: string) + ~GeneralFundCard() + execute(p: Player, gs: GameState): void

GeneralFundCard adalah kelas turunan dari Card yang merepresentasikan kartu Dana Umum. Kelas ini tidak terlihat memiliki atribut tambahan pada file implementasi yang dilampirkan. Fungsinya adalah menjadi kelas dasar untuk kartu-kartu dana umum yang memiliki efek keuangan, seperti menerima uang dari pemain lain, membayar biaya tertentu, atau membayar kepada semua pemain. Konstruktor GeneralFundCard hanya meneruskan deskripsi kartu ke kelas Card.

Relasi kelas:

- GeneralFundCard adalah child class dari Card.
- GeneralFundCard menjadi parent class dari BirthdayCard, DoctorFeeCard, dan ElectionCard.
- GeneralFundCard digunakan oleh DeckFactory saat membuat deck Dana Umum.
- Kartu turunan GeneralFundCard berinteraksi dengan Player, GameState, dan GameMaster untuk menjalankan efek pembayaran.

2.7.2.1. Kelas BirthdayCard

GeneralFundCard
+ GeneralFundCard() + GeneralFundCard(description: string) + ~GeneralFundCard() + execute(p: Player, gs: GameState): void

BirthdayCard adalah kelas turunan dari GeneralFundCard yang merepresentasikan kartu ulang tahun. Atribut amountPerPlayer menyimpan jumlah uang yang harus dibayarkan oleh setiap pemain lain kepada pemain yang menarik kartu. Method getAmountPerPlayer() digunakan untuk mengambil nominal tersebut. Method execute() bertanggung jawab membuat antrian pembayaran dari semua pemain aktif lain kepada pemain penerima kartu, lalu meminta GameMaster memproses pembayaran pertama. Dengan cara ini, pembayaran banyak pemain tidak langsung diproses sekaligus, tetapi dikelola melalui pending payment queue pada GameState.

Relasi kelas:

- BirthdayCard adalah child class dari GeneralFundCard.
- BirthdayCard mewarisi deskripsi kartu dari Card.
- BirthdayCard menggunakan GameState untuk mengambil daftar pemain aktif dan mengatur antrian pembayaran.
- BirthdayCard menggunakan GameMaster untuk memproses pembayaran kartu secara bertahap.
- BirthdayCard berelasi dengan Player karena efek kartunya memindahkan uang dari pemain lain ke pemain penerima kartu.

2.7.2.2. Kelas DoctorFeeCard

DoctorFeeCard
- doctorFee: int
+ DoctorFeeCard() + DoctorFeeCard(type: string, description: string, amount: int) + ~DoctorFeeCard() + getDoctorFee(): int + execute(p: Player, gs: GameState): void

DoctorFeeCard adalah kelas turunan dari GeneralFundCard yang merepresentasikan kartu biaya dokter. Atribut doctorFee menyimpan jumlah uang yang harus dibayar pemain. Method getDoctorFee() digunakan untuk mengambil nominal biaya tersebut. Method execute() memeriksa status pemain terlebih dahulu; jika pemain sudah bangkrut, efek kartu tidak dilanjutkan. Jika saldo pemain cukup, saldo langsung dikurangi. Jika saldo tidak cukup, pembayaran diteruskan ke GameMaster melalui handleDebtPayment() agar mekanisme utang atau kebangkrutan dapat diproses sesuai aturan permainan.

Relasi kelas:

- DoctorFeeCard adalah child class dari GeneralFundCard.
- DoctorFeeCard menggunakan Player untuk mengecek status, saldo, dan mengurangi uang pemain.
- DoctorFeeCard menggunakan GameState untuk mendapatkan pointer ke GameMaster.
- DoctorFeeCard menggunakan GameMaster untuk menangani pembayaran ketika saldo pemain tidak cukup.
- DoctorFeeCard termasuk bagian dari deck Dana Umum yang dibuat oleh DeckFactory.

2.7.2.3. Kelas ElectionCard

ElectionCard
- amountPerPlayer: int
+ ElectionCard() + ElectionCard(type: string, description: string, amount: int) + ~ElectionCard() + getAmountPerPlayer(): int + execute(p: Player, gs: GameState): void

ElectionCard adalah kelas turunan dari GeneralFundCard yang merepresentasikan kartu biaya pencalonan atau pemilihan. Atribut amountPerPlayer menyimpan jumlah uang yang harus dibayar pemain kepada setiap pemain lain. Method getAmountPerPlayer() digunakan untuk mengambil nominal pembayaran tersebut. Method execute() memeriksa apakah pemain sudah bangkrut; jika tidak, method ini membuat antrian pembayaran dari pemain yang terkena kartu kepada semua pemain aktif lain. Setelah antrian dibuat, GameMaster diminta untuk memproses pembayaran pertama melalui processNextCardPayment().

Relasi kelas:

- ElectionCard adalah child class dari GeneralFundCard.
- ElectionCard menggunakan Player sebagai pembayar utama.
- ElectionCard menggunakan GameState untuk mengambil daftar pemain aktif dan membuat pending payment queue.
- ElectionCard menggunakan GameMaster untuk memproses pembayaran kartu satu per satu.
- ElectionCard termasuk kartu Dana Umum yang dimasukkan ke deck oleh DeckFactory.

2.7.3. Kelas SkillCard

SkillCard
- type: string - used: bool
+ SkillCard() + SkillCard(description: string, type: string) + SkillCard(type: string, description: string, used: bool) + ~SkillCard() + isUsed(): bool + markUsed(): void + getType(): string + execute(p: Player, gs: GameState): void + successMessage(): string

SkillCard adalah kelas abstrak turunan dari Card yang merepresentasikan kartu kemampuan. Kelas ini menambahkan atribut type untuk menyimpan jenis kartu kemampuan dan used untuk menandai apakah kartu sudah digunakan. Method isUsed() digunakan untuk mengecek status penggunaan kartu, sedangkan markUsed() menandai kartu sebagai sudah digunakan. Method getType() dipakai untuk mengambil nama jenis kartu, biasanya untuk kebutuhan UI, save/load, atau pesan hasil penggunaan kartu. Method execute() tetap virtual murni karena setiap skill card memiliki efek berbeda. Method successMessage() menyediakan pesan default ketika kartu berhasil digunakan, tetapi masih bisa dioverride oleh kelas turunan.

Relasi kelas:

- SkillCard adalah child class dari Card.
- SkillCard menjadi parent class untuk MoveCard, ShieldCard, TeleportCard, LassoCard, DemolitionCard, DiscountCard, dan FreeFromJailCard.
- SkillCard dapat disimpan dalam CardDeck<SkillCard>.
- Player dapat menyimpan daftar kartu kemampuan dalam bentuk pointer SkillCard*.
- CardFactory membuat objek turunan SkillCard berdasarkan tipe kartu.
- DeckFactory membuat deck kartu kemampuan berisi objek turunan SkillCard.

2.7.3.1. Kelas MoveCard

MoveCard
- steps: int
+ MoveCard() + MoveCard(steps: int) + MoveCard(type: string, description: string, used: bool, steps: int) + ~MoveCard() + execute(p: Player, gs: GameState): void + getSteps(): int + successMessage(): string

MoveCard adalah kelas turunan dari SkillCard yang digunakan untuk memindahkan pemain maju sejumlah langkah tertentu. Atribut steps menyimpan jumlah langkah perpindahan. Konstruktor default membuat nilai steps secara acak dari 1 sampai 6, sedangkan konstruktor lain memungkinkan nilai langkah ditentukan langsung, terutama saat load kartu dari file. Method execute() mengambil GameMaster dan Board dari GameState, memvalidasi ukuran board, menghitung posisi target dengan operasi modulo, lalu memindahkan pemain menggunakan teleportPlayer(). Setelah efek berhasil dijalankan, kartu ditandai sudah digunakan dengan markUsed().

Relasi kelas:

- MoveCard adalah child class dari SkillCard.
- MoveCard mewarisi atribut type, used, dan description.
- MoveCard menggunakan GameState untuk mengambil GameMaster dan Board.
- MoveCard menggunakan Board untuk mengetahui ukuran papan.
- MoveCard menggunakan GameMaster untuk memindahkan pemain ke posisi target.

- MoveCard berelasi dengan Player karena mengubah posisi pemain.
- MoveCard dapat dibuat oleh CardFactory dan dimasukkan ke skill deck oleh DeckFactory.

2.7.3.2. Kelas DiscountCard

DiscountCard
- discountPercent: int - duration: int
+ DiscountCard() + DiscountCard(discountPercent: int, duration: int) + DiscountCard(type: string, description: string, used: bool, discountPercent: double, duration: int) + ~DiscountCard() + getDuration(): int + decreaseDuration(): void + getDiscountPercent(): int + execute(p: Player, gs: GameState): void + successMessage(): string

DiscountCard adalah kelas turunan dari SkillCard yang memberikan efek diskon kepada pemain. Atribut discountPercent menyimpan besar diskon dalam persen, sedangkan duration menyimpan durasi efek diskon. Konstruktor default menghasilkan nilai diskon secara acak, yaitu sekitar 10 sampai 59 persen, dengan durasi awal 1 giliran. Method getDiscountPercent() dan getDuration() digunakan untuk membaca efek kartu, sedangkan decreaseDuration() mengurangi sisa durasi. Method execute() menerapkan diskon ke pemain melalui p.setDiscount(discountPercent). Method successMessage() menghasilkan pesan keberhasilan penggunaan kartu untuk ditampilkan di UI.

Relasi kelas:

- DiscountCard adalah child class dari SkillCard.
- DiscountCard berelasi dengan Player karena efek diskon disimpan pada objek pemain.
- DiscountCard dapat dibuat oleh CardFactory saat load data skill card.

- DiscountCard dimasukkan ke skill deck oleh DeckFactory.
- GameState diterima pada execute(), tetapi pada implementasi ini tidak digunakan langsung.

2.7.3.3. Kelas ShieldCard

ShieldCard
- duration: int
+ ShieldCard() + ShieldCard(type: string, description: string, used: bool, duration: int) + ~ShieldCard() + getDuration(): int + decreaseDuration(): void + execute(p: Player, gs: GameState): void + successMessage(): string

ShieldCard adalah kelas turunan dari SkillCard yang memberikan perlindungan kepada pemain dari tagihan atau sanksi selama durasi tertentu. Atribut duration menyimpan lama efek shield. Method getDuration() digunakan untuk membaca sisa durasi, sedangkan decreaseDuration() mengurangi durasi tersebut. Method execute() mengaktifkan shield pada pemain melalui p.activateShield(). Method successMessage() menghasilkan pesan khusus bahwa pemain sedang kebal terhadap sewa, pajak, atau sanksi merugikan selama giliran tersebut.

Relasi kelas:

- ShieldCard adalah child class dari SkillCard.
- ShieldCard menggunakan Player untuk mengaktifkan efek shield.
- ShieldCard menerima GameState pada execute(), tetapi implementasi saat ini tidak menggunakannya langsung.
- ShieldCard dapat dibuat oleh CardFactory.
- ShieldCard dimasukkan ke skill deck oleh DeckFactory.

- Efek ShieldCard berhubungan dengan sistem pembayaran, pajak, sewa, atau sanksi karena shield dapat melindungi pemain dari efek merugikan.

2.7.3.4. Kelas TeleportCard

TeleportCard
- targetPosition: int
+ TeleportCard() + TeleportCard(type: string, description: string, used: bool) + ~TeleportCard() + execute(p: Player, gs: GameState): void + setTargetPosition(pos: int, gs: GameState): void + getTargetPosition(): int + successMessage(): string

TeleportCard adalah kelas turunan dari SkillCard yang memungkinkan pemain berpindah ke petak tertentu pada board. Atribut targetPosition menyimpan indeks petak tujuan. Method setTargetPosition() digunakan untuk mengisi target setelah memvalidasi bahwa posisi berada dalam rentang ukuran board. Method getTargetPosition() digunakan untuk membaca posisi target yang sudah dipilih. Method execute() memvalidasi bahwa target sudah dipilih dan GameMaster tersedia, lalu memindahkan pemain ke targetPosition menggunakan teleportPlayer(). Setelah berhasil digunakan, target direset dan kartu ditandai sebagai sudah digunakan.

Relasi kelas:

- TeleportCard adalah child class dari SkillCard.
- TeleportCard menggunakan GameState untuk mengakses Board dan GameMaster.
- TeleportCard menggunakan Board untuk memvalidasi indeks target.
- TeleportCard menggunakan GameMaster untuk memindahkan pemain.
- TeleportCard berelasi dengan Player karena efeknya mengubah posisi pemain.
- TeleportCard dapat dibuat oleh CardFactory dan dimasukkan ke skill deck oleh DeckFactory.
- UI atau dialog pemilihan target dapat memanggil setTargetPosition() sebelum kartu dieksekusi.

2.7.3.5. Kelas LassoCard

LassoCard
- targetPlayerUsername: string
+ LassoCard() + LassoCard(type: string, description: string, used: bool) + ~LassoCard() + setTargetPlayerUsername(target: string): void + execute(p: Player, gs: GameState): void + successMessage(): string

LassoCard adalah kelas turunan dari SkillCard yang digunakan untuk menarik pemain lawan ke posisi pemain pengguna kartu. Atribut targetPlayerUsername menyimpan username pemain target apabila target dipilih secara eksplisit. Method setTargetPlayerUsername() digunakan untuk menentukan pemain target sebelum kartu dieksekusi. Method execute() mengambil board dan daftar pemain aktif dari GameState, lalu mencari target valid. Jika targetPlayerUsername diisi, kartu mencari pemain dengan username tersebut. Jika tidak diisi, kartu

memilih pemain terdekat di depan posisi pengguna. Setelah target ditemukan, kartu menggunakan GameMaster untuk memindahkan pemain target ke posisi pengguna kartu. Setelah berhasil, target direset dan kartu ditandai sudah digunakan.

Relasi kelas:

- LassoCard adalah child class dari SkillCard.
- LassoCard menggunakan GameState untuk mengambil Board, daftar pemain aktif, dan GameMaster.
- LassoCard menggunakan Board untuk mengetahui ukuran papan dan menghitung jarak antar pemain.
- LassoCard menggunakan Player sebagai pengguna kartu dan pemain target.
- LassoCard menggunakan GameMaster untuk melakukan teleport pemain target.
- LassoCard dapat dibuat oleh CardFactory dan dimasukkan ke skill deck oleh DeckFactory.
- UI atau dialog target pemain dapat memanggil setTargetPlayerUsername() sebelum kartu dieksekusi.

2.7.3.6. Kelas DemolitionCard

DemolitionCard
- targetPropertyId: int
+ DemolitionCard() + DemolitionCard(type: string, description: string, used: bool) + ~DemolitionCard() + getTargetPropertyId(): int + setTargetProperty(targetPropertyId: int): void + execute(p: Player, gs: GameState): void + successMessage(): string

DemolitionCard adalah kelas turunan dari SkillCard yang digunakan untuk menghancurkan bangunan pada properti milik lawan. Atribut targetPropertyId menyimpan ID properti yang dipilih sebagai target. Method setTargetProperty() digunakan untuk menentukan target sebelum

kartu dieksekusi, sedangkan `getTargetPropertyId()` digunakan untuk membaca target tersebut. Method `execute()` melakukan validasi lengkap, mulai dari memastikan target sudah dipilih, `GameMaster` tersedia, `Board` tersedia, properti target ditemukan, properti bukan milik sendiri, status properti adalah `OWNED`, properti bertipe `StreetProperty`, dan properti tersebut memiliki bangunan. Jika seluruh validasi lolos, semua bangunan pada properti target dijual/dihapus melalui `sellAllBuildings()`, lalu kartu ditandai sudah digunakan dengan `markUsed()`.

Relasi kelas:

- `DemolitionCard` adalah child class dari `SkillCard`.
- `DemolitionCard` berelasi dengan `GameState` untuk mendapatkan `GameMaster`.
- `DemolitionCard` berelasi dengan `GameMaster` untuk mengakses state dan board.
- `DemolitionCard` berelasi dengan `Board` untuk mencari properti berdasarkan ID.
- `DemolitionCard` berelasi dengan `Property` sebagai target umum.
- `DemolitionCard` menggunakan `StreetProperty` karena hanya properti street yang memiliki bangunan untuk dihancurkan.
- `DemolitionCard` berelasi dengan `Player` untuk memastikan kartu tidak digunakan pada properti sendiri.

2.7.3.7. Kelas `FreeFromJailCard`

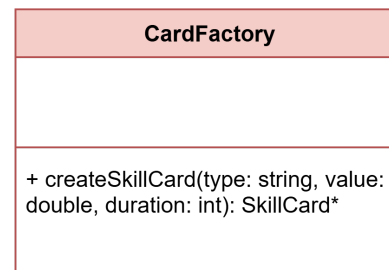
FreeFromJailCard
<pre> + FreeFromJailCard() + FreeFromJailCard(type: string, description: string, used: bool) + ~FreeFromJailCard() + execute(p: Player, gs: GameState): void + successMessage(): string </pre>

FreeFromJailCard adalah kelas turunan dari SkillCard yang digunakan untuk membebaskan pemain dari penjara. Kelas ini tidak memiliki atribut tambahan karena efeknya langsung diterapkan pada status pemain. Method execute() memeriksa terlebih dahulu apakah pemain sedang berada dalam status JAILED. Jika pemain tidak berada di penjara, method akan melempar exception. Jika valid, pemain dibebaskan dengan memanggil p.releaseFromJail(), lalu kartu ditandai sudah digunakan dengan markUsed(). Method successMessage() memberikan pesan bahwa kartu berhasil digunakan.

Relasi kelas:

- FreeFromJailCard adalah child class dari SkillCard.
- FreeFromJailCard berelasi dengan Player karena mengubah status pemain dari penjara menjadi bebas.
- FreeFromJailCard dapat dibuat oleh CardFactory.
- FreeFromJailCard dimasukkan ke skill deck oleh DeckFactory.
- GameState diterima oleh execute(), tetapi pada implementasi ini tidak digunakan langsung.

2.8. Kelas CardFactory



CardFactory adalah kelas utilitas yang bertanggung jawab membuat objek SkillCard berdasarkan string tipe kartu. Kelas ini tidak memiliki atribut karena hanya menyediakan method static. Method createSkillCard() menerima nama tipe kartu, nilai tambahan, dan durasi, lalu mengembalikan objek skill card yang sesuai. Misalnya, tipe MoveCard akan menghasilkan objek MoveCard, tipe DiscountCard akan menghasilkan objek DiscountCard, dan seterusnya. Jika tipe kartu tidak dikenali, method mengembalikan nullptr. Kelas ini terutama berguna saat proses load data, ketika kartu yang tersimpan dalam file perlu dikonversi kembali menjadi objek C++.

Relasi kelas:

- CardFactory membuat objek turunan SkillCard.
- CardFactory berelasi dengan MoveCard, DiscountCard, ShieldCard, TeleportCard, LassoCard, DemolitionCard, dan FreeFromJailCard.
- CardFactory kemungkinan digunakan oleh SaveLoadManager ketika melakukan load kartu pemain.
- Objek hasil factory dikembalikan sebagai pointer SkillCard* agar dapat dipakai secara polymorphic.

2.9. Kelas CardDeck

CardDeck
- drawPile: vector<T*> - discardPile: vector<T*>
+ CardDeck() + ~CardDeck() + pushToDrawPile(card: T*): void + setCards(cards: vector<T*>): void + draw(): T* + discard(card: T*): void + shuffle(): void + reshuffle(): void + isEmpty(): bool + drawPileSize(): size_t + discardPileSize(): size_t + getDrawPile(): vector<T*> + getDiscardPile(): vector<T*> + setCardsNoShuffle(cards: vector<T*>): void

CardDeck<T> adalah template class yang merepresentasikan kumpulan kartu dalam bentuk deck. Atribut drawPile menyimpan kartu yang masih dapat diambil, sedangkan discardPile menyimpan kartu yang sudah digunakan atau dibuang. Method draw() mengambil kartu dari drawPile; jika drawPile kosong, method ini akan mencoba melakukan reshuffle() dari discardPile. Method discard() memasukkan kartu ke

tumpukan discard. Method `shuffle()` mengacak `drawPile`, sedangkan `reshuffle()` memindahkan seluruh isi `discardPile` ke `drawPile` lalu mengacaknya. Kelas ini tidak memiliki ownership terhadap pointer kartu, sehingga proses delete kartu tetap harus dilakukan di luar deck, misalnya melalui helper pada `DeckFactory`.

Relasi kelas:

- `CardDeck<Card>` digunakan untuk menyimpan deck Kesempatan dan Dana Umum.
- `CardDeck<SkillCard>` digunakan untuk menyimpan deck kartu kemampuan.
- `DeckFactory` bertanggung jawab membuat dan mengisi objek `CardDeck`.
- `GameMaster` atau sistem permainan mengambil kartu dari deck menggunakan `draw()`.
- Kartu yang selesai digunakan dapat dikembalikan ke deck melalui `discard()`.

2.10. Kelas DeckFactory

DeckFactory
<pre> + createChanceDeck(): CardDeck<Card>* + createCommunityDeck(): CardDeck<Card>* + createSkillDeck(): CardDeck<SkillCard>* + deleteDeckCards(deck: CardDeck<T>*): void </pre>

`DeckFactory` adalah kelas utilitas yang bertanggung jawab membuat deck awal permainan. Method `createChanceDeck()` membuat deck Kesempatan yang berisi kartu seperti `GoToJailCard`, `MoveBackThreeCard`, dan `NearestStationCard`. Method `createCommunityDeck()` membuat deck Dana Umum yang berisi `BirthdayCard`, `DoctorFeeCard`, dan `ElectionCard`. Method `createSkillDeck()` membuat deck kartu kemampuan dengan jumlah kartu tertentu untuk setiap jenis skill card, seperti `MoveCard`, `DiscountCard`, `ShieldCard`, `TeleportCard`, `LassoCard`,

DemolitionCard, dan FreeFromJailCard. Method deleteDeckCards() menjadi helper generic untuk menghapus kartu yang masih berada di drawPile dan discardPile.

Relasi kelas:

- DeckFactory membuat objek CardDeck<Card> untuk deck Kesempatan dan Dana Umum.
- DeckFactory membuat objek CardDeck<SkillCard> untuk deck kartu kemampuan.
- DeckFactory berelasi dengan seluruh kelas kartu yang dimasukkan ke dalam deck.
- DeckFactory menggunakan polymorphism karena kartu berbeda disimpan sebagai pointer ke Card atau SkillCard.
- GameState atau GameMaster dapat menggunakan hasil dari DeckFactory sebagai deck utama permainan.

2.11. Kelas Property

Property
- id: int - code: string - name: string - colorGroup: string - purchasePrice: int - mortgageValue: int - status: PropertyStatus - ownerId: string
+ Property() + Property(id: int, code: string, name: string, colorGroup: string, purchasePrice: int, mortgageValue: int, ownerId: string) + ~Property() + getId(): int + getCode(): string + getName(): string + getColorGroup(): string + getPurchasePrice(): int + getMortgageValue(): int + getStatus(): PropertyStatus + getOwnerId(): string + setOwner(newOwnerId: string): void + clearOwner(): void + setStatus(newStatus: PropertyStatus): void + moneyToString(value: int): string + statusToString(status: PropertyStatus): string + printLine(out: ostream, c: char): void + printRow(out: ostream, leftText: string, rightText: string): void + printFullRow(out: ostream, text: string): void + printCenteredRow(out: ostream, text: string): void + printBasicInfo(out: ostream): void + printFooterStatus(out: ostream): void + calculateRentPrice(diceRoll: int, ownerSameColorCount: int, monopoly: bool): int + calculateSellPrice(): int + cetakAkta(): string + formattingTXT(): string + printList(): string

Property adalah kelas abstrak yang menjadi dasar untuk seluruh jenis properti dalam permainan. Kelas ini menyimpan informasi umum yang pasti dimiliki oleh setiap properti, seperti identitas properti, kode petak, nama, kelompok warna, harga beli, nilai gadai, status kepemilikan, dan pemilik. Method getter digunakan untuk mengakses informasi properti, sedangkan setOwner, clearOwner, dan setStatus digunakan untuk mengubah kondisi kepemilikan properti. Selain itu, kelas ini juga menyediakan method bantu untuk memformat uang, status, dan tampilan akta. Karena perhitungan sewa, harga jual, tampilan akta, format penyimpanan, dan tampilan daftar berbeda untuk setiap jenis properti, maka method seperti calculateRentPrice, calculateSellPrice, cetakAkta, formattingTXT, dan printList dibuat sebagai method virtual murni yang wajib diimplementasikan oleh kelas turunannya.

Relasi kelas:

- Property menjadi parent class dari StreetProperty, RailroadProperty, dan UtilityProperty.
- Property dapat disimpan secara polymorphic menggunakan pointer atau unique_ptr<Property>.
- Player dapat memiliki daftar properti dalam bentuk pointer ke Property.
- GameMaster atau sistem permainan dapat memanggil method calculateRentPrice() tanpa perlu mengetahui jenis properti spesifik.
- PropertyFactory menghasilkan objek turunan properti, tetapi menyimpannya sebagai Property.

2.11.1. Kelas StreetProperty

StreetProperty
- houseUpgCost: int - hotelUpgCost: int - rentPrice: map<int, int> - buildingCount: int - hasHotel: bool - festivalMultiplier: int - festivalDuration: int - MAX_HOUSES: int - MAX_FESTIVAL_MULTIPLIER: int - FESTIVAL_DURATION: int
+ StreetProperty() + StreetProperty(id: int, code: string, name: string, colorGroup: string, purchasePrice: int, mortgageValue: int, ownerId: string, houseUpgCost: int, hotelUpgCost: int, rentPrice: map<int, int>, buildingCount: int, hasHotel: bool, festivalMultiplier: int, festivalDuration: int) + ~StreetProperty() + getHouseUpgCost(): int + getHotelUpgCost(): int + getRentPrice(): map<int, int> + getBuildingCount(): int + gethasHotel(): bool + getFestivalMultiplier(): int + getFestivalDuration(): int + setFestivalMultiplier(multiplier: int): void + setFestivalDuration(duration: int): void + buildHouse(): void + upgToHotel(): void + sellAllBuildings(): int + downgradeToHouse(): void + removeBuilding(): void + resetBuildings(): void + resetFestival(): void + activateFestival(newMultiplier: int): void + decrementFestivalDuration(): void + computeBaseRent(monopoly: bool): int + applyFestivalMultiplier(rent: int): int + calculateRentPrice(diceRoll: int, ownerSameColorCount: int, monopoly: bool): int + calculateSellPrice(): int + cetakAkta(): string + formattingTXT(): string + printList(): string

StreetProperty adalah kelas turunan dari Property yang merepresentasikan properti jalan atau tanah biasa. Kelas ini memiliki tanggung jawab untuk mengelola bangunan di atas properti, yaitu rumah dan hotel, serta menghitung sewa berdasarkan jumlah bangunan, status monopoli, dan efek festival. Atribut seperti houseUpgCost, hotelUpgCost, buildingCount, dan hasHotel digunakan untuk mengatur pembangunan properti. Atribut festivalMultiplier dan festivalDuration digunakan untuk menyimpan efek festival yang dapat menaikkan harga sewa untuk sementara waktu. Method seperti buildHouse, upgToHotel, sellAllBuildings, downgradeToHouse, dan removeBuilding digunakan untuk mengatur perubahan bangunan, sedangkan calculateRentPrice bertugas menghitung sewa akhir berdasarkan kondisi properti. Kelas ini juga mengimplementasikan method abstrak dari Property, sehingga objek StreetProperty dapat diperlakukan sebagai Property oleh kelas lain.

Relasi kelas:

- StreetProperty adalah child class dari Property.
- StreetProperty mewarisi atribut umum properti dari Property, seperti ID, nama, harga beli, status, dan pemilik.
- GameMaster dapat menggunakan calculateRentPrice() ketika pemain mendarat di properti jalan.
- Player dapat memiliki objek StreetProperty sebagai bagian dari daftar properti miliknya.
- PropertyFactory membuat objek StreetProperty berdasarkan data konfigurasi bertipe STREET.
- Board atau tile properti dapat menghubungkan posisi petak dengan objek StreetProperty.

2.11.2. Kelas RailroadProperty

RailroadProperty
- rentFactor: map<int, int>
+ RailroadProperty() + RailroadProperty(id: int, code: string, name: string, colorGroup: string, purchasePrice: int, mortgageValue: int, ownerId: string, rentFactor: map<int, int>) + ~RailroadProperty() + getRentFactor(): map<int, int> + calculateRentPrice(diceRoll: int, ownerSameColorCount: int, monopoly: bool): int + calculateSellPrice(): int + cetakAkta(): string + formattingTXT(): string + printList(): string

RailroadProperty adalah kelas turunan dari Property yang merepresentasikan properti stasiun atau railroad. Kelas ini lebih sederhana dibandingkan StreetProperty karena tidak memiliki rumah, hotel, maupun festival. Tanggung jawab utamanya adalah menyimpan faktor harga sewa berdasarkan jumlah railroad yang dimiliki oleh pemilik yang sama. Atribut rentFactor digunakan sebagai pemetaan antara jumlah railroad yang dimiliki dengan harga sewa yang harus dibayar pemain lain. Method calculateRentPrice menggunakan jumlah railroad milik pemilik untuk menentukan harga sewa. Selain itu, kelas ini juga mengimplementasikan method untuk menghitung harga jual, mencetak akta, membuat format teks penyimpanan, dan menampilkan properti dalam bentuk daftar.

Relasi kelas:

- RailroadProperty adalah child class dari Property.
- RailroadProperty mewarisi data dasar seperti kode, nama, harga beli, nilai gadai, status, dan pemilik dari Property.
- GameMaster perlu mengetahui jumlah railroad yang dimiliki pemilik untuk menghitung sewa melalui calculateRentPrice().
- Player dapat memiliki satu atau lebih objek RailroadProperty.
- PropertyFactory membuat objek RailroadProperty dari data konfigurasi bertipe RAILROAD.
- Board menghubungkan petak railroad dengan objek RailroadProperty.

2.11.3. Kelas UtilityProperty

UtilityProperty
- rentPrice: map<int, int>
+ UtilityProperty() + UtilityProperty(id: int, code: string, name: string, colorGroup: string, purchasePrice: int, mortgageValue: int, ownerId: string, rentPrice: map<int, int>) + ~UtilityProperty() + getRentPrice(): map<int, int> + calculateRentPrice(diceRoll: int, ownerSameColorCount: int, monopoly: bool): int + calculateSellPrice(): int + cetakAkta(): string + formattingTXT(): string + printList(): string

UtilityProperty adalah kelas turunan dari Property yang merepresentasikan properti utilitas. Berbeda dari railroad dan street, harga sewa utility bergantung pada hasil lemparan dadu pemain dan jumlah utility yang dimiliki oleh pemilik yang sama. Atribut rentPrice menyimpan faktor pengali sewa berdasarkan jumlah utility yang dimiliki. Method calculateRentPrice menggunakan nilai diceRoll dan ownerSameColorCount untuk menentukan total sewa. Kelas ini juga bertanggung jawab untuk mencetak akta, menghitung harga jual, membuat format teks untuk

penyimpanan, dan menampilkan properti dalam bentuk daftar. Karena kelas ini mengimplementasikan method abstrak dari Property, objek UtilityProperty tetap dapat diperlakukan secara polymorphic sebagai objek Property.

Relasi kelas:

- UtilityProperty adalah child class dari Property.
- UtilityProperty mewarisi atribut dasar dari Property, seperti ID, kode, nama, harga beli, nilai gadai, status, dan pemilik.
- GameMaster memberikan informasi hasil dadu dan jumlah utility milik pemilik saat menghitung sewa.
- Player dapat memiliki objek UtilityProperty.
- PropertyFactory membuat objek UtilityProperty dari data konfigurasi bertipe UTILITY.
- Board menghubungkan petak utility dengan objek UtilityProperty.

2.12. Kelas PropertyFactory

PropertyFactory
<pre>+ createProperties(propertyDataList: vector<PropertyData>, railroadRentMap: map<int, int>, utilityFactorMap: map<int, int>): vector<unique_ptr<Property>></pre>

PropertyFactory adalah kelas utilitas yang bertanggung jawab untuk membuat objek properti berdasarkan data konfigurasi. Kelas ini tidak memiliki atribut karena seluruh proses pembuatan properti dilakukan melalui method static createProperties. Method tersebut menerima daftar PropertyData, membaca tipe properti dari setiap data, lalu membuat objek yang sesuai, yaitu StreetProperty, RailroadProperty, atau UtilityProperty. Hasil akhirnya dikembalikan dalam bentuk vector<unique_ptr<Property>>, sehingga seluruh objek properti dapat disimpan

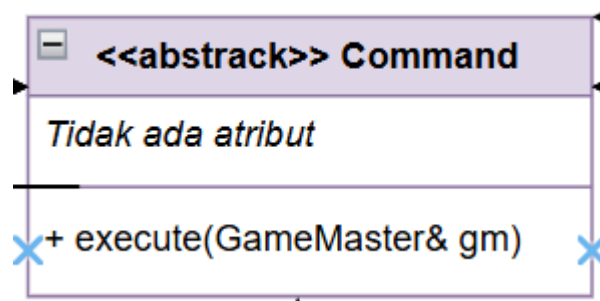
secara polymorphic sebagai objek Property. Dengan adanya factory ini, proses pembentukan objek properti tidak dicampur langsung dengan proses pembacaan konfigurasi maupun logika permainan.

Relasi kelas:

- PropertyFactory membuat objek StreetProperty, RailroadProperty, dan UtilityProperty.
- PropertyFactory mengembalikan hasil dalam bentuk `unique_ptr<Property>`.
- PropertyFactory bergantung pada PropertyData sebagai data mentah hasil konfigurasi.
- ConfigLoader berperan menyediakan data konfigurasi yang kemudian diproses oleh PropertyFactory.
- GameState, Board, atau sistem permainan dapat memakai hasil dari PropertyFactory sebagai daftar properti permainan.

2.13. Kelas Command

2.13.1. kelas <<abstract>> command



Kelas <<abstract>> command bertanggung jawab sebagai base class dari command-command yang diimplementasikan di bawah. Void `execute()` bersifat pure virtual. Desain ini secara eksplisit mengikuti Command Pattern, di mana setiap aksi pemain dienkapsulasi sebagai objek command tersendiri sehingga memisahkan antara siapa yang meminta aksi dari siapa yang mengeksekusinya.

2.13.2. Kelas AturDaduCommand

AturDaduCommand
- Dice* dice - Player* activePlayer - int val1 - int val2
+ execute(GameMaster&)

Memiliki fungsi yang identik dengan LemparDaduCommand, namun digunakan khusus untuk mode testing dengan nilai dadu yang ditentukan secara manual.

List Atribut:

- Dice* dice
- Player* activePlayer
- int val1, int val2

List Method:

- void execute(GameMaster& gm): Melakukan validasi apakah val1 dan val2 berada di rentang 1-6. Jika valid, memanggil dice->setManual(val1, val2) dan menjalankan logika pergerakan serta trigger petak yang sama dengan LemparDaduCommand. Jika tidak valid, melempar InvalidDiceValueException.

Jenis Relasi: Inheritance dari Command; Aggregation ke Dice dan Player.

2.13.3. Kelas BangunCommand

BangunCommand
- player: Player* - bank: Bank* - logger: TransactionLogger* - turn: int - guiTargetProp: StreetProperty* - guilsHotel: bool
+ BangunCommand(Player*, Bank*, TransactionLogger*, int): void + BangunCommand(Player*, StreetProperty*, bool, TransactionLogger*, int): void

Mengelola logika pembangunan rumah atau upgrade ke hotel pada properti bertipe jalan (Street). Menangani validasi aturan monopoli (kepemilikan satu grup warna) dan keseimbangan jumlah bangunan antar properti dalam grup yang sama.

List Atribut:

- Player* player: Pemain yang ingin membangun.
- Bank* bank: Referensi bank untuk transaksi biaya pembangunan.
- TransactionLogger* logger: Mencatat aktivitas pembangunan.
- int turn: Nomor giliran saat ini.
- StreetProperty* guiTargetProp: Properti target (khusus mode GUI).
- bool guilsHotel: Flag apakah yang dibangun adalah hotel.

List Metode:

- `execute(GameMaster& gm)`: Menjalankan logika pembangunan baik secara interaktif (CLI) maupun langsung (GUI).
- `collectEligibleGroups(GameMaster& gm)`: Mencari daftar grup warna yang sudah dimiliki sepenuhnya oleh pemain dan siap dibangun.
- `canBuildOnTile(...)`: Validasi apakah satu petak boleh ditambah rumah (berdasarkan selisih jumlah bangunan dengan petak lain di grup yang sama).
- `buildHouse(...)`: Mengurangi saldo pemain dan menambah jumlah rumah di properti.
- `upgradeToHotel(...)`: Mengurangi saldo pemain dan mengubah status rumah menjadi hotel.

Jenis Relasi: Inheritance dari `Command`; Aggregation ke `Player`, `Bank`, `TransactionLogger`, dan `StreetProperty`.

2.13.4. Kelas `BankruptCommand`

BankruptCommand
- <code>gm: GameMaster&</code> - <code>state: GameState&</code> - <code>from: Player*</code> - <code>to: Player*</code> - <code>debt: int</code> - <code>sellOverMortgage: bool</code> - <code>propToSell: int</code>
+ <code>BankruptCommand(GameMaster&, GameState&, Player*, Player*, int, bool, int): void</code> + <code>execute(GameMaster&): void</code>

Menangani proses kebangkrutan pemain, termasuk likuidasi aset (properti/bangunan) dan penyerahan sisa kekayaan kepada kreditor (pemain lain atau bank).

List Atribut:

- GameMaster& gm: Referensi pengatur permainan.
- GameState& state: Referensi status permainan untuk menghapus pemain.
- Player* from: Pemain yang bangkrut.
- Player* to: Penerima aset (bisa nullptr jika bank).
- int debt: Jumlah hutang yang tidak terbayar.
- bool sellOverMortgage: Opsi pilihan likuidasi aset.
- int propToSell: Properti spesifik yang akan dilikuidasi

List Metode:

- execute(GameMaster& gm): Memindahkan semua properti pemain yang bangkrut ke penerima, menghapus pemain dari daftar aktif, dan membersihkan status properti.

Jenis Relasi: Inheritance dari Command; Aggregation ke Player, GameState, dan GameMaster.

2.13.5. Kelas BayarPajakCommand

BayarPajakCommand
- p: Player* - tile: TaxTile* - bank: Bank* - logger: TransactionLogger* - taxConfig: TaxConfig - turnNumber: int - gm: GameMaster*
+ BayarPajakCommand(Player*, TaxTile*, Bank*, TransactionLogger*, TaxConfig, int): void + execute(GameMaster&): void + handlePPHChoice(int): void + getPphFlatAmount(): int + getPphPctAmount(): int + getPlayer(): Player* - handlePPH(): void - handlePBM(): void

Mengelola kewajiban pembayaran pajak (PPH atau PBM) saat pemain mendarat di petak pajak, termasuk memberikan pilihan skema pembayaran kepada pemain.

List Atribut:

- Player* p: Pemain pembayar pajak.
- TaxTile* tile: Lokasi petak pajak.
- Bank* bank: Penerima uang pajak.
- TransactionLogger* logger: Mencatat transaksi pajak.
- TaxConfig taxConfig: Konfigurasi nilai pajak dari file eksternal.
- int turnNumber: Nomor giliran saat ini.

List Metode:

- execute(GameMaster& gm): Menentukan jenis pajak berdasarkan petak dan mengeksekusi penagihan.
- handlePPHChoice(int choice): Memproses pilihan pemain antara bayar flat (tetap) atau persentase dari total kekayaan.
- handlePPH() & handlePBM(): Logika internal penghitungan nilai pajak sesuai kategori.

Jenis Relasi: Inheritance dari Command; Aggregation ke Player, TaxTile, Bank, dan TransactionLogger.

2.13.6. Kelas BayarSewaCommand

BayarSewaCommand
- Player* tenant - Player* owner - PropertyTile* prop
+ execute(GameMaster&)

Bertanggung jawab menghitung tarif sewa yang harus dibayarkan pemain dan memproses transfer dana antar pemain secara otomatis.

List Atribut:

- Player* tenant
- Player* owner
- PropertyTile* prop

List Method:

- void execute(GameMaster& gm): Mengambil tarif sewa dasar dari prop, lalu mengalikannya dengan status monopoli, level bangunan, dan multiplier Festival jika aktif. Jika saldo tenant cukup, melakukan transfer ke owner. Jika tidak cukup, melempar InsufficientFundsException untuk memicu alur kebangkrutan.

Jenis Relasi: Inheritance dari Command; Aggregation ke Player (tenant & owner) dan PropertyTile.

2.13.7. Kelas BeliCommand

BeliCommand
- Player* p - PropertyTile* prop - Bank* bank
+ execute(GameMaster&)

Mengelola proses pembelian properti saat pemain mendarat di petak berstatus BANK, termasuk menangani peralihan otomatis untuk Railroad/Utility.

List Atribut:

- Player* p
- PropertyTile* prop
- Bank* bank

List Method:

- void execute(GameMaster& gm): Mengecek jenis properti. Jika Railroad/Utility, kepemilikan langsung berpindah secara otomatis. Jika Street, menampilkan dialog konfirmasi beli. Jika pemain setuju dan uang cukup, memanggil

bank->receiveFromPlayer() untuk memproses transaksi. Jika pemain menolak atau uang tidak cukup, otomatis memicu LelangCommand.

Jenis Relasi: Inheritance dari Command; Aggregation ke Player, PropertyTile, dan Bank.

2.13.8. Kelas Card Command

CardCommand
- p: Player* - deck: CardDeck<Card>* - gs: GameState* - logger: TransactionLogger* - turnNumber: int - deckLabel: string - lastDescription: string
+ CardCommand(Player*, CardDeck<Card>*, GameState*, TransactionLogger*, int, const string&): void + execute(GameMaster&): void + getLastDescription(): const string& + getDeckLabel(): const string&

Menangani pengambilan kartu (Dana Umum/DNU atau Kesempatan/KSP) dari tumpukan kartu dan mengeksekusi efek kartu yang ditarik.

List Atribut:

- Player* p: Pemain yang menarik kartu.

- CardDeck<Card>* deck: Tumpukan kartu yang digunakan.
- GameState* gs: Status permainan untuk memproses efek kartu (seperti pindah petak).
- TransactionLogger* logger: Mencatat kartu yang didapat.
- std::string deckLabel: Nama tumpukan (DNU atau KSP).
- std::string lastDescription: Teks penjelasan isi kartu terakhir.

List Metode:

- execute(GameMaster& gm): Mengambil satu kartu dari deck secara acak, memanggil fungsi action() pada kartu tersebut, dan memasukkan kembali kartu ke bawah tumpukan.

Jenis Relasi: Inheritance dari Command; Aggregation ke Player, CardDeck, GameState, dan TransactionLogger.

2.13.9. Kelas Tebus Command

TebusCommand
- Player* p - PropertyTile* prop - Bank* bank
+ execute(GameMaster&)

Bertanggung jawab memulihkan properti yang sedang digadaikan agar kembali ke status OWNED dengan membayar harga beli penuh.

List Atribut:

- Player* p
- PropertyTile* prop

- Bank* bank

List Method:

- void execute(GameMaster& gm): Memvalidasi apakah status properti adalah MORTGAGED. Jika pemain memiliki uang sebesar harga beli penuh, memanggil bank->receiveFromPlayer() dan mengubah status properti kembali menjadi OWNED. Melempar InsufficientFundsException jika uang tidak cukup.

Jenis Relasi: Inheritance dari Command; Aggregation ke Player, PropertyTile, dan Bank.

2.13.10. Kelas LempardaduCommand

LempardaduCommand
- Dice* dice - Player* activePlayer
+ execute(GameMaster&)

Bertanggung jawab memanggil logika acak dari objek Dice, memperbarui posisi bidak pemain di GameMaster, dan memvalidasi aturan double. Jika pemain mendapatkan *double* tiga kali, kelas ini bertanggung jawab memindahkan pemain ke penjara.

List atribut:

- Dice* dice
- Player* activePlayer

List method:

- void execute(): Memanggil method rollRandom() milik object dice dan menghitung counter double untuk validasi penjara. Jika tidak double 3 kali, menggerakkan bidak pemain sebanyak dice->getTotal() secara sirkular di atas papan (melewati petak GO akan memanggil Bank untuk membayar gaji). Lalu memanggil event pada petak tempat bidak mendarat (misal: tile->onLanded()), yang nantinya akan men-trigger perintah otomatis lain seperti Beli, Sewa, atau Pajak. Relasi Inheritance dari Command, Aggregation ke Dice dan Player.

2.13.11. Kelas LelangCommand

LelangCommand
- AuctionManager* am - PropertyTile* prop - vector<Player*> participants
+ execute(GameMaster&)

Mengelola sistem pelelangan properti secara interaktif antar pemain yang masih aktif di dalam permainan.

List Atribut:

- AuctionManager* am
- PropertyTile* prop
- std::vector<Player*> participants

List Method:

- void execute(GameMaster& gm): Menjalankan rotasi penawaran harga (bid) antar pemain. Melakukan validasi bid melalui objek am. Jika lelang berakhir, memproses perpindahan aset ke pemenang dan memotong saldo pemenang sesuai angka bid terakhir.

Jenis Relasi: Inheritance dari Command; Aggregation ke AuctionManager, PropertyTile, dan Player.

2.13.12. Kelas JualBangunanCommand

JualBangunanCommand
- player: Player* - prop: StreetProperty* - turn: int
+ JualBangunanCommand(Player*, StreetProperty*, int): void + execute(GameMaster&): void

Menangani penjualan rumah atau hotel kembali ke bank saat pemain membutuhkan dana tunai secara mendesak atau ingin mengosongkan lahan sebelum digadaikan.

List Atribut:

- Player* player: Pemain penjual bangunan.

- StreetProperty* prop: Properti yang bangunannya akan dijual.
- int turn: Nomor giliran saat ini.

List Metode:

- execute(GameMaster& gm): Memvalidasi keberadaan bangunan, mengurangi jumlah rumah/hotel di properti, dan memberikan uang kepada pemain sebesar setengah dari harga beli bangunan semula.

Jenis Relasi: Inheritance dari Command; Aggregation ke Player dan StreetProperty.

2.13.13. Kelas GadaiCommand

GadaiCommand
- Player* p - PropertyTile* prop - Bank* bank
+ execute(GameMaster&)

Bertanggung jawab mengubah status properti menjadi digadaikan (MORTGAGED) dan memberikan dana gadai dari Bank kepada pemain.

List Atribut:

- Player* p
- PropertyTile* prop
- Bank* bank

List Method:

- `void execute(GameMaster& gm)`: Memvalidasi kepemilikan dan ketiadaan bangunan pada color group terkait. Jika valid, mengubah status properti menjadi MORTGAGED dan memanggil `bank->payPlayer()` untuk memberikan uang nilai gadai kepada pemain. Jika ada bangunan, melempar `PropertyHasBuildingsException`.

Jenis Relasi: Inheritance dari `Command`; Aggregation ke `Player`, `PropertyTile`, dan `Bank`.

2.13.14. Kelas `FestivalCommand`

FestivalCommand
- p: <code>Player*</code> - logger: <code>TransactionLogger*</code> - turnNumber: <code>int</code>
+ <code>FestivalCommand(Player*, TransactionLogger*, int): void</code> + <code>execute(GameMaster&): void</code> + <code>getEligibleStreets(): vector<StreetProperty*></code> + <code>executeWithProperty(StreetProperty*): void</code> + <code>getPlayer(): Player*</code> - <code>applyFestival(StreetProperty*): void</code>

Memungkinkan pemain untuk mengadakan festival di salah satu properti miliknya guna melipatgandakan nilai sewa (Rent) untuk periode tertentu.

List Atribut:

- `Player* p`: Pemain penyelenggara festival.

- TransactionLogger* logger: Mencatat penyelenggaraan festival.
- int turnNumber: Nomor giliran saat ini.

List Metode:

- execute(GameMaster& gm): Menjalankan pemilihan properti secara interaktif.
- getEligibleStreets(): Mengembalikan daftar properti yang sah untuk diadakan festival (biasanya yang tidak digadaikan).
- executeWithProperty(StreetProperty* selected): Menerapkan pengganda sewa pada properti yang dipilih pemain melalui GUI.

Jenis Relasi: Inheritance dari Command; Aggregation ke Player, TransactionLogger, dan StreetProperty.

2.13.15. Kelas CetakAktaCommand

CetakAktaCommand
- kodePetak: string
+ CetakAktaCommand(kodePetak: string) + execute(gm: GameMaster&): void + getOutput(gm: GameMaster&): string

CetakAktaCommand adalah kelas turunan dari Command yang bertanggung jawab untuk mencetak akta dari suatu properti berdasarkan kode petak yang diberikan. Atribut kodePetak menyimpan kode petak yang ingin dicari, dan nilainya diubah menjadi huruf kapital agar pencarian tidak bergantung pada format input pengguna. Method getOutput() mengambil GameState dari GameMaster, lalu mencari properti menggunakan getPropertyByCode(kodePetak). Jika kode kosong atau properti tidak ditemukan, method ini mengembalikan pesan error. Jika

properti ditemukan, method ini mengembalikan hasil cetakAkta() dari objek Property. Method execute() hanya mencetak hasil dari getOutput() ke terminal.

Relasi kelas:

- CetakAktaCommand adalah child class dari Command.
- CetakAktaCommand menggunakan GameMaster sebagai akses utama ke kondisi permainan.
- CetakAktaCommand menggunakan GameState melalui gm.getState().
- CetakAktaCommand berelasi dengan Property karena mengambil dan mencetak akta properti.
- Property dapat berupa StreetProperty, RailroadProperty, atau UtilityProperty, sehingga cetakAkta() bekerja secara polymorphic.

2.13.16. Kelas CetakPropertiCommand

CetakPropertiCommand
+ CetakPropertiCommand() + execute(gm: GameMaster&): void + getOutput(gm: GameMaster&): string

CetakPropertiCommand adalah kelas turunan dari Command yang bertanggung jawab untuk mencetak daftar properti milik pemain aktif. Kelas ini tidak memiliki atribut karena command ini selalu bekerja terhadap pemain yang sedang mendapat giliran saat ini. Method getOutput() mengambil pemain aktif melalui gm.getState().getCurrPlayer(). Jika tidak ada pemain aktif, method ini mengembalikan pesan "Tidak ada pemain aktif.". Jika pemain aktif tersedia, method ini mengembalikan hasil player->cetakProperti(). Method execute() mencetak output tersebut ke terminal.

Relasi kelas:

- CetakPropertiCommand adalah child class dari Command.
- CetakPropertiCommand menggunakan GameMaster untuk mengakses GameState.
- CetakPropertiCommand menggunakan GameState untuk mengambil pemain aktif.
- CetakPropertiCommand berelasi dengan Player karena daftar properti dicetak dari method cetakProperti().
- Player berelasi dengan Property karena daftar properti pemain berasal dari properti yang dimilikinya.

2.13.17. Kelas GunakanKartuKemampuanCommand

GunakanKartuKemampuanCommand
- AuctionManager* am - PropertyTile* prop - vector<Player*> participants
+ execute(GameMaster&)

GunakanKartuKemampuanCommand adalah kelas turunan dari Command yang bertanggung jawab untuk menggunakan kartu kemampuan dari tangan pemain aktif. Atribut cardIndex menyimpan indeks kartu yang dipilih dalam bentuk 0-based index. Method execute() mengambil GameState dari GameMaster, lalu mengambil pemain aktif menggunakan getCurrPlayer(). Setelah itu, command memvalidasi apakah pemain tersedia, apakah pemain memiliki kartu kemampuan, apakah indeks kartu valid, dan apakah kartu yang dipilih tidak bernilai nullptr. Jika seluruh validasi lolos, command memanggil gm.useSkillCard(player, selectedCard, gs) agar eksekusi kartu dilakukan melalui mekanisme utama GameMaster, bukan langsung dari command.

Relasi kelas:

- GunakanKartuKemampuanCommand adalah child class dari Command.
- GunakanKartuKemampuanCommand menggunakan GameMaster untuk mengakses state dan menjalankan kartu melalui useSkillCard().
- GunakanKartuKemampuanCommand menggunakan GameState untuk mengambil pemain aktif.
- GunakanKartuKemampuanCommand berelasi dengan Player karena kartu diambil dari hand milik pemain.
- GunakanKartuKemampuanCommand berelasi dengan SkillCard karena kartu yang digunakan adalah kartu kemampuan.
- GunakanKartuKemampuanCommand berelasi dengan exception seperti SkillCardOverflowNotFoundException, SkillCardInvalidActionException, InvalidSkillCardIndexException, dan SkillCardDiscardFailedException.
- Eksekusi detail kartu tetap diserahkan ke GameMaster, sehingga command hanya berperan sebagai validasi input dan penghubung antara input pengguna dengan logika permainan.

2.13.18. Kelas DropKartuKemampuanCommand

DropKartuKemampuanCommand
- cardIndex: int
+ DropKartuKemampuanCommand(cardIndex: int) + execute(gm: GameMaster&): void

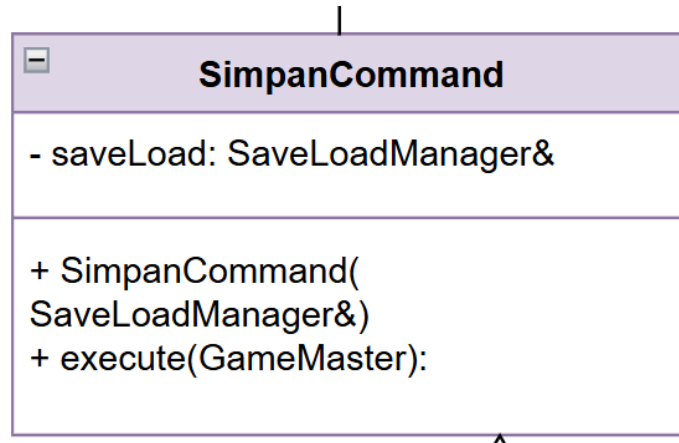
DropKartuKemampuanCommand adalah kelas turunan dari Command yang bertanggung jawab untuk membuang kartu kemampuan dari tangan pemain. Atribut cardIndex menyimpan indeks kartu yang ingin dibuang dalam bentuk 0-based index. Method execute() mengambil

GameState dari GameMaster, lalu menentukan pemain yang harus membuang kartu. Jika ada pending drop player, pemain tersebut digunakan. Jika tidak, command memakai pemain aktif. Command ini kemudian memvalidasi apakah pemain tersedia, apakah jumlah kartu memang lebih dari 3, dan apakah indeks kartu valid. Jika valid, kartu dihapus dari tangan pemain menggunakan `discardSkillCard(cardIndex)`, lalu kartu tersebut dimasukkan ke discard pile dari skillDeck jika deck tersedia. Setelah itu, command mencatat aksi menggunakan `gm.log()`, membersihkan status pending drop, dan mengembalikan fase permainan ke `PLAYER_TURN` jika sebelumnya berada pada fase `AWAITING_DROP_SKILL_CARD`.

Relasi kelas:

3. DropKartuKemampuanCommand adalah child class dari Command.
4. DropKartuKemampuanCommand menggunakan GameMaster untuk mengakses state dan mencatat log aksi.
5. DropKartuKemampuanCommand menggunakan GameState untuk mengambil pending drop player, pemain aktif, fase permainan, dan skill deck.
6. DropKartuKemampuanCommand berelasi dengan Player karena menghapus kartu dari tangan pemain.
7. DropKartuKemampuanCommand berelasi dengan SkillCard karena kartu yang dibuang bertipe SkillCard.
8. DropKartuKemampuanCommand berelasi dengan CardDeck<SkillCard> karena kartu yang dibuang dimasukkan ke discard pile.
9. DropKartuKemampuanCommand berelasi dengan exception seperti SkillCardOverflowNotFoundException,
10. SkillCardDropNotRequiredException, InvalidSkillCardIndexException, dan SkillCardDiscardFailedException.

10.1.1. Kelas SimpanCommand



SimpanCommand bertanggung jawab mengeksekusi logika inti perintah SIMPAN, mengecek keberadaan file dan menyimpan seluruh GameState ke file via SaveLoadManager. Input nama file dan konfirmasi timpa sudah ditangani lebih dulu oleh SavePopup di GameScreen sebelum command ini dibuat dan di-push ke antrian.

Atribut:

- - saveLoad: SaveLoadManager&

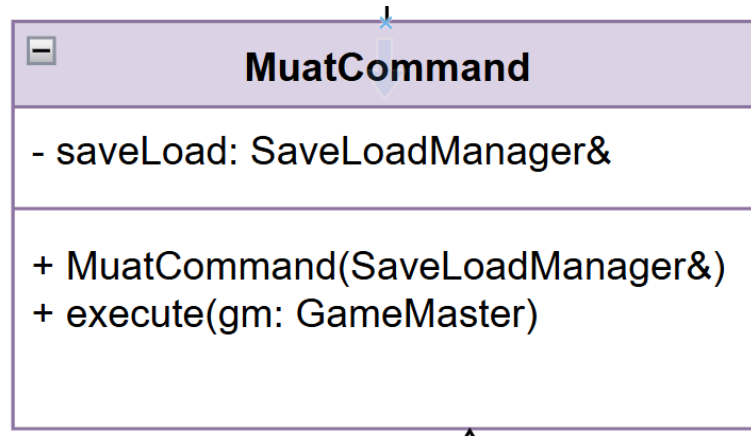
Metode:

- execute(gm: GameMaster) mengambil GameState dari GameMaster, memanggil SaveLoadManager::fileExists() untuk cek file, lalu SaveLoadManager::save() untuk menulis state ke file. SaveLoadManager::load() untuk membaca file dan mengisi ulang GameState di GameMaster.

Relasi:

- Agregasi ke SaveLoadManager (disimpan sebagai atribut referensi). Dependency ke GameMaster (hanya muncul di parameter execute()).

10.1.2. Kelas MuatCommand



MuatCommand bertanggung jawab mengeksekusi logika inti perintah MUAT, mengecek keberadaan file dan merestorasi GameState dari file via SaveLoadManager. Input nama file sudah ditangani lebih dulu oleh panel load di MainMenuScreen sebelum command ini dibuat dan di-push ke antrian. Validasi bahwa MUAT hanya boleh dipanggil sebelum game dimulai ada di GameMaster.

Atribut:

- - saveLoad: SaveLoadManager&

Metode:

- execute(gm: GameMaster) memanggil SaveLoadManager::fileExists() untuk validasi, lalu SaveLoadManager::load() untuk membaca file dan mengisi ulang GameState di GameMaster.

Relasi:

- Agregasi ke SaveLoadManager (disimpan sebagai atribut referensi). Dependency ke GameMaster (hanya muncul di parameter execute()).

10.2. Utils

10.2.1. Kelas SaveLoadManager

SaveLoadManager
<i>Tidak ada atribut</i>
<pre> + SaveLoadManager() + save(state: GameState, filename: string): void + load(filename: string, state: GameState): void + fileExists(filename: string): bool + savePlayers(out: ofstream, state: GameState): void + saveProperties(out: ofstream, state: GameState): void + saveDeck(out: ofstream, state: GameState): void + saveLogs(out: ofstream, state: GameState): void + loadPlayers(in: ifstream, state: GameState): void + loadProperties(in: ifstream, state: GameState): void + loadDeck(in: ifstream, state: GameState): void + loadLogs(in: ifstream, state: GameState): void - tokenize(line: string): vector<string> </pre>

SaveLoadManager bertanggung jawab membaca dan menulis seluruh state permainan dari/ke file eksternal berformat .txt. Ia adalah satu-satunya kelas yang boleh menyentuh file save, sesuai prinsip Single Responsibility dan Layered Architecture sebagai Data Access Layer untuk fitur save/load.

Atribut:

- SaveLoadManager tidak memiliki atribut, ia murni kelas utilitas stateless. Semua data yang dibutuhkan diambil dari parameter yang diterima tiap method, bukan disimpan sebagai state internal.

Metode:

- Public utama:
 - save() mengorkestrasi penulisan seluruh state ke file dengan memanggil keempat save helper secara berurutan. load() mengorkestrasi pembacaan file dan mengisi ulang objek GameState yang diberikan. fileExists() mengecek apakah file save dengan nama tertentu sudah ada, dipakai sebelum save untuk konfirmasi timpa dan sebelum load untuk validasi.
- Save Helpers:
 - savePlayers() menulis state tiap pemain (uang, posisi, status, kartu tangan). saveProperties() menulis state tiap properti (pemilik, status, festival, bangunan). saveDeck() menulis sisa kartu di deck kemampuan. saveLogs() menulis seluruh entri log dari TransactionLogger.
- Load Helpers:
 - Masing-masing load*() adalah kebalikan dari pasangan save-nya. membaca bagian file yang sesuai dan merestorasi state ke objek GameState.
- tokenize() adalah helper private yang memecah satu baris teks menjadi token, dipakai oleh semua load helper untuk mem-parsing format file.

Relasi:

- SaveLoadManager berelasi dependency dengan GameState, Player, Board, dan TransactionLogger, semuanya hanya muncul sebagai parameter method, tidak disimpan sebagai atribut. Ia membaca data dari objek-objek tersebut saat save, dan mengisi ulang objek-objek tersebut saat load.
- GameState adalah pintu masuk utama, dari satu objek ini SaveLoadManager bisa mengakses semua sub-state yang dibutuhkan (pemain, papan, deck, log). TransactionLogger diakses khusus oleh saveLogs() dan loadLogs() untuk membaca dan merestorasi riwayat transaksi.
- Di sisi GUI, MainMenuScreen yang memicu load (via triggerLoad()) dan GameScreen yang memicu save (via SavePopup), namun keduanya tidak memanggil SaveLoadManager langsung, melainkan lewat Command yang diteruskan ke GUIManager dan dieksekusi oleh GameMaster.

10.2.2. Kelas ConfigLoader

ConfigLoader
- basePath: string - openFile(string): ifstream - parseLine(string): vector<string>
+ ConfigLoader(string) + loadProperties(): vector<PropertyData> + loadRailroad(): map<int, int> + loadUtility(): map<int, int> + loadTax(): TaxConfig + loadSpecial(): SpecialConfig + loadMisc(): MiscConfig + loadActions(): vector<ActionData>

Kelas yang membaca dan mem-parsing file konfigurasi statis game dari disk menjadi objek data terstruktur, bekerja sekali saat inisialisasi.

Atribut:

- basePath: string menyimpan lokasi direktori file konfigurasi.

Metode:

- openFile() membuka file berdasarkan nama relatif ke basePath. parseLine() memecah baris teks menjadi token. Setiap load*() membaca satu file spesifik dan mengembalikan objek data yang sesuai.

Relasi:

- Dependency ke PropertyData, ActionData, TaxConfig, SpecialConfig, dan MiscConfig — ia memproduksi objek-objek tersebut sebagai return value lalu menyerahkannya ke pemanggil.

10.2.3. Kelas PropertyData

PropertyData
- id: int - code, name, type, color: string - price, mortgage, upgHouse, upgHotel: double - rentLevels: vector<double>
+ getId(): int + getCode(): string + getName(): string + getType(): string + getColor(): string + getPrice(): double + getMortgage(): double + getUpgHouse(): double + getUpgHotel(): double + getRentLevel(int level): double + getRentLevels(): vector<double> + setPrice(double): void + setMortgage(double): void + setUpgHouse(double): void + setUpgHotel(double): void

Kelas yang membaca dan mem-parsing file konfigurasi statis game dari disk menjadi objek data terstruktur, bekerja sekali saat inisialisasi.

Atribut:

- basePath: string menyimpan lokasi direktori file konfigurasi.

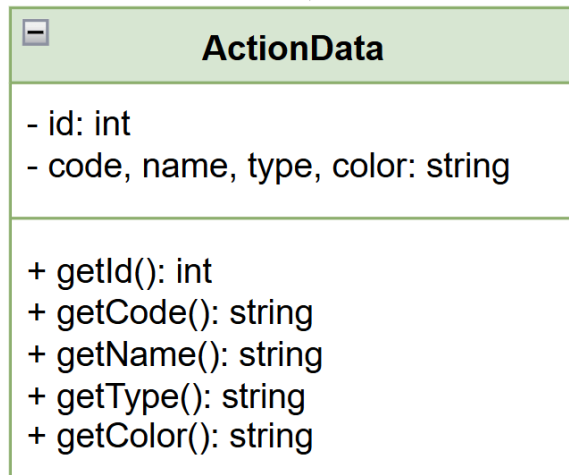
Metode:

- openFile() membuka file berdasarkan nama relatif ke basePath. parseLine() memecah baris teks menjadi token. Setiap load*() membaca satu file spesifik dan mengembalikan objek data yang sesuai.

Relasi:

- Dependency ke PropertyData, ActionData, TaxConfig, SpecialConfig, dan MiscConfig — ia memproduksi objek-objek tersebut sebagai return value lalu menyerahkannya ke pemanggil.

10.2.4. Kelas ActionData



Kelas yang merepresentasikan data satu kartu aksi.

Atribut:

- `id: int, code, name, type, color: string.`

Metode:

- Hanya getter karena data kartu aksi bersifat read-only setelah di-load.

Relasi:

- Diproduksi oleh ConfigLoader dan digunakan oleh sistem kartu game.

10.2.5. Kelas TaxConfig

TaxConfig
- pphFlat, pphPercentage, pbmFlat: double
+ getPphFlat(): double + getPphPercentage(): double + getPbmFlat(): double + setPphFlat(double): void + setPphPercentage(double): void + setPbmFlat(double): void + calculatePph(double income): double ← tanggung jawab kalkulasi + calculatePbm(double income): double

Kelas yang menyimpan konfigurasi dan logika kalkulasi pajak.

Atribut:

- pphFlat, pphPercentage, pbmFlat: double.

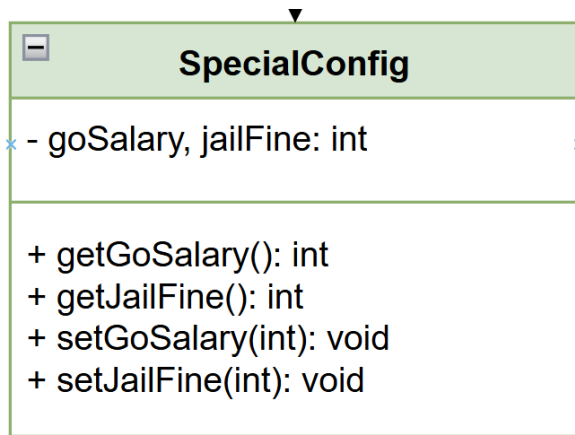
Metode:

- Getter dan setter untuk tiap atribut. calculatePph() dan calculatePbm() memusatkan logika kalkulasi pajak di dalam kelas pemilik datanya.

Relasi:

- Diproduksi oleh ConfigLoader dan digunakan oleh sistem pajak game.

10.2.6. Kelas SpecialConfig



Kelas yang menyimpan konstanta aturan khusus permainan.

Atribut:

- `goSalary, jailFine: int.`

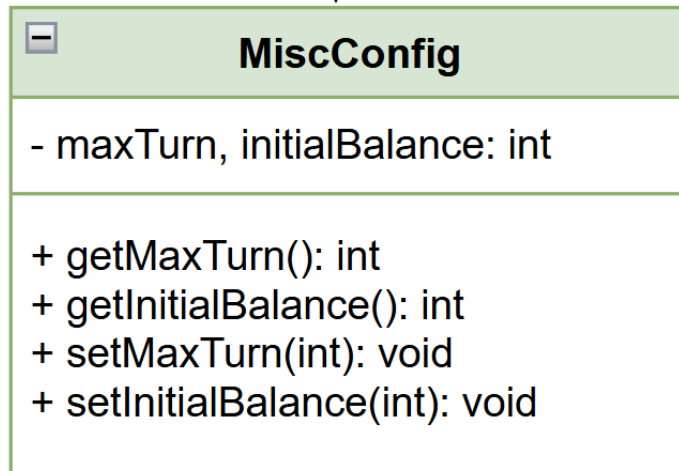
Metode:

- Getter dan setter untuk tiap atribut.

Relasi:

- Diproduksi oleh `ConfigLoader` dan digunakan oleh sistem aturan game, khususnya saat pemain melewati GO atau keluar dari penjara.

10.2.7. Kelas **MiscConfig**



Kelas yang menyimpan konfigurasi umum permainan.

Atribut:

- `maxTurn, initialBalance: int.`

Metode:

- Getter dan setter untuk tiap atribut.

Relasi:

- Diproduksi oleh `ConfigLoader` dan digunakan saat inisialisasi game untuk menentukan batas giliran dan uang awal tiap pemain.

10.2.8. Kelas `TransactionLogger`

TransactionLogger
- logs: vector<LogEntry>
+ TransactionLogger() + ~TransactionLogger() + printBoard(GameMaster& gm) + log(int turn, string user, string action, string detail) + getAll(): const vector& + getLast(int n): vector + clear() + serialize(): string + deserialize(string data) + size(): int

TransactionLogger bertanggung jawab mencatat semua kejadian signifikan selama permainan ke dalam log terstruktur. Ia bekerja di latar belakang secara otomatis tanpa perintah khusus dari pemain.

Atribut:

- logs: vector<LogEntry> adalah satu-satunya atribut, menyimpan seluruh riwayat kejadian selama sesi berlangsung di memori.

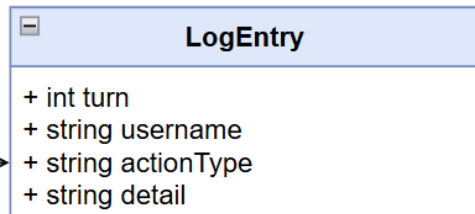
Metode:

- addLog() dipanggil oleh sistem setiap kali kejadian signifikan terjadi. getLogs() mengembalikan seluruh log (dipakai saat save dan CETAK_LOG tanpa argumen). getLastLogs(n) mengembalikan n entri terakhir (dipakai saat CETAK_LOG dengan argumen). clear() dipakai saat reset atau new game.

Relasi:

- TransactionLogger diintegrasikan oleh GameMaster (atau GameState), disimpan sebagai atribut dan di-pass ke seluruh Command yang membutuhkan pencatatan. Setiap Command yang mengeksekusi aksi signifikan memanggil addLog() setelah aksinya selesai. Saat SaveLoadManager menyimpan game, ia membaca getLogs() untuk menulis <STATE_LOG> ke file, dan saat load ia merestorasi log ke TransactionLogger via addLog() per entri.

10.2.9. Kelas LogEntry



LogEntry adalah data class yang merepresentasikan satu entri log, satu kejadian signifikan yang tercatat selama permainan. Ia tidak memiliki behavior aktif, tugasnya murni menyimpan data satu baris log secara terstruktur.

Atribut:

- turn: int
- username: string
- actionType: string
- detail: string

Metode:

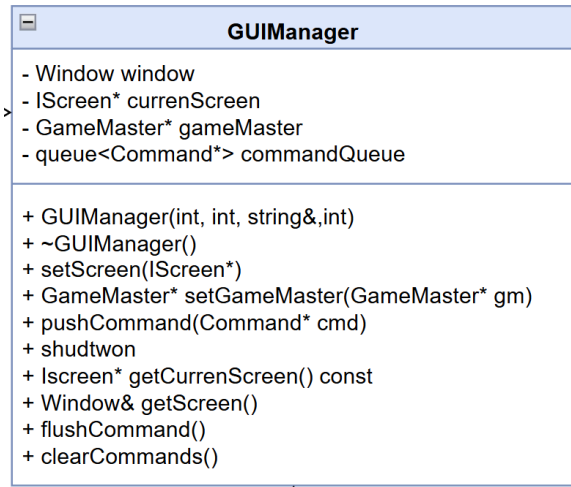
- Hanya getter untuk tiap atribut karena LogEntry bersifat read-only setelah dibuat — sebuah log yang sudah tercatat tidak boleh diubah. Tidak ada setter.

Relasi:

- LogEntry berelasi komposisi dengan TransactionLogger, ia hanya hidup di dalam `vector<LogEntry> logs` milik TransactionLogger dan tidak punya makna di luar konteks tersebut. TransactionLogger yang menciptakan dan mengelola seluruh lifecycle-nya via `addLog()`.

10.3. GUI

10.3.1. Kelas GUIManager



GUIManager adalah pusat kendali seluruh aplikasi, ia mengelola jendela, layar aktif, game master, dan antrian command. Semua alur besar aplikasi melewatinya.

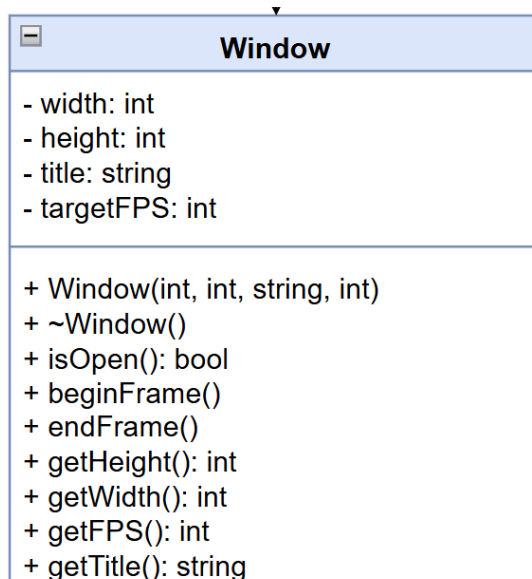
Atribut & Tanggung Jawab:

- Window window dimiliki langsung (bukan pointer) — GUIManager yang bertanggung jawab penuh atas lifetime jendela. IScreen* currentScreen menunjuk layar yang sedang aktif; setScreen() mengganti layar dengan memanggil onExit() layar lama dan onEnter() layar baru.
- GameMaster* gameMaster adalah referensi opsional ke logika inti permainan — di-set via setGameMaster() dan bisa diakses layar manapun yang memegang referensi GUIManager. Ini yang menghubungkan lapisan GUI dengan lapisan game logic.
- queue<Command*> commandQueue mengimplementasikan command pattern — layar tidak langsung mengeksekusi aksi game, melainkan mendorong Command* ke antrian via pushCommand(). GUIManager mengeksekusi semua command di awal frame berikutnya via flushCommands(), menjaga pemisahan antara input/render dan mutasi state game.

Relasi:

- GUIManager berelasi komposisi dengan Window (dimiliki langsung) dan agregasi dengan IScreen*, GameMaster*, serta Command* dalam antrian. Ia adalah penghubung antara tiga dunia: tampilan (IScreen), jendela (Window), dan logika game (GameMaster + Command).
- Alur frame-nya: flushCommands() eksekusi command → currentScreen->update(dt) → currentScreen->render(window), diulang selama window.isOpen() bernilai true.

10.3.2. Kelas Window



Window adalah kelas wrapper tipis di atas raylib yang mengabstraksi manajemen jendela aplikasi. Tanggung jawabnya sederhana: membuka jendela, mengatur FPS target, dan menyediakan interface untuk game loop.

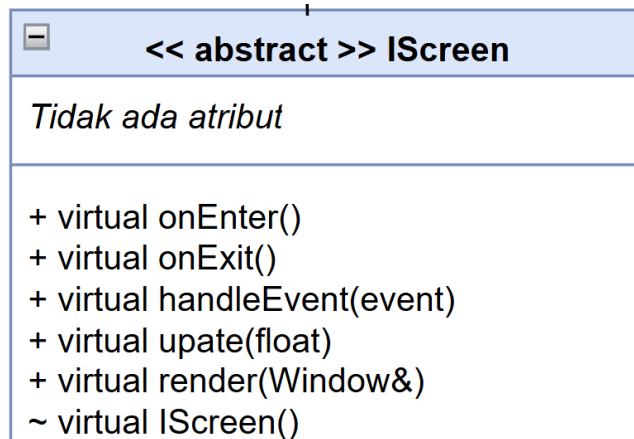
Atribut & Tanggung Jawab:

- Menyimpan width, height, targetFPS, dan title sebagai konfigurasi jendela. Method beginFrame() dan endFrame() mengapit logika render tiap frame, isOpen() digunakan sebagai kondisi game loop utama, dan getter sisanya dipakai oleh kelas lain yang butuh info dimensi layar.

Relasi:

- Window dipakai oleh hampir semua kelas dalam sistem GUI. GUIManager memilikinya secara agregasi (bukan pointer, langsung sebagai member), dan setiap IScreen::render(Window& window) menerima referensi ke Window untuk bisa menggambar ke layar. Jadi Window adalah shared resource yang mengalir dari GUIManager ke semua layar.

10.3.3. Kelas IScreen



IScreen adalah antarmuka abstrak (interface) yang mendefinisikan kontrak perilaku setiap layar dalam aplikasi. Ia tidak menyimpan state apapun, hanya menyediakan virtual method yang harus diimplementasikan kelas turunan.

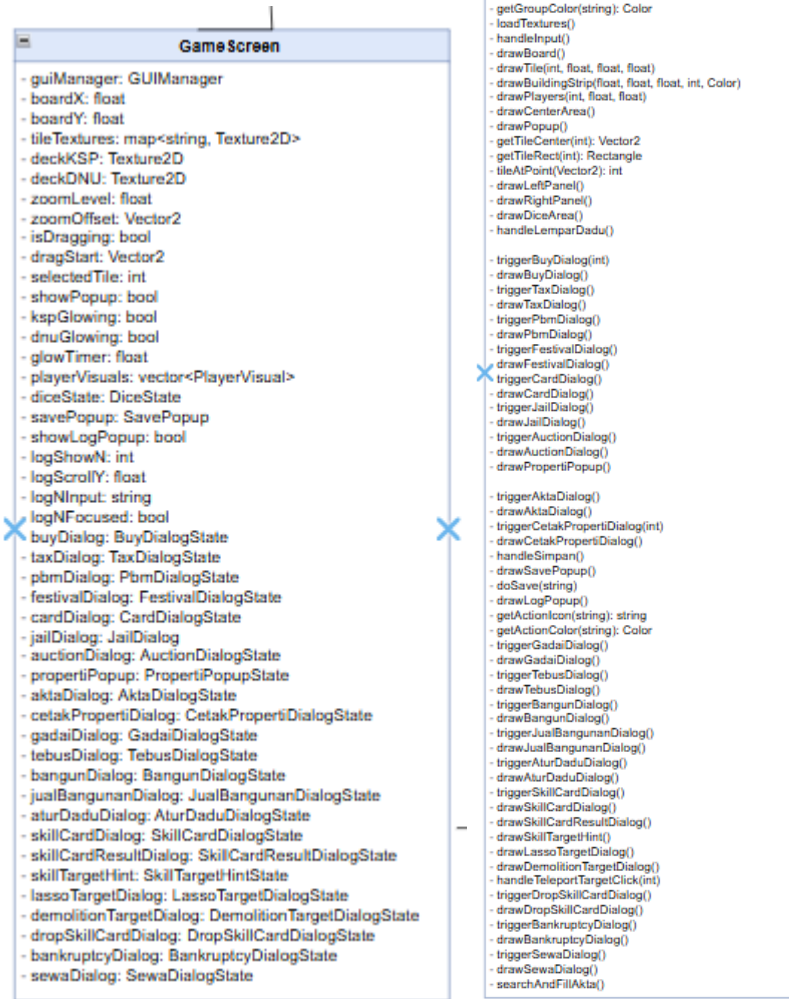
Method & Tanggung Jawab:

- Empat virtual method membentuk siklus hidup sebuah layar: onEnter() dipanggil saat layar pertama kali diaktifkan, onExit() saat layar ditinggalkan, update(dt) tiap frame untuk logika, dan render(Window&) tiap frame untuk menggambar.

Relasi:

- IScreen diimplementasikan oleh tiga layar konkret: MainMenuScreen, GameScreen, dan WinScreen. GUIManager menyimpan pointer IScreen*, artinya ia tidak peduli layar apa yang aktif, cukup panggil method virtual-nya. Ini adalah penerapan polimorfisme yang memungkinkan GUIManager berpindah layar tanpa coupling ke implementasi spesifik.

10.3.4. Kelas GameScreen



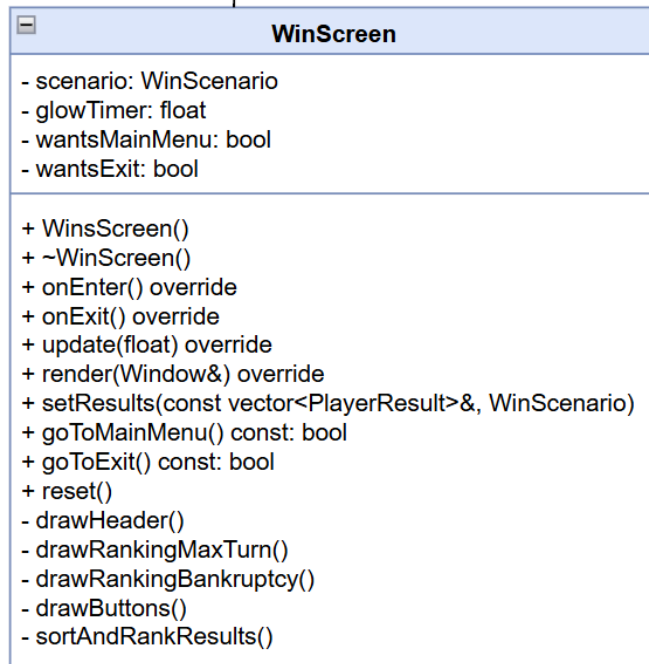
GameScreen adalah kelas utama yang mengelola seluruh tampilan dan logika layar permainan Monopoly yang sedang berjalan. Ia bertindak sebagai orchestrator, menerima input pemain, memperbarui state, lalu merender semua elemen visual ke layar.

GameScreen menyimpan referensi ke GUIManager (untuk navigasi antar layar), boardX/Y dan tileTextures untuk merender papan, serta deckKSP untuk kartu. Ia juga menyimpan state interaksi seperti isDragging, dragStart, selectedTile, dan berbagai glowTimer untuk efek visual.

GameScreen mengagregasi banyak kelas secara langsung:

- Dialog States (BuyDialogState, TaxDialogState, JailDialog, AuctionDialogState, FestivalDialogState, CardDialogState, PbmDialogState, GadaDialogState, TebusDialogState, AksiDialogState, SkillCardDialogState, dll.): setiap kelas ini menyimpan data yang dibutuhkan satu popup/dialog tertentu. GameScreen yang memutuskan kapan tiap dialog ditampilkan via method trigger*().
- PlayerVisual: menyimpan posisi animasi tiap pemain di papan (currentTileIdx, targetTileIdx).
- DiceState: menyimpan nilai dadu, status rolling, animasi, dan apakah double.
- SavePopup: mengelola UI popup untuk aksi save.
- TileDef: definisi tiap tile di papan (kode, sisi, sudut).

10.3.5. Kelas WinScreen



WinScreen adalah layar yang ditampilkan saat permainan berakhir, baik karena batas giliran habis maupun karena hanya tersisa satu pemain. Tanggung jawabnya adalah merekap hasil permainan dan menampilkan ranking pemain, lalu mengarahkan pemain ke menu utama atau keluar.

WinScreen menyimpan results berupa vector of PlayerResult, setiap objek berisi data akhir satu pemain: username, money, propertyCount, cardCount, bankrupt, rank, isWinner, dan color. Data ini diisi dari luar sebelum layar ditampilkan via setResults().

WinScenario menentukan mode tampilan: MAX_TURN menampilkan ranking lengkap semua pemain, sedangkan BANKRUPTCY hanya menampilkan satu pemenang yang tersisa. glowTimer dipakai untuk efek animasi visual.

Untuk navigasi, WinScreen hanya menyimpan dua flag boolean: wantsMainMenu dan wantsExit. GUIManager akan mengecek flag ini tiap frame untuk memutuskan perpindahan layar.

WinScreen berelasi agregasi dengan PlayerResult: ia memiliki dan mengelola koleksi objek tersebut. `sortAndRankResults()` mengurutkan vector ini berdasarkan urutan prioritas: uang → properti → kartu,

WinScreen diimplementasikan sebagai IScreen, sehingga masuk ke dalam siklus layar yang dikelola GUIManager bersama GameScreen dan MainMenuScreen.

10.3.6. Kelas MainMenuScreen

MainMenuScreen
- selectedPlayerCount: int - focusedInput: int - glowTimer: float - nameBuffers: string[4] - isBot: bool[4] - selectedDifficulty: COMDifficulty - saveFileExists: bool - saveFiles: vector<string> - selectedSaveIdx: int - showLoadPanel: bool - lastClickTime: double - readyToStart: bool - setup: GameSetup - errorMsg: string - errorTimer: float
+ onEnter() override + onExit() override + update(float) override + render(Window&) override + bool isReadyToStart() const + const GameSetup& getSetup() const + resetReady() - handleInput() - drawTitle() - drawLeftPanel() - drawRightPanel() - drawLoadPanel - drawError() - validateInputs(): bool - scanSaveFiles() - triggerLoad(const string&) - applySetup()

MainMenuScreen adalah layar pembuka yang bertanggung jawab mengumpulkan konfigurasi permainan baru atau memuat game yang tersimpan, lalu mengemas hasilnya ke dalam GameSetup untuk diteruskan ke GameScreen.

MainMenuScreen dibagi menjadi beberapa kelompok state:

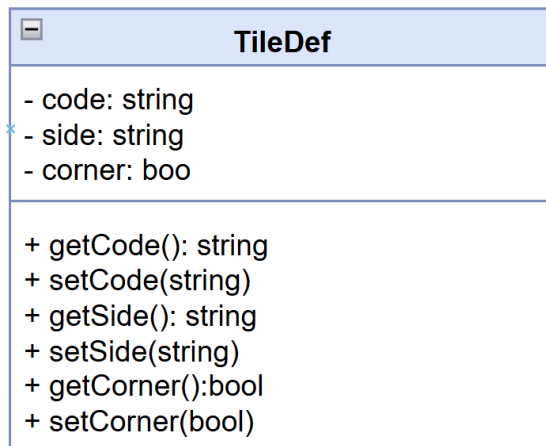
- State UI: selectedPlayerCount menentukan berapa pemain aktif, focusedInput melacak input field mana yang sedang aktif, nameBuffers[4] menyimpan ketikan nama tiap slot, dan glowTimer untuk efek visual.
- State COM: isBot[4] adalah toggle per slot apakah pemain itu human atau bot, dan selectedDifficulty adalah difficulty global yang berlaku untuk semua slot COM.
- State Save/Load: saveFiles menyimpan daftar file save yang ditemukan, selectedSaveIdx melacak pilihan user, showLoadPanel mengontrol visibilitas panel load, dan lastClickTime untuk deteksi double-click.
- Output: readyToStart adalah flag yang dicek GUIManager, dan setup adalah objek GameSetup yang berisi semua konfigurasi hasil input user.

MainMenuScreen berelasi agregasi dengan GameSetup, ia yang membangun dan mengisi objek ini via applySetup() setelah semua input divalidasi.

GameSetup sendiri menyimpan: playerCount, names[4], isBot[4], botDifficulty (yang bertipe COMDifficulty dari kelas ComputerPlayer), saveFile, dan isLoadGame. Objek ini lalu dibaca GameScreen untuk menginisialisasi seluruh permainan.

MainMenuScreen juga bergantung pada COMDifficulty dari ComputerPlayer.hpp untuk mengisi pilihan difficulty bot.

10.3.7. Kelas TileDef



TileDef adalah kelas data sederhana yang merepresentasikan definisi satu tile di papan permainan. Tanggung jawabnya murni menyimpan metadata tile, bukan logika game.

Atribut & Tanggung Jawab:

- code (kode unik tile), side (sisi papan mana tile berada), dan corner (apakah tile ini sudut papan). Getter dan setter-nya straightforward. Tidak ada logika kompleks di sini, TileDef hanya sebagai value object yang dibaca kelas lain.

Relasi:

- GameScreen mengagregasi TileDef dalam bentuk array/collection untuk memetakan posisi visual tiap tile ke papan. Saat GameScreen merender papan (drawBoard()), ia membaca code dan side dari TileDef untuk menentukan tekstur dan orientasi yang tepat.

10.3.8. Kelas SavePopUp

SavePopUp
- visible: bool - confirmOverwrite: bool - resultVisible: bool - fileNameInput: string - resultMsg: string
+ isVisible(): bool + getFileName(): string + setVisible(b: bool): void + setResultMsg(s: string): void

SavePopup adalah kelas dialog state khusus untuk UI popup save game di dalam GameScreen. Tanggung jawabnya mengelola interaksi user saat menyimpan permainan, mulai dari input nama file hingga konfirmasi overwrite.

Atribut & Tanggung Jawab:

- visible mengontrol apakah popup ditampilkan. confirmOverwrite adalah flag saat user mencoba menyimpan ke nama file yang sudah ada. resultVisible dan resultMsg dipakai untuk menampilkan pesan sukses/gagal setelah proses save selesai. fileNameInput menyimpan nama file yang diketik user.

Relasi: SavePopup diintegrasikan oleh GameScreen. GameScreen memanggilnya lewat triggerSavePopup() dan drawSave(). Secara tidak langsung, SavePopup berinteraksi dengan SaveLoadManager, saat user konfirmasi save, GameScreen mengambil getFileName() dari SavePopup lalu meneruskannya ke SaveLoadManager::save().

10.3.9. Kelas TaxDialogState

TaxDialogState
- visible: bool - flatAmount: int - pctAmount: int - wealth: int - canAffordFlat: bool - taxAmtPct: int
+ isVisible(): bool + getFlatAmount(): int + getPctAmount(): int + setVisible(b: bool): void + setWealth(w: int): void

Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog pajak dan melakukan kalkulasi visual sebelum keputusan diambil oleh pemain.

List Atribut:

- bool visible: Menentukan apakah dialog pajak sedang ditampilkan.
- int flatAmount: Nilai nominal pajak tetap (Flat).
- int pctAmount: Persentase pajak yang akan dikenakan.
- int wealth: Total kekayaan pemain saat ini.
- int taxAmtPct: Hasil kalkulasi nominal pajak dari persentase kekayaan.
- bool canAffordFlat: Status apakah saldo cukup untuk bayar flat.
- bool canAffordPct: Status apakah saldo cukup untuk bayar persentase.

List Method:

- bool isVisible(): Mengembalikan status apakah dialog pajak saat ini sedang aktif dan harus ditampilkan di layar.
- int getFlatAmount(): Mengambil nilai nominal pajak tetap (flat) yang harus dibayar oleh pemain.
- int getPctAmount(): Mengambil nilai persentase pajak yang akan digunakan untuk menghitung potongan dari total kekayaan.
- void setVisible(bool b): Mengatur status visibilitas dialog; digunakan untuk memunculkan atau menutup popup pajak.
- void setWealth(int w): Memperbarui data total kekayaan pemain untuk menghitung estimasi nominal pajak persentase secara real-time pada UI.

Jenis Relasi: Composition dari GameScreen (karena merupakan nested class yang siklus hidupnya bergantung pada kelas induknya). Dependency terhadap BayarPajakCommand (saat data dari state ini digunakan untuk mengeksekusi perintah pembayaran).

10.3.10. Kelas BuyDialogState

BuyDialogState
- visible: bool - tileIdx: int - canAfford: bool
+ isVisible(): bool + getTileIdx(): int + canAffordPurchase(): bool + setVisible(b: bool): void

Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog pembelian properti dan memvalidasi apakah pemain mampu membeli properti di petak tersebut.

List Atribut:

- bool visible: Menentukan apakah dialog pembelian properti sedang aktif dan ditampilkan di layar.
- int tileIdx: Menyimpan indeks posisi petak (tile) papan yang sedang ditawarkan untuk dibeli.
- bool canAfford: Status validasi yang menunjukkan apakah saldo pemain mencukupi untuk harga beli properti tersebut.

List Metode:

- bool isVisible(): Mengembalikan status visibilitas dialog untuk menentukan apakah UI popup harus dirender.
- int getTileIdx(): Mengambil indeks petak yang sedang diproses dalam dialog pembelian.
- bool canAffordPurchase(): Mengembalikan nilai kebenaran (true/false) mengenai kemampuan finansial pemain untuk melakukan transaksi pembelian.
- void setVisible(bool b): Mengatur status visibilitas dialog; digunakan untuk memicu kemunculan popup saat pemain mendarat di properti kosong.

Jenis Relasi: * Composition dari GameScreen (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh GameScreen). Dependency terhadap BeliPropertyCommand (saat data dari state ini digunakan untuk mengeksekusi logika transaksi pembelian di Core).

10.3.11. Kelas JailDialog

JailDialog
- visible: bool - jailFine: int - canAffordFine: bool - jailTurnsLeft: int - forcedPay: bool
+ isVisible(): bool + getJailFine(): int + getTurnsLeft(): int + setVisible(b: bool): void + setForcedPay(b: bool): void

Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog penjara dan melacak opsi serta sisa durasi tahanan pemain sebelum mengambil keputusan untuk keluar.

List Atribut:

- bool visible: Menentukan apakah dialog penjara sedang aktif dan ditampilkan di layar.
- int jailFine: Nilai nominal denda yang harus dibayar pemain untuk keluar dari penjara secara instan.
- bool canAffordFine: Status validasi yang menunjukkan apakah saldo pemain mencukupi untuk membayar denda penjara.
- int jailTurnsLeft: Jumlah sisa giliran yang harus dilewati pemain di dalam penjara.
- bool forcedPay: Status yang menunjukkan apakah pemain dipaksa membayar denda (biasanya setelah 3 giliran gagal melempar dadu double).

List metode:

- `bool isVisible()`: Mengembalikan status visibilitas dialog untuk menentukan apakah UI popup penjara harus dirender.
- `int getJailFine()`: Mengambil nilai denda penjara yang berlaku saat ini.
- `int getTurnsLeft()`: Mengambil informasi sisa jumlah giliran pemain mendekam di penjara.
- `void setVisible(bool b)`: Mengatur status visibilitas dialog; digunakan untuk menampilkan popup saat pemain yang dipenjara memulai gilirannya.
- `void setForcedPay(bool b)`: Mengatur status kewajiban bayar denda secara paksa berdasarkan aturan batas waktu di penjara.

Jenis Relasi:

- Composition dari `GameScreen` (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Dependency terhadap `KeluarJailCommand` (saat data dari state ini digunakan sebagai parameter untuk mengeksekusi aksi keluar dari penjara, baik lewat denda maupun lemparan dadu).

10.3.12. Kelas AuctionDialogState

AuctionDialogState
- visible: bool - propertyName: string - propertyGroup: string - currentBid: int - highestBidder: string - isForcedBid: bool - bidInput: string
+ isVisible(): bool + getCurrentBid(): int + getBidInput(): string + setVisible(b: bool): void + setCurrentBid(b: int): void

Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog lelang properti dan melacak progres penawaran harga antar pemain hingga pemenang ditentukan.

List Atribut:

- bool visible: Menentukan apakah dialog lelang sedang aktif dan ditampilkan di layar.
- string propertyName: Nama properti yang sedang dilelang.
- string propertyGroup: Grup warna atau kategori dari properti yang sedang dilelang.
- int currentBid: Nilai penawaran harga tertinggi saat ini dalam sesi lelang.
- string highestBidder: Nama pemain yang memegang penawaran tertinggi saat ini.
- bool isForcedBid: Menentukan apakah penawaran tersebut bersifat wajib atau otomatis berdasarkan kondisi tertentu
- string bidInput: Menyimpan input teks sementara dari pemain saat mengetikkan jumlah penawaran baru.

List Metode:

- `bool isVisible()`: Mengembalikan status visibilitas dialog untuk menentukan apakah UI popup lelang harus dirender.
- `int getCurrentBid()`: Mengambil nilai penawaran tertinggi yang sedang berlangsung untuk ditampilkan di UI.
- `string getBidInput()`: Mengambil teks input penawaran yang sedang diketik oleh pemain untuk ditampilkan pada kotak input.
- `void setVisible(bool b)`: Mengatur status visibilitas dialog; digunakan untuk membuka popup lelang jika properti tidak dibeli oleh pemain pertama.
- `void setCurrentBid(int b)`: Memperbarui nilai penawaran tertinggi dengan nilai baru yang diajukan oleh peserta lelang.

Jenis relasi:

- Composition dari `GameScreen` (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Dependency terhadap `AuctionCommand` (saat data dari state ini digunakan untuk memproses transaksi perpindahan kepemilikan properti kepada pemenang lelang).

10.3.13. Kelas `FestivalDialogState`

FestivalDialogState
- visible: bool - scrollY: float - streets: vector<StreetProperty*> - hoveredIdx: int
+ isVisible(): bool + getScrollY(): float + setVisible(b: bool): void + setStreets(v): void

Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog pemilihan lokasi festival, mengelola daftar properti yang memenuhi syarat, serta menangani navigasi scrolling pada daftar tersebut.

List Atribut:

- bool visible: Menentukan apakah dialog pemilihan festival sedang aktif dan ditampilkan di layar.
- float scrollY: Menyimpan posisi vertikal gulir (scroll) pada daftar properti agar tampilan daftar tetap konsisten saat digeser.
- vector<StreetProperty*> streets: Daftar referensi ke objek properti bertipe jalan (street) yang dimiliki pemain dan memenuhi syarat untuk mengadakan festival.
- int hoveredIdx: Menyimpan indeks dari elemen daftar properti yang sedang disorot oleh cursor mouse pemain.

List Metode:

- bool isVisible(): Mengembalikan status visibilitas dialog untuk menentukan apakah antarmuka pemilihan festival harus dirender.
- float getScrollY(): Mengambil nilai posisi vertikal scroll saat ini untuk kalkulasi posisi elemen visual.
- void setVisible(bool b): Mengatur status visibilitas dialog; digunakan untuk memicu kemunculan popup saat pemain mendarat di petak festival.
- void setStreets(vector<StreetProperty*> v): Mengisi daftar properti yang tersedia untuk dipilih pemain berdasarkan kepemilikan yang sah.

Jenis Relasi:

- Composition dari GameScreen (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Dependency terhadap FestivalCommand (saat data properti yang dipilih dari state ini digunakan untuk menjalankan efek penggandaan sewa di Core).

10.3.14. **Kelas CardDialogState**

CardDialogState
- visible: bool - deckLabel: string - description: string
+ isVisible(): bool + getDeckLabel(): string + setVisible(b: bool): void + setDescription(s: string): void

Menyimpan status data sementara (*state*) yang diperlukan untuk merender UI dialog kartu (Dana Umum atau Kesempatan) dan menampilkan deskripsi efek kartu yang ditarik oleh pemain.

List Atribut:

- bool visible: Menentukan apakah dialog kartu sedang aktif dan ditampilkan di layar.
- string deckLabel: Label yang menunjukkan jenis dek kartu yang ditarik (misalnya: "Kesempatan" atau "Dana Umum").
- string description: Teks penjelasan mengenai efek atau instruksi yang tertera pada kartu yang didapatkan pemain.

List Metode:

- bool isVisible(): Mengembalikan status visibilitas dialog untuk menentukan apakah antarmuka kartu harus dirender.
- string getDeckLabel(): Mengambil teks label dek kartu untuk ditampilkan pada judul dialog.
- void setVisible(bool b): Mengatur status visibilitas dialog; digunakan untuk memicu kemunculan popup setelah pemain menarik kartu.

- void setDescription(string s): Mengisi atau memperbarui teks deskripsi kartu berdasarkan data kartu yang ditarik dari dek.

Jenis relasi:

- Composition dari GameScreen (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Dependency terhadap CardCommand (saat deskripsi hasil eksekusi perintah kartu dikirimkan ke state ini untuk ditampilkan secara visual).

10.3.15. Kelas PbmDialogState

PbmDialogState
- visible: bool - amount: int - balanceBefore: int - balanceAfter: int
+ isVisible(): bool + getAmount(): int + setVisible(b: bool): void + setAmount(a: int): void

Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog Pajak Bumi dan Bangunan (PBM) serta menampilkan perbandingan saldo pemain sebelum dan sesudah pembayaran pajak.

List Atribut:

- bool visible: Menentukan apakah dialog PBM sedang aktif dan ditampilkan di layar.
- int amount: Nilai nominal pajak bumi dan bangunan yang wajib dibayarkan.
- int balanceBefore: Menyimpan nilai saldo pemain sebelum melakukan pembayaran pajak untuk keperluan tampilan komparasi.
- int balanceAfter: Menyimpan nilai estimasi saldo pemain setelah dikurangi pembayaran pajak.

List Metode:

- `bool isVisible()`: Mengembalikan status visibilitas dialog untuk menentukan apakah antarmuka PBM harus dirender.
- `int getAmount()`: Mengambil nilai nominal pajak yang harus dibayar untuk ditampilkan pada dialog.
- `void setVisible(bool b)`: Mengatur status visibilitas dialog; digunakan untuk memicu kemunculan popup saat pemain mendarat di petak pajak PBM.
- `void setAmount(int a)`: Mengisi atau memperbarui nilai nominal pajak yang akan ditagihkan kepada pemain.

Jenis Relasi:

- Composition dari `GameScreen` (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Dependency terhadap `BayarPajakCommand` (saat data nominal dari state ini digunakan untuk mengeksekusi pengurangan saldo pemain di `Core`).

10.3.16. Kelas PropertiPopupState

PropertiPopupState
- visible: bool - scrollY: float
+ isVisible(): bool + getScrollY(): float + setScrollY(f: float): void

Menyimpan status data visual sementara untuk mengelola tampilan jendela sembul (popup) daftar properti, termasuk melacak posisi gulir (scrolling) saat pemain meninjau aset yang dimiliki.

List Atribut:

- bool visible: Menentukan apakah popup daftar properti sedang aktif dan ditampilkan di layar.
- float scrollY: Menyimpan posisi vertikal gulir (scroll) untuk memastikan daftar properti dapat digeser dengan mulus oleh pemain.

List metode:

- bool isVisible(): Mengembalikan status visibilitas untuk menentukan apakah unit UI popup properti harus dirender.
- float getScrollY(): Mengambil nilai koordinat Y dari posisi gulir saat ini untuk menentukan elemen daftar mana yang harus muncul di layar.
- void setScrollY(float f): Memperbarui posisi vertikal gulir berdasarkan input dari pemain (seperti penggunaan mouse wheel atau tarikan scroll bar).

Jenis relasi:

- Composition dari GameScreen (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Association ke Property (karena digunakan untuk menampilkan detail dari objek-objek properti yang terdaftar di dalam papan permainan).

10.3.17. Kelas AktaDialogState

AktaDialogState
- visible: bool - inputFocused: bool - scrollY: float - inputCode: string - content: string
+ isVisible(): bool + getInputCode(): string + setVisible(b: bool): void + setContent(s: string): void

Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog pencarian akta properti, menangani input kode properti dari pemain, serta mengelola tampilan konten informasi properti.

List Atribut:

- bool visible: Menentukan apakah dialog akta sedang aktif dan ditampilkan di layar.
- bool inputFocused: Menentukan apakah fokus keyboard saat ini berada pada kotak input kode (aktif untuk mengetik).
- float scrollY: Menyimpan posisi vertikal gulir (scroll) untuk menavigasi teks konten akta yang panjang.
- string inputCode: Menyimpan teks kode properti yang sedang diketik oleh pemain di kotak input.
- string content: Menyimpan teks lengkap informasi atau detail properti yang berhasil ditemukan berdasarkan kode.

List Metode:

- bool isVisible(): Mengembalikan status visibilitas dialog untuk menentukan apakah UI dialog akta harus dirender.
- string getInputCode(): Mengambil teks kode yang telah diinputkan oleh pemain untuk keperluan verifikasi atau pencarian.
- void setVisible(bool b): Mengatur status visibilitas dialog; digunakan untuk membuka atau menutup popup pencarian akta.
- void setContent(string s): Memperbarui teks informasi yang ditampilkan pada dialog setelah sistem menemukan data properti yang sesuai.

Jenis Relasi:

- Composition dari GameScreen (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Association ke Property (karena berfungsi untuk menampilkan detail data teknis dari objek properti tertentu berdasarkan input pengguna).

10.3.18. Kelas CetakPropertiDialogState

CetakPropertiDialogState
- visible: bool - scrollY: float - content: string - playerId: int
+ isVisible(): bool + getContent(): string + setVisible(b: bool): void + setPlayerId(i: int): void

Menyimpan status data sementara yang diperlukan untuk merender UI dialog daftar properti milik pemain tertentu, termasuk teks konten daftar dan manajemen navigasi gulir (scroll).

List Atribut:

- bool visible: Menentukan apakah dialog daftar properti sedang ditampilkan.
- float scrollY: Posisi vertikal gulir pada tampilan daftar properti.
- string content: Teks atau informasi detail mengenai daftar properti yang akan dicetak ke layar.
- int playerId: Indeks pemain yang daftar propertinya sedang dilihat.

List Metode:

- `bool isVisible()`: Mengembalikan status visibilitas dialog daftar properti.
- `string getContent()`: Mengambil konten teks informasi properti untuk ditampilkan pada UI.
- `void setVisible(bool b)`: Mengatur status visibilitas dialog daftar properti.
- `void setPlayerIdx(int i)`: Menentukan pemain mana yang datanya akan ditampilkan dalam dialog.

Jenis Relasi: Composition dari GameScreen. Association ke Player.

10.3.19. Kelas GadaiDialogState

GadaiDialogState
- visible: bool - scrollY: float - entries: vector<Entry> - hoveredIdx: int
+ isVisible(): bool + getEntries(): vector<Entry> + setVisible(b: bool): void + setHoveredIdx(i: int): void

Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog penggadaan aset, mengelola daftar properti yang dapat digadaikan, serta menangani interaksi kursor dan navigasi daftar.

List Atribut:

- `bool visible`: Menentukan apakah dialog penggadaan sedang aktif dan ditampilkan di layar.
- `float scrollY`: Menyimpan posisi vertikal gulir (scroll) untuk menavigasi daftar aset yang akan digadaikan.
- `vector<Entry> entries`: Daftar entri data properti milik pemain yang tersedia untuk diproses dalam mekanisme gadai.

- `int hoveredIdx`: Menyimpan indeks elemen dalam daftar yang sedang disorot oleh kursor pemain untuk memberikan efek visual hover.

List Metode:

- `bool isVisible()`: Mengembalikan status visibilitas dialog untuk menentukan apakah antarmuka penggadaian harus dirender.
- `vector<Entry> getEntries()`: Mengambil seluruh daftar entri properti yang dapat digadaikan untuk ditampilkan pada UI.
- `void setVisible(bool b)`: Mengatur status visibilitas dialog; digunakan untuk membuka atau menutup popup gadai saat pemain membutuhkan dana darurat.
- `void setHoveredIdx(int i)`: Memperbarui indeks entri yang sedang disorot oleh kursor berdasarkan posisi koordinat mouse pemain.

Jenis Relasi:

- Composition dari `GameScreen` (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Dependency terhadap `GadaiPropertyCommand` (saat data properti dari state ini dipilih oleh pemain untuk dieksekusi menjadi status mortgaged di Core).

10.3.20. Kelas `TebusDialogState`

TebusDialogState
- visible: bool - scrollY: float - entries: vector<Entry> - hoveredIdx: int
+ isVisible(): bool + getEntries(): vector<Entry> + setVisible(b: bool): void + setHoveredIdx(i: int): void

Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog penebusan properti, mengelola daftar aset yang sedang digadaikan, serta menangani interaksi navigasi dan pemilihan aset oleh pemain.

List Atribut:

- bool visible: Menentukan apakah dialog penebusan properti sedang aktif dan ditampilkan di layar.
- float scrollY: Menyimpan posisi vertikal gulir (scroll) untuk menavigasi daftar properti yang sedang dalam status digadaikan.
- vector<Entry> entries: Daftar entri data properti milik pemain yang saat ini berstatus MORTGAGED dan dapat ditebus kembali.
- int hoveredIdx: Menyimpan indeks elemen dalam daftar yang sedang disorot oleh kursor pemain untuk memberikan umpan balik visual (hover).

List Metode:

- bool isVisible(): Mengembalikan status visibilitas dialog untuk menentukan apakah antarmuka penebusan harus dirender.
- vector<Entry> getEntries(): Mengambil daftar entri properti yang sedang digadaikan untuk ditampilkan pada UI dialog.
- void setVisible(bool b): Mengatur status visibilitas dialog; digunakan untuk membuka atau menutup popup tebus properti.

- `void setHoveredIdx(int i)`: Memperbarui indeks entri yang sedang disorot oleh kursor berdasarkan posisi input pemain.

Jenis relasi:

- Composition dari `GameScreen` (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Dependency terhadap `TebusPropertyCommand` (saat data properti dari state ini dipilih oleh pemain untuk dieksekusi agar kembali menjadi status OWNED).

10.3.21. Kelas `JualBangunanDialogState`

<code>JualBangunanDialogState</code>
- <code>visible</code>: bool - <code>scrollY</code>: float - <code>entries</code>: <code>vector<Entry></code> - <code>hoveredIdx</code>: int
+ <code>isVisible()</code>: bool + <code>getEntries()</code>: <code>vector<Entry></code> + <code>setVisible(b: bool)</code>: void + <code>setHoveredIdx(i: int)</code>: void

Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog penjualan bangunan (rumah/hotel), mengelola daftar aset yang memiliki bangunan untuk dijual, serta menangani navigasi dan interaksi pilihan pemain.

List Atribut:

- bool `visible`: Menentukan apakah dialog penjualan bangunan sedang aktif dan ditampilkan di layar.

- float scrollY: Menyimpan posisi vertikal gulir (scroll) untuk menavigasi daftar properti yang memiliki bangunan.
- vector<Entry> entries: Daftar entri data properti milik pemain yang memiliki bangunan dan tersedia untuk dijual kembali ke bank.
- int hoveredIdx: Menyimpan indeks elemen dalam daftar yang sedang disorot oleh kursor pemain untuk memberikan efek visual (hover).

List Metode:

- bool isVisible(): Mengembalikan status visibilitas dialog untuk menentukan apakah antarmuka penjualan bangunan harus dirender.
- vector<Entry> getEntries(): Mengambil seluruh daftar entri properti yang bangunannya dapat dijual untuk ditampilkan pada UI.
- void setVisible(bool b): Mengatur status visibilitas dialog; digunakan untuk membuka atau menutup popup jual bangunan saat pemain membutuhkan dana atau ingin mengatur strategi aset.
- void setHoveredIdx(int i): Memperbarui indeks entri yang sedang disorot oleh kursor berdasarkan posisi input koordinat pemain.

Jenis Relasi:

- Composition dari GameScreen (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Dependency terhadap JualBangunanCommand (saat data bangunan yang dipilih dari state ini digunakan untuk mengeksekusi pengurangan jumlah bangunan dan penambahan saldo di Core).

10.3.22. Kelas AturDaduDialogState

AturDaduDialogState
- visible: bool - input1: string - input2: string - focusField: int
+ isVisible(): bool + getInput1(): string + setVisible(b: bool): void + setInput1(s: string): void

Deskripsi Menyimpan status data sementara (*state*) yang diperlukan untuk merender UI dialog pengaturan nilai dadu (fitur *cheat/debug*), menangani input angka dari pemain, serta mengelola fokus kursor pada kolom input.

List Atribut:

- bool visible: Menentukan apakah dialog pengaturan dadu sedang aktif dan ditampilkan di layar.
- string input1: Menyimpan teks angka untuk dadu pertama yang diinputkan oleh pemain.
- string input2: Menyimpan teks angka untuk dadu kedua yang diinputkan oleh pemain.
- int focusField: Menentukan indeks kolom input yang sedang aktif (fokus) untuk menerima ketikan keyboard.

List Metode:

- bool isVisible(): Mengembalikan status visibilitas dialog untuk menentukan apakah antarmuka pengaturan dadu harus dirender.
- string getInput1(): Mengambil nilai teks dari input dadu pertama untuk diproses menjadi angka integer.
- void setVisible(bool b): Mengatur status visibilitas dialog; digunakan untuk membuka atau menutup popup pengaturan dadu.
- void setInput1(string s): Memperbarui isi teks pada kolom input dadu pertama berdasarkan masukan dari pemain.

Relasi:

- Composition dari GameScreen (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).

- Dependency terhadap KocokDaduCommand (saat nilai dadu kustom yang diinputkan dari state ini digunakan untuk menggantikan hasil acak pada mesin permainan).

10.3.23. Kelas DropSkillCardDialogState

DropSkillCardDialogState
- visible: bool - hoveredIdx: int - errorMsg: string
+ isVisible(): bool + getHoveredIdx(): int + setVisible(b: bool): void + setErrorMsg(s: string): void

Deskripsi Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog pembuangan kartu skill, mengelola interaksi pemilihan kartu yang akan dibuang, serta menampilkan pesan kesalahan jika tindakan tidak valid.

List Atribut:

- bool visible: Menentukan apakah dialog pembuangan kartu sedang aktif dan ditampilkan di layar.
- int hoveredIdx: Menyimpan indeks kartu dalam daftar yang sedang disorot oleh kursor pemain untuk memberikan umpan balik visual.
- string errorMsg: Menyimpan pesan kesalahan yang akan ditampilkan ke UI jika pemain mencoba melakukan aksi yang tidak valid (misal: membuang kartu yang tidak bisa dibuang).

List Metode:

- `bool isVisible()`: Mengembalikan status visibilitas dialog untuk menentukan apakah antarmuka pembuangan kartu harus dirender.
- `int getHoveredIdx()`: Mengambil indeks elemen yang sedang disorot oleh pemain untuk keperluan kalkulasi rendering posisi kursor.
- `void setVisible(bool b)`: Mengatur status visibilitas dialog; digunakan untuk membuka atau menutup popup pembuangan kartu.
- `void setErrorMsg(string s)`: Mengisi atau memperbarui pesan teks peringatan yang akan muncul pada dialog.

Relasi:

- Composition dari `GameScreen` (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Dependency terhadap `SkillCard` (saat data dari kartu yang dipilih dalam state ini digunakan untuk menentukan kartu mana yang akan dihapus dari inventaris pemain).

10.3.24. Kelas `SkillTargetHintState`

<code>SkillTargetHintState</code>
- visible: bool - message: string
+ isVisible(): bool + getMessage(): string + setVisible(b: bool): void

Deskripsi Menyimpan status data sementara (state) yang diperlukan untuk merender UI petunjuk target skill (seperti instruksi memilih pemain atau petak) agar pemain mengetahui tindakan apa yang harus dilakukan saat menggunakan kartu skill tertentu.

List Atribut:

- bool visible: Menentukan apakah pesan petunjuk target skill sedang aktif dan ditampilkan di layar.
- string message: Menyimpan teks instruksi atau pesan arahan yang akan ditampilkan kepada pemain mengenai target penggunaan skill.

List Metode:

- bool isVisible(): Mengembalikan status visibilitas untuk menentukan apakah antarmuka petunjuk target harus dirender di UI.
- string getMessage(): Mengambil konten teks pesan petunjuk untuk ditampilkan pada elemen label di layar.
- void setVisible(bool b): Mengatur status visibilitas dialog; digunakan untuk memunculkan petunjuk saat kartu skill yang membutuhkan target diaktifkan, dan menyembunyikannya setelah target dipilih.

Relasi:

- Composition dari GameScreen (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Dependency terhadap SkillCard (saat kartu skill tertentu diaktifkan, ia akan mengirimkan pesan instruksi spesifik ke dalam state ini untuk ditampilkan kepada pemain).

10.3.25. Kelas SkillCardDialogState

SkillCardResultDialogState
- visible: bool - title: string - message: string
+ isVisible(): bool + getTitle(): string + setVisible(b: bool): void + setMessage(s: string): void

Deskripsi Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog hasil penggunaan kartu skill, menampilkan judul informasi dan pesan detail mengenai efek yang telah berhasil dijalankan oleh pemain.

List Atribut:

- bool visible: Menentukan apakah dialog hasil penggunaan kartu skill sedang aktif dan harus ditampilkan di layar.
- string title: Menyimpan teks judul dialog (misalnya nama kartu skill yang baru saja digunakan).
- string message: Menyimpan teks pesan atau detail informasi mengenai hasil akhir dari efek kartu skill tersebut.

List Metode:

- bool isVisible(): Mengembalikan status visibilitas dialog untuk menentukan apakah antarmuka hasil skill harus dirender oleh sistem UI.
- string getTitle(): Mengambil teks judul yang tersimpan untuk ditampilkan pada bagian header dialog.
- void setVisible(bool b): Mengatur status visibilitas dialog; digunakan untuk memunculkan popup setelah eksekusi skill selesai atau menutupnya kembali.
- void setMessage(string s): Mengisi atau memperbarui konten teks pesan hasil skill berdasarkan data yang diterima dari command.

Relasi:

- Composition dari GameScreen (karena merupakan nested class yang siklus hidupnya bergantung sepenuhnya pada kelas induknya).
- Dependency terhadap SkillCard (saat informasi hasil eksekusi dari objek kartu skill dikirimkan ke state ini untuk ditampilkan secara visual kepada pemain).

10.3.26. Kelas SkillCardDialogState

SkillCardDialogState
- visible: bool - hoveredIdx: int - awaitingTeleportTile: bool - awaitingLassoTarget: bool - selectedCardIdx: int
+ isVisible(): bool + getSelectedCardIdx(): int + setVisible(b: bool): void + setSelectedCardIdx(i:int):void

Deskripsi Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog pemilihan kartu skill, melacak interaksi kursor pemain pada daftar kartu, serta mengelola status penungguan target khusus (seperti teleport atau lasso) setelah kartu dipilih.

List Atribut:

- bool visible: Menentukan apakah dialog kartu skill sedang ditampilkan di layar.
- int hoveredIdx: Menyimpan indeks kartu yang sedang disorot oleh kursor pemain untuk keperluan visualisasi hover.
- bool awaitingTeleportTile: Status yang menandakan sistem sedang menunggu pemain memilih petak tujuan setelah mengaktifkan kartu teleport.

- `bool awaitingLassoTarget`: Status yang menandakan sistem sedang menunggu pemain memilih pemain target setelah mengaktifkan kartu lasso.
- `int selectedCardIdx`: Menyimpan indeks kartu skill yang telah dipilih oleh pemain untuk digunakan.

List Metode:

- `bool isVisible()`: Mengembalikan status apakah dialog kartu skill saat ini sedang aktif dan harus ditampilkan di layar.
- `int getSelectedCardIdx()`: Mengambil indeks kartu yang telah dipilih pemain untuk diproses lebih lanjut oleh sistem.
- `void setVisible(bool b)`: Mengatur status visibilitas dialog; digunakan untuk membuka daftar kartu skill atau menutupnya.
- `void setSelectedCardIdx(int i)`: Memperbarui indeks kartu yang dipilih berdasarkan interaksi klik pemain pada UI.

Jenis Relasi:

- Composition dari `GameScreen` (karena merupakan nested class yang siklus hidupnya bergantung sepenuhnya pada kelas induknya).
- Dependency terhadap `SkillCard` (saat data dari kartu yang dipilih digunakan untuk menentukan jenis aksi atau target yang diperlukan).

10.3.27. Kelas `DemolitionTargetDialogState`

DemolitionTargetDialogState
- visible: bool - selectedCardIdx: int - errorMsg: string
+ isVisible(): bool + getSelectedCardIdx(): int + setVisible(b: bool): void + setErrorMsg(s: string): void

Deskripsi Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog pemilihan target penghancuran bangunan (efek dari kartu Demolition), melacak kartu yang digunakan, serta menangani tampilan pesan kesalahan jika pemilihan target tidak valid.

List Atribut:

- bool visible: Menentukan apakah dialog pemilihan target penghancuran sedang aktif dan ditampilkan di layar.
- int selectedCardIdx: Menyimpan indeks kartu skill (Demolition) yang sedang diproses untuk menentukan target bangunan.
- string errorMsg: Menyimpan pesan teks peringatan atau kesalahan yang akan ditampilkan jika pemain memilih target yang tidak dapat dihancurkan.

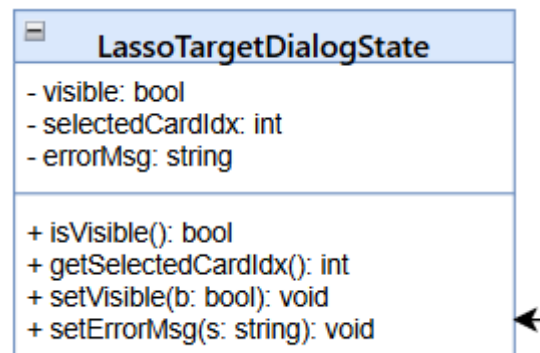
List Metode:

- bool isVisible(): Mengembalikan status visibilitas dialog untuk menentukan apakah antarmuka pemilihan target harus dirender.
- int getSelectedCardIdx(): Mengambil indeks kartu yang sedang digunakan dalam proses penghancuran ini.
- void setVisible(bool b): Mengatur status visibilitas dialog; digunakan untuk membuka atau menutup popup pemilihan target.
- void setErrorMsg(string s): Memperbarui isi pesan kesalahan yang akan dimunculkan pada antarmuka dialog.

Relasi:

- Composition dari GameScreen (karena merupakan nested class yang siklus hidupnya bergantung sepenuhnya pada kelas induknya).
- Dependency terhadap SkillCard (saat kartu Demolition digunakan untuk memicu dialog ini) dan DemolitionCommand (saat target bangunan telah dipilih untuk dieksekusi penghancurannya).

10.3.28. Kelas LassoTargetDialogState



Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog ketika player menggunakan kartu Lasso dan ingin menentukan target.

List Atribut:

- bool visible: Menentukan apakah dialog Lasso sedang ditampilkan di layar
- int selectedCardIdx: Posisi kartu yang dipilih untuk dihandle
- string errorMsg: Menyimpan pesan teks peringatan atau kesalahan.

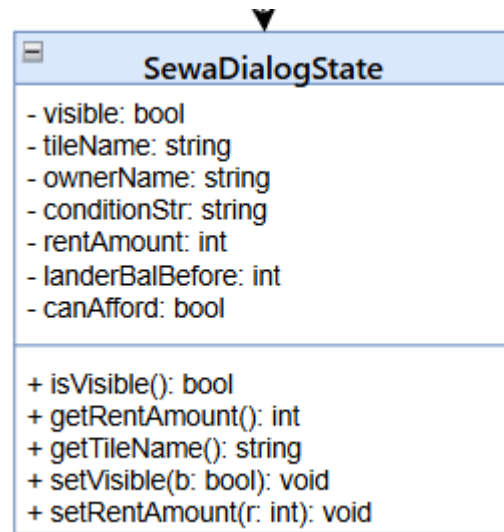
List Metode:

- bool isVisible(): Mengembalikan atribut yang menentukan apakah dialog Lasso sedang ditampilkan di layar
- int getSelectedCardIdx(): Mengembalikan posisi kartu yang dipilih
- void setVisible(b: bool): Merubah atribut yang menentukan apakah dialog Lasso sedang ditampilkan di layar
- void setErrorMsg(s:string): Merubah atribut yang menentukan pesan teks peringatan atau kesalahan

Relasi:

- Composition dari GameScreen (karena merupakan nested class yang siklus hidupnya bergantung sepenuhnya pada kelas induknya).
- Dependency terhadap SkillCard (saat dialog dipicu)

10.3.29. Kelas SewaDialogState



Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog ketika player perlu membayar biaya sewa.

List Atribut:

- `bool visible`: Menentukan apakah dialog pembayaran sewa sedang ditampilkan di layar.
- `string tileName`: Menyimpan nama properti beserta kodenya yang sedang dikunjungi pemain.
- `string ownerName`: Menyimpan nama pemain yang memiliki properti tersebut.
- `string conditionStr`: Menyimpan informasi status bangunan di atas properti, seperti jumlah rumah, hotel, atau adanya multiplier festival.
- `int rentAmount`: Menyimpan total nilai sewa yang harus dibayarkan kepada pemilik.
- `int landerBalBefore`: Menyimpan saldo pemain yang menginjak petak sebelum melakukan pembayaran sewa.
- `int ownerBalBefore`: Menyimpan saldo pemilik properti sebelum menerima pembayaran sewa.

- `bool canAfford`: Menyatakan apakah saldo pemain yang menginjak petak mencukupi untuk membayar biaya sewa.

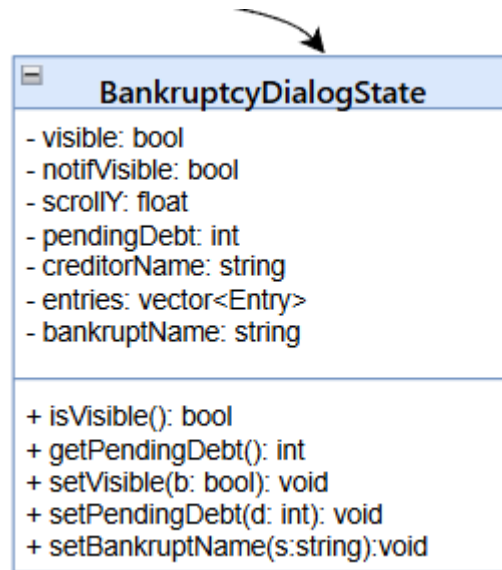
List Metode:

- `void triggerSewaDialog()`: Mengambil data dari `GameState`, melakukan kalkulasi biaya sewa berdasarkan jenis properti (`Street`, `Railroad`, atau `Utility`), dan menginisialisasi seluruh data yang diperlukan untuk menampilkan dialog sewa.
- `void drawSewaDialog()`: Menangani visualisasi antarmuka dialog sewa, menampilkan rincian biaya, simulasi perubahan saldo, peringatan saldo tidak cukup, serta memproses input tombol bayar.
- `bool hasMonopoly()`: Memeriksa apakah pemilik properti memiliki seluruh grup warna yang sama untuk menentukan penggandaan harga sewa pada `StreetProperty`.
- `int countPlayerRailroads()`: Menghitung jumlah properti stasiun yang dimiliki pemain untuk menentukan tarif sewa pada `RailroadProperty`.
- `int countPlayerUtilities()`: Menghitung jumlah properti utilitas yang dimiliki pemain untuk menentukan tarif sewa pada `UtilityProperty`.
- `void pushCommand()`: Memasukkan perintah pembayaran sewa ke dalam antrian perintah agar dapat dieksekusi oleh sistem setelah tombol konfirmasi ditekan.

Relasi:

- Composition dari `GameScreen` (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Association ke `Player`, `Property` dan `BayarSewaCommand` (karena digunakan untuk membayar sewa berdasarkan properti pemain).
-

10.3.30. Kelas `BankruptcyDialogState`



Menyimpan status data sementara (state) yang diperlukan untuk merender UI dialog ketika player Bankrupt.

List Atribut:

- bool visible: Menentukan apakah dialog likuidasi aset sedang ditampilkan di layar.
- bool notifVisible: Menentukan apakah notifikasi pengumuman pemain bangkrut sedang ditampilkan.
- float scrollY: Menyimpan nilai posisi vertikal scroll pada daftar properti yang bisa dilikuidasi.
- int pendingDebt: Menyimpan jumlah total hutang yang harus dilunasi oleh pemain.
- string creditorName: Menyimpan nama pihak penagih hutang, baik itu nama pemain lain atau "Bank".
- string bankruptName: Menyimpan nama pemain yang sedang dalam status terancam bangkrut atau sudah dinyatakan bangkrut.
- vector entries: Daftar koleksi data properti milik pemain yang mencakup status, nilai jual, dan nilai gadai untuk ditampilkan dalam dialog.
- int hoveredSellIdx: Indeks properti dalam daftar yang sedang ditunjuk oleh cursor tetikus untuk aksi penjualan.

- `int hoveredMortgageIdx`: Indeks properti dalam daftar yang sedang ditunjuk oleh kursor tetikus untuk aksi pengadaan.

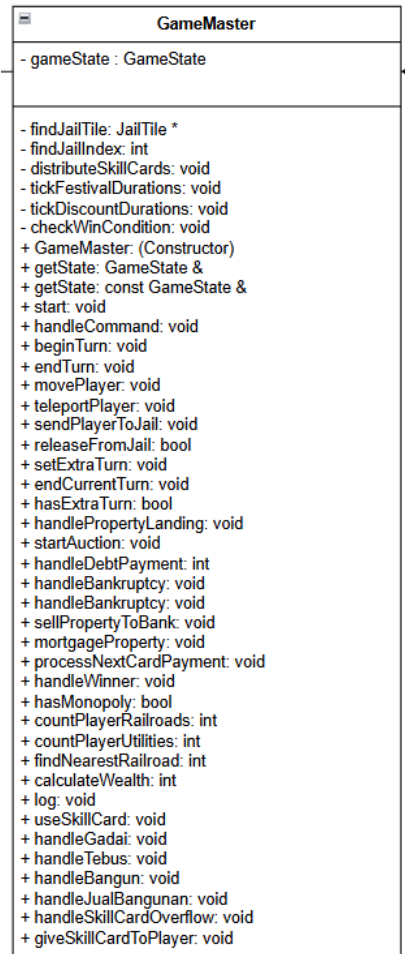
List Metode:

- `void triggerBankruptcyDialog()`: Menginisialisasi data dialog likuidasi, menghitung hutang, serta mendata seluruh properti milik pemain yang dapat dijual atau digadaikan.
- `void drawBankruptcyDialog()`: Menangani seluruh logika visualisasi antarmuka dialog, termasuk rendering teks, tombol aksi, mekanisme scroll, serta pemrosesan input klik untuk bayar hutang atau menyatakan kebangkrutan.
- `void sellPropertyToBank()`: Memproses penjualan aset properti pemain kembali ke Bank untuk mendapatkan saldo tambahan secara instan.
- `void mortgageProperty()`: Memproses pengadaan aset properti pemain untuk mendapatkan pinjaman saldo tanpa kehilangan hak kepemilikan.
- `void processNextCardPayment()`: Melanjutkan antrean pembayaran hutang jika kebangkrutan atau pembayaran dipicu oleh kartu efek multi-pemain.
- `void handleBankruptcy()`: Melakukan proses likuidasi total dan transfer seluruh sisa aset pemain yang bangkrut kepada kreditur, lalu mengeluarkan pemain tersebut dari permainan.

Jenis Relasi:

- Composition dari `GameScreen` (sebagai nested class yang siklus hidupnya dikelola sepenuhnya oleh kelas induknya).
- Association ke `Board`, `Property`, dan `Player` (karena digunakan untuk menampilkan Properti yang dimiliki Player).

10.4. GameMaster



Kelas GameMaster berfungsi sebagai pusat kendali (controller) dari alur permainan (Main loop, turn, gameplay). Kelas ini menangani transisi giliran, evaluasi kondisi kemenangan, serta penanganan kebangkrutan

Atribut & Tanggung Jawab

- Bertanggung jawab sebagai controller permainan yang mencakup penggunaan kartu, penggunaan command, mengevaluasi kemenangan, manipulasi properti (jual/gadai), hingga menangani kebangkrutan.

Relasi:

- Relasi dengan GameState, Board, Card, Command, GUI, dan GameScreen. GameMaster bertanggung jawab memanipulasi/menyediakan method untuk manipulasi antarkelas. Seperti kebangkrutan dengan Board atau DoctorFeeCard yang merubah GameState melalui GameMaster.

10.5. GameState



Kelas GameState berfungsi sebagai snapshot keadaan game

Atribut & Tanggung Jawab

- Bertanggung jawab menyimpan kondisi permainan

Relasi:

- Berhubungan dengan class lain seperti GameMaster, Player, Board, Bank, Dice, Card, ConfigLoader, dan TransactionLogger. Fungsi pada GameState digunakan untuk mengubah kondisi permainan, seperti oleh BayarSewaCommand untuk mengubah uang yang dimiliki Player pada GameState

10.6. Player

Kelas player merepresentasikan entitas yang memainkan permainan. Kelas ini bertanggung jawab atas data dan tindakan individu pemain seperti status, pergerakan, kondisi finansial, pengelolaan properti, dan penggunaan kartu pemain.

List Atribut:

- string username
- int balance
- int position
- PlayerStatus status
- int jailTurns
- vector<Property*> properties
- vector<SkillCard*> skillCards
- bool cardUsedThisTurn
- double activeDiscount
- bool shieldActive
- string id

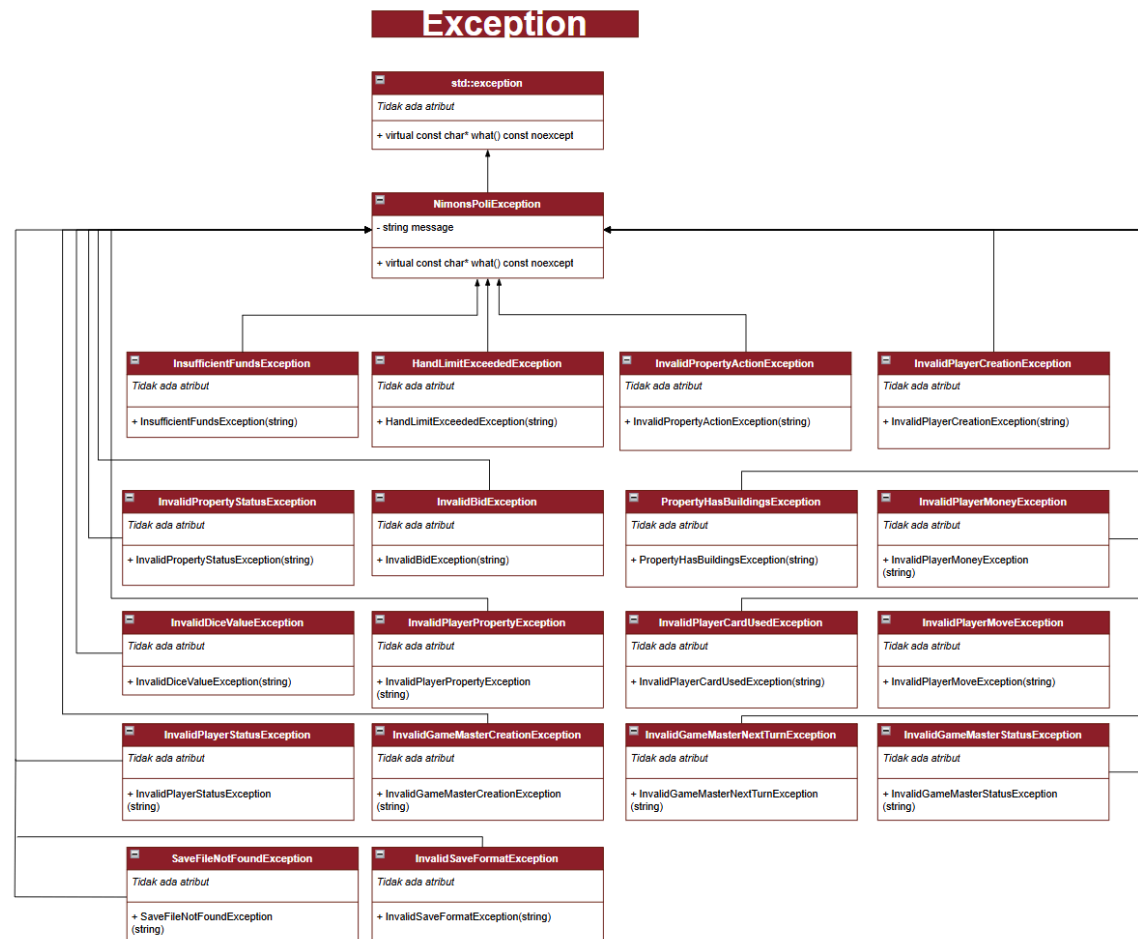
List Method:

- `Player(const std::string &username, int startingBalance)`: Menginisialisasi objek Player dengan username dan saldo awal serta mengatur status awal ke ACTIVE.
- `virtual ~Player()`: Destruktor virtual untuk membersihkan sumber daya objek Player.
- `void setUsername(const std::string &name)`: Mengubah nama pengguna pemain.
- `string getUsername() const`: Mengembalikan nama pengguna pemain.
- `int getBalance() const`: Mengembalikan jumlah saldo uang pemain saat ini.
- `int getPosition() const`: Mengembalikan indeks lokasi posisi pemain di papan.
- `PlayerStatus getStatus() const`: Mengembalikan status aktif, bangkrut, atau dipenjara dari pemain.
- `int getJailTurns() const`: Mengembalikan jumlah giliran yang telah dilewati pemain saat di penjara.
- `void setPosition(int tileIndex)`: Mengatur posisi pemain ke indeks petak yang ditentukan.
- `void setStatus(PlayerStatus newStatus)`: Mengubah status kondisi pemain.
- `void setJailTurns(int turns)`: Mengatur jumlah hitungan giliran pemain selama di penjara.
- `void setBalance(int amount)`: Mengatur nominal saldo pemain secara langsung.
- `Player &operator+=(int amount)`: Menambahkan jumlah uang tertentu ke saldo pemain.
- `Player &operator-=(int amount)`: Mengurangi saldo pemain dengan jumlah uang tertentu.
- `bool canAfford(int amount) const`: Memeriksa apakah saldo pemain mencukupi untuk nominal pembayaran tertentu.
- `bool operator>(const Player &other) const`: Membandingkan apakah saldo pemain lebih besar dari pemain lain.
- `bool operator<(const Player &other) const`: Membandingkan apakah saldo pemain lebih kecil dari pemain lain.
- `void addProperty(Property *prop)`: Menambahkan properti ke daftar koleksi milik pemain.
- `void removeProperty(Property *prop)`: Menghapus properti tertentu dari daftar koleksi pemain.
- `void clearProperties()`: Menghapus seluruh properti yang dimiliki oleh pemain.
- `const vector<Property *> &getProperties() const`: Mengembalikan referensi daftar properti yang dimiliki pemain.
- `int getPropertyCount() const`: Mengembalikan jumlah total properti yang dimiliki saat ini.
- `bool addSkillCard(SkillCard *card)`: Menambahkan kartu kemampuan ke tangan jika belum penuh (maksimal 3).
- `SkillCard *discardSkillCard(int index)`: Mengeluarkan kartu kemampuan dari tangan pemain berdasarkan indeks dan mengembalikannya.
- `const vector<SkillCard *> &getHand() const`: Mengembalikan daftar kartu kemampuan yang ada di tangan pemain.
- `int getHandSize() const`: Mengembalikan jumlah kartu kemampuan yang sedang dipegang pemain.
- `void markCardUsedThisTurn()`: Mengatur penanda bahwa kartu kemampuan telah digunakan pada giliran ini.
- `bool hasUsedCardThisTurn() const`: Mengecek apakah pemain sudah menggunakan kartu pada giliran saat ini.
- `string printSkillCards() const`: Mengembalikan representasi string daftar kartu kemampuan beserta deskripsinya.

- `void forceAddSkillCard(SkillCard *card)`: Menambahkan kartu kemampuan ke tangan tanpa mengecek batasan jumlah kartu.
- `void goToJail()`: Mengubah status pemain menjadi JAILED dan mereset hitungan giliran penjara.
- `void releaseFromJail()`: Mengubah status pemain kembali menjadi ACTIVE.
- `bool isInJail() const`: Memeriksa apakah pemain sedang dalam kondisi dipenjara.
- `void incrementJailTurns()`: Menambah hitungan jumlah giliran yang telah dihabiskan di penjara.
- `void setDiscount(double percent)`: Mengatur nilai persentase diskon aktif untuk pemain.
- `double getDiscount() const`: Mengambil nilai persentase diskon yang sedang berlaku pada pemain.
- `void clearDiscount()`: Menghapus diskon aktif dengan mengatur nilainya menjadi nol.
- `bool hasActiveDiscount() const`: Mengecek apakah pemain memiliki potongan harga yang sedang aktif.
- `int applyDiscount(int originalPrice) const`: Menghitung harga akhir setelah dikurangi persentase diskon pemain.
- `void activateShield()`: Mengaktifkan status perlindungan perisai pada pemain.
- `bool isShielded() const`: Memeriksa apakah pemain saat ini sedang terlindungi oleh perisai.
- `void deactivateShield()`: Menghapus status perlindungan perisai pemain.
- `void onTurnStart()`: Mereset status penggunaan kartu dan perlindungan perisai setiap kali giliran dimulai.
- `int getWealth() const`: Menghitung total kekayaan pemain berdasarkan saldo dan harga beli properti yang tidak digadaikan.
- `string cetakProperti() const`: Mengembalikan string daftar properti pemain yang dikelompokkan berdasarkan grup warna.

Relasi:

10.7. EXCEPTIONS



10.7.1. NimonsPoliException

Kelas Abstrak yang merupakan turunan dari kelas bawaan `std::exception` dan juga merupakan parent class dari seluruh exception class. Terdapat sebuah method `what()` yang nanti akan diimplementasikan oleh child class-nya.

List Atribut:

- string message

List method:

- virtual const char* what() const noexcept: Mengembalikan pesan kesalahan dalam bentuk string C.

Jenis relasi: Inheritance dari std::exception

10.7.2. **InsufficientFundsException**

Dilempar ketika seorang pemain tidak memiliki saldo yang cukup untuk melakukan transaksi ekonomi seperti beli, sewa, pajak, atau tebus.

List Atribut: (Tidak ada atribut tambahan)

List Method:

- InsufficientFundsException(std::string message): Constructor untuk menginisialisasi pesan kekurangan uang.

Jenis Relasi: Inheritance dari NimonsPoliException.

10.7.3. **InvalidBidException**

Dilempar ketika pemain memasukkan nilai penawaran lelang yang tidak valid (lebih rendah dari penawaran tertinggi atau melebihi uang tunai).

List Atribut: (Tidak ada atribut tambahan)

List Method:

- InvalidBidException(std::string message): Constructor untuk menginisialisasi pesan kesalahan penawaran lelang.

Jenis Relasi: Inheritance dari NimonsPoliException.

10.7.4. **PropertyHasBuildingsException**

Dilempar ketika pemain mencoba menggadaikan properti yang di dalam color group-nya masih terdapat bangunan (rumah/hotel) yang berdiri.

List Atribut: (Tidak ada atribut tambahan)

List Method:

- `PropertyHasBuildingsException(std::string message)`: Constructor untuk menginisialisasi peringatan keberadaan bangunan saat gadai.

Jenis Relasi: Inheritance dari `NimonsPoliException`.

10.7.5. **InvalidPropertyStatusException**

Dilempar ketika pemain melakukan aksi yang tidak sesuai dengan status properti saat ini (misal: menebus properti yang tidak digadaikan).

List Atribut: (Tidak ada atribut tambahan)

List Method:

- `InvalidPropertyStatusException(std::string message)`: Constructor untuk menginisialisasi pesan status properti ilegal.

Jenis Relasi: Inheritance dari `NimonsPoliException`.

10.7.6. **InvalidDiceValueException**

Dilempar ketika tester memasukkan nilai dadu yang tidak valid (di luar angka 1-6) pada perintah Atur Dadu.

List Atribut: (Tidak ada atribut tambahan)

List Method:

- `InvalidDiceValueException(std::string message)`: Constructor untuk menginisialisasi pesan kesalahan input nilai dadu.

Jenis Relasi: Inheritance dari `NimonsPoliException`.

10.7.7. InvalidGameMasterCreationException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- `InvalidGameMasterCreationException(std::string message)`: Constructor untuk menginisialisasi pesan status properti ilegal.

Jenis Relasi: Inheritance dari `NimonsPoliException`.

10.7.8. InvalidGameMasterStatusException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- `InvalidGameMasterStatusException(std::string message)`: Constructor untuk menginisialisasi pesan status properti ilegal.

Jenis Relasi: Inheritance dari `NimonsPoliException`.

10.7.9. InvalidGameMasterNextTurnException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- `InvalidGameMasterNextTurnException(std::string message)`: Constructor untuk menginisialisasi pesan status properti ilegal.

Jenis Relasi: Inheritance dari `NimonsPoliException`.

10.7.10. InvalidPlayerCreationException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- `InvalidPlayerCreationException(std::string message)`: Constructor untuk menginisialisasi pesan status properti ilegal.

Jenis Relasi: Inheritance dari `NimonsPoliException`.

10.7.11. InvalidPlayerMoneyException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- `InvalidPlayerMoneyException(std::string message)`: Constructor untuk menginisialisasi pesan status properti ilegal.

Jenis Relasi: Inheritance dari `NimonsPoliException`.

10.7.12. InvalidPlayerPropertyException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- InvalidPlayerPropertyException(std::string message): Constructor untuk menginisialisasi pesan status properti ilegal.

Jenis Relasi: Inheritance dari NimonsPoliException.

10.7.13. InvalidPlayerCardUsedException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- InvalidPlayerCardUsedException(std::string message): Constructor untuk menginisialisasi pesan status properti ilegal.

Jenis Relasi: Inheritance dari NimonsPoliException.

10.7.14. InvalidPlayerMoveException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- InvalidPlayerMoveException(std::string message): Constructor untuk menginisialisasi pesan status properti ilegal.

Jenis Relasi: Inheritance dari NimonsPoliException.

10.7.15. SaveFileNotFoundException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- `SaveFileNotFoundException(std::string message)`: Constructor untuk menginisialisasi pesan status file tidak ditemukan.

Jenis Relasi: Inheritance dari `NimonsPoliException`.

10.7.16. InvalidSaveFormatException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- `InvalidSaveFormatException(std::string message)`: Constructor untuk menginisialisasi pesan status file yang disave memiliki format yang tidak valid

10.7.17. InvalidSkillCardIndexException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- `InvalidSkillCardIndexException(int index)`: Constructor untuk menginisialisasi pesan kesalahan ketika indeks kartu kemampuan yang dipilih tidak valid.

Jenis Relasi: Inheritance dari `NimonsPoliException`.

10.7.18. SkillCardOverflowNotFoundException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- SkillCardOverflowNotFoundException(): Constructor untuk menginisialisasi pesan kesalahan ketika tidak ada pemain yang sedang wajib membuang kartu kemampuan.

Jenis Relasi: Inheritance dari NimonspoliException.

10.7.19. SkillCardDropNotRequiredException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- SkillCardDropNotRequiredException(): Constructor untuk menginisialisasi pesan kesalahan ketika pemain tidak perlu membuang kartu karena jumlah kartu kemampuan tidak melebihi batas.

Jenis Relasi: Inheritance dari NimonspoliException.

10.7.20. SkillCardDiscardFailedException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- SkillCardDiscardFailedException(): Constructor untuk menginisialisasi pesan kesalahan ketika kartu kemampuan gagal dibuang.

Jenis Relasi: Inheritance dari NimonspoliException.

10.7.21. SkillCardInvalidActionException

List Atribut: (Tidak ada atribut tambahan)

List Method:

- `SkillCardInvalidActionException(std::string message)`: Constructor untuk menginisialisasi pesan kesalahan ketika aksi kartu kemampuan tidak valid, dengan detail pesan yang diberikan melalui parameter `message`.

Jenis Relasi: Inheritance dari `NimonspoliException`.

11. Justifikasi Desain

Desain arsitektur `Nimonspoli` ini dibangun dengan memprioritaskan prinsip SOLID, khususnya Single Responsibility Principle (SRP) dan Layered Architecture. Desain kelas ini juga sudah menerapkan ketentuan yang terdapat di spesifikasi, seperti:

1. Penerapan SRP dan pencegahan God-Class

Pemisahan entitas seperti `Bank` dan `AuctionManager` dari `GameMaster` mencegah terbentuknya God-Class yang menangani segalanya. `AuctionManager` khusus menangani antrean lelang, `Bank` murni mengelola perpindahan uang, dan `Dice` murni menangani probabilitas.

2. Penerapan Command Pattern

Pemilihan Command Pattern menyelesaikan masalah interaksi (coupling) yang ketat antara UI Layer dan Game Logic Layer. Dengan membungkus setiap transaksi (Beli, Gadai, Sewa) ke dalam kelas Command yang terpisah, menambahkan aksi baru di masa depan (misal: fitur kreativitas) dapat dilakukan tanpa perlu mengubah kode `GameMaster` yang sudah ada (mendukung Open-Closed Principle).

3. Penerapan DRY (Don't Repeat Yourself)

Hierarki Exception buatan sendiri dirancang agar tidak redundan. Dengan mendelegasikan atribut `message` dan method `what()` murni di base class `NimonspoliException`, kelas-kelas turunannya cukup berfokus pada inisialisasi pesan error melalui constructor, menjadikannya sangat ringan.

4. Penerapan SRP dan pemisahan tanggung jawab output

`DisplayManager` dan `renderer-renderer` (`BoardRenderer`, `PropertyRenderer`) dipisahkan menjadi kelas berbeda meskipun keduanya berurusan dengan output. `BoardRenderer` dan `PropertyRenderer` hanya bertanggung jawab menghasilkan string representasi data. mereka tidak tahu ke

mana string itu akan ditulis. DisplayManager hanya bertanggung jawab menerima hasil render dan menuliskannya ke terminal. Pemisahan ini mencegah terbentuknya satu kelas besar yang sekaligus tahu cara merender papan, merender properti, dan mengelola output yang akan melanggar SRP.

5. Penerapan SRP pemisahan input dan Output

InputHandler dipisahkan dari DisplayManager meskipun keduanya berurusan dengan interaksi user. InputHandler hanya membaca dari cin, DisplayManager hanya menulis ke cout. Pemisahan ini mengikuti prinsip bahwa perubahan pada cara membaca input tidak boleh memaksa perubahan pada cara menampilkan output, dan sebaliknya.

6. Pemisahan Data Access Layer dari UI Layer

ConfigLoader dan SaveLoadManager dipisahkan ke layer tersendiri dan tidak memiliki relasi ke DisplayManager, BoardRenderer, maupun PropertyRenderer. ConfigLoader murni membaca file konfigurasi dan mengembalikan data mentah. Ia tidak tahu bagaimana data itu akan ditampilkan. SaveLoadManager murni mengurus serialisasi dan deserialisasi state game. Pemisahan ini memastikan perubahan format file tidak berdampak pada logika tampilan, dan perubahan tampilan tidak berdampak pada logika penyimpanan.

7. Penanganan Command Pattern untuk Operasi UI

Operasi-operasi yang bisa dipanggil user seperti CETAK_PAPAN, CETAK_AKTA, CETAK_PROPERTI, CETAK_LOG, SIMPAN, dan MUAT masing-masing dibungkus dalam kelas Command tersendiri. Setiap Command hanya bertanggung jawab atas satu perintah. Ini mencegah terbentuknya satu prosedur panjang yang menangani semua perintah sekaligus, sekaligus memudahkan penambahan perintah baru tanpa mengubah kelas yang sudah ada, mengikuti Open/Closed Principle.

8. Aggregation pada dependency antar kelas UI

Semua dependency antar kelas UI Layer menggunakan referensi (&) yang di-inject melalui constructor, bukan membuat objek baru di dalamnya. Ini menghasilkan relasi aggregation, bukan composition. DisplayManager tidak memiliki BoardRenderer secara eksklusif, melainkan meminjamnya. Pendekatan ini memungkinkan satu instance BoardRenderer dipakai bersama oleh DisplayManager dan command-command lain tanpa duplikasi objek, sekaligus memudahkan penggantian implementasi renderer tanpa mengubah DisplayManager.

Kelebihan Desain:

Modularitas yang sangat tinggi. Modifikasi aturan lelang atau penambahan jenis pajak baru hanya akan berdampak pada kelas AuctionManager atau BayarPajakCommand yang bersangkutan, tanpa memecahkan (break) sistem giliran permainan. Mekanisme pelemparan Exception dari kelas Command ke UI juga membebaskan Game Logic dari operasi I/O (cout/cin), membuat kelas core bisa digunakan ulang jika nantinya aplikasi dikembangkan menjadi GUI (Graphical User Interface). Pemisahan UI Layer menjadi kelas-kelas yang spesifik (DisplayManager, BoardRender, PropertyRenderer) juga berkontribusi pada modularitas ini. Penggantian seluruh tampilan terminal menjadi kelas GUI baru yang memanggil renderer existing, tanpa menyentuh logika game di core layer sama sekali. BoardRenderer dan PropertyRenderer yang sudah menghasilkan data terstruktur dapat langsung dipakai ulang oleh layer GUI untuk mengambil informasi warna, status properti, dan format akta, menjadikan renderer bukan hanya alat CLI melainkan juga jembatan data antara core dan GUI. Pemisahan ConfigLoader dan SaveLoadManager ke Data Access Layer yang terpisah juga memberikan fleksibilitas. Perubahan format file, konfigurasi atau format file save tidak berdampak pada logika tampilan, dan sebaliknya.

Kekurangan Desain:

Pendekatan ini berpotensi menimbulkan Class Explosion, di mana jumlah kelas di dalam repositori menjadi sangat banyak (satu file kelas untuk satu command spesifik). Selain itu, karena objek Command terus menerus dibuat (instantiated) dan dihancurkan di memori setiap kali pemain melakukan aksi, manajemen pointer memori (khususnya alokasi objek ke kelas Bank, Player, dan Property) harus ditangani dengan sangat teliti untuk menghindari dangling pointer atau memory leak selama permainan berjalan tanpa henti (sebelum kondisi bankruptcy atau max turn tercapai). Metodologi Rule of Zero diterapkan sejauh mungkin untuk meminimalisasi overhead pengelolaan "4 Sekawan" (Constructor, Destructor, Cctor, Assignment Operator) secara manual. Konsekuensi dari desain ini juga terlihat pada kelas-kelas UI Layer yang seluruhnya menyimpan referensi (&) sebagai atribut. Pendekatan aggregation ini mengharuskan semua objek yang direferensikan dijamin masa hidupnya lebih panjang dari kelas UI yang memakainya, sehingga urutan inisialisasi dan penghancuran objek di main harus diatur untuk menghindari dangling reference.

12. Penerapan Konsep OOP**12.1. Inheritance & Polymorphism**

Pada sistem Nimonspoli, inheritance dan polymorphism dipakai untuk memodelkan objek-objek yang secara konsep berada dalam hubungan umum-khusus. Pendekatan ini dipakai pada beberapa hierarki utama, yaitu Tile, Card, dan Property. Sistem dapat memperlakukan objek-objek turunan sebagai objek parent, tetapi tetap menjalankan perilaku spesifik sesuai kelas konkretnya. Hal ini sesuai dengan kebutuhan permainan yang memiliki banyak entitas sejenis tetapi dengan aksi berbeda, misalnya berbagai jenis petak, berbagai jenis kartu, dan berbagai jenis command. Spesifikasi juga secara eksplisit mendorong penggunaan inheritance dan polymorphism, terutama pada Tile dan Card. Contoh paling jelas ada pada hierarki kartu. Kelas Card didesain sebagai abstraksi umum kartu dan memiliki operasi virtual seperti execute() dan getDescription(). Di atasnya terdapat turunan ChanceCard, GeneralFundCard, dan SkillCard, lalu masing-masing bercabang lagi menjadi kelas konkret seperti NearestStationCard, BirthdayCard, MoveCard, ShieldCard, dan lain-lain. Rancangan ini sudah tampak langsung pada dokumen kelasmu. Cuplikan kode yang dapat dituliskan:

12.2. Method/Operator Overloading

Berikut beberapa penggunaan operator dan method overloading yang digunakan pada program ini:

1. Class Player (Operator Overloading)

```
Player &Player::operator+=(int amount)
{
    balance += amount;
    return *this;
}

Player &Player::operator-=(int amount)
{
    balance -= amount;
    return *this;
}

bool Player::canAfford(int amount) const
{
    return balance >= amount;
}

bool Player::operator>(const Player &other) const
{
    return balance > other.balance;
}
```

2. Class GameMaster (method overloading)

```

void GameMaster::handleBankruptcy(Player *from, Player *to)
{
    if (!from || !to)
        return;

    log(from->getUsername(), "BANKRUPT",
        from->getUsername() + " bangkrut ke " + to->getUsername());

    // Pindahkan uang sisa
    int remaining = from->getBalance();
    if (remaining > 0)
    {
        *from -= remaining;
        *to += remaining;
    }
}

```

```

void GameMaster::handleBankruptcy(Player *from, Bank *bank)
{
    if (!from || !bank)
        return;

    log(from->getUsername(), "BANKRUPT",
        from->getUsername() + " bangkrut ke Bank");

    // Serahkan uang sisa ke Bank (hilang dari peredaran)
    int remaining = from->getBalance();
    if (remaining > 0)
    {
        *from -= remaining;
    }
}

```

Pada fungsi `GameMaster::handleBankruptcy`, terdapat 2 fungsi yang memiliki nama yang sama, namun dengan parameter yang berbeda. Fungsi pertama meng-handle bangkrut dari player ke bank (dimana hanya uang player yang bangkrut yang akan dikurangi), sementara fungsi kedua meng-handle bankrut ke pemain lain (dimana uang player yang bankrut akan berkurang, dan uang player *To* akan bertambah).

12.3. Template & Generic Classes

Terdapat pada `CardDeck<T>`:

```
template <class T>
class CardDeck {
private:
    std::vector<T*> drawPile;
    std::vector<T*> discardPile;
```

Terdapat beberapa alasan kenapa kelas CardDeck dijadikan sebuah generic:

1. Struktur deck identik untuk semua jenis kartu

Semua deck membutuhkan operasi yang persis sama. Draw(), discard(), shuffle(), dengan dua pile internal (drawPile dan discardPile). Tanpa template, harus ada tiga class terpisah. Namun dengan template, cukup satu implementasi saja.

2. Type safety antar deck terjaga

Dengan CardDeck<ChanceCard>, compiler mencegah ChanceCard masuk ke deck SkillCard secara tidak sengaja. Tanpa template dan hanya pakai CardDeck dengan Card*, error seperti ini baru ketahuan saat runtime.

3. Prinsip DRY (Don't Repeat Yourself)

Perubahan logika deck (misalnya menambahkan peek() atau mengubah algoritma shuffle) cukup dilakukan di satu tempat, otomatis berlaku untuk semua jenis deck.

12.4. Exception

1. InvalidDiceValueException: Kelas ini menangani error saat player menggunakan fitur ATUR DADU dan memasukkan angka di luar nilai valid 1-6. Disini, program tidak langsung crash atau bug saat menerima angka yang tidak valid, namun melemparkan exception untuk ditangkap di dialog ATUR DADU nanti.

```

class InvalidDiceValueException : public NimonspoliException {
public:
    // Dipanggil saat AturDaduCommand menerima input di luar 1-6
    InvalidDiceValueException(int v1, int v2)
        : NimonspoliException("Error Dadu: Nilai (" + std::to_string(v1) + ", " +
                               std::to_string(v2) + ") tidak valid! Harus 1-6.") {}
};

```

```

if (IsKeyPressed(KEY_ENTER) || (okHover && IsMouseButtonPressed(MOUSE_LEFT_BUTTON))) {
    // — Validasi: pastikan input bisa di-parse —————
    int v1 = 0, v2 = 0;
    bool parseOk = true;

    try {
        v1 = std::stoi(aturDaduDialog.input1);
        v2 = std::stoi(aturDaduDialog.input2);
    } catch (const std::invalid_argument&) {
        aturDaduDialog.errorMsg = "Input tidak valid — masukkan angka 1-6!";
        parseOk = false;
    } catch (const std::out_of_range&) {
        aturDaduDialog.errorMsg = "Angka terlalu besar!";
        parseOk = false;
    }
}

```

2. SaveLoadException: Kelas ini menghandle exception saat file config tidak bisa dibuka sehingga program tidak langsung error (segfault).

```

void SaveLoadManager::save(const GameState &state, const string &filename)
{
    ofstream out(filename);
    if (!out.is_open())
    {
        throw SaveFileNotFoundException("SaveLoadManager: Tidak bisa membuka file " + filename);
    }
}

```

```
class SaveFileNotFoundException : public NimonspoliException {
public:
    explicit SaveFileNotFoundException(const std::string& message)
        : NimonspoliException(message) {}
};
```

3. `InsufficientFundsException`: Kelas ini handle player yang ingin melakukan pembayaran, namun tidak memiliki cukup uang untuk membayarnya. Ini menghindari program dari bug uang yang minus dan membuat program crash.

```
void BayarPajakCommand::handlePPHChoice(int choice)
{
    int flatAmount = static_cast<int>(taxConfig.pphFlat);
    int pctAmount = static_cast<int>(taxConfig.pphPercentage);

    if (choice == 1)
    {
        if (!p->canAfford(flatAmount))
            throw InsufficientFundsException(p->getUsername(), "Pajak Penghasilan (flat)");

        *p -= flatAmount;
        logger->addLog(turnNumber, p->getUsername(), "PAJAK",
            "Bayar PPH flat M" + std::to_string(flatAmount));
    }
}
```

```
class InsufficientFundsException : public NimonspoliException {
public:
    // Dipanggil saat uang pemain tidak cukup untuk Beli, Bayar Pajak, atau Tebus
    InsufficientFundsException(const std::string& playerName, const std::string& action)
        : NimonspoliException("Error Finansial: Uang " + playerName + " tidak mencukupi untuk " + action + "!") {}
};
```

12.5. C++ Standard Template Library

Pada program ini, banyak STL yang digunakan:

1. `std::vector`
 - a. Menyimpan property dan kartu yang dimiliki seorang pemain. Tujuannya agar penambahan dan penghapusan mudah untuk dilakukan, dibanding dengan menggunakan array biasa.

```
class Player
{
private:
    string username;
    int balance;
    int position; // indeks petak (0-39)
    PlayerStatus status;
    int jailTurns; // sudah berapa giliran di penjara

    vector<Property*> properties; // daftar properti yang dimiliki
    vector<SkillCard*> skillCards; // kartu kemampuan di tangan (maks 3)
```

- b. Menyimpan daftar pemain yang berpartisipasi dalam lelang. Urutan bid penting (mulai dari pemain setelah trigger), dan pemain yang PASS dihapus secara sekuensial. vector dengan erase cukup untuk jumlah maksimal 3 peserta.

```
class AuctionManager {
private:
    Property*      auctionedProperty;
    Player*        highestBidder;
    int            currentBid;
    std::vector<Player*> activeParticipants; // urutan peserta lelang
    int            currentParticipantIdx;
    int            consecutivePassCount; // reset ke 0 setiap ada BID
    bool           isAuctionOngoing;
    bool           allPassedWithoutBid; // semua pass, 1 pemain wajib bid
    Player*        forcedBidder;       // pemain yang wajib bid jika semua pass
```

- c. Pada board, Jumlah tile adalah 40, tapi akses by-index (getTile(idx)) sangat sering dilakukan setiap frame render dan setiap movePlayer(). vector memberikan akses $O(1)$ by-index, yang tidak bisa dilakukan dengan list atau map.

```

class Board {
private:
    vector<Tile*> tile;
    int size;
public:
    Board(const vector<Tile*>& tiles, int size);
    ~Board();
    Tile* getTile(int idx) const;
    Tile* getNextTile(int cur, int steps) const;
    int getSize() const;
};

```

- d. Pada GameState, Urutan pemain penting (giliran searah jarum jam), dan sering diakses by-index via currPlayerIdx. vector menjaga urutan dan akses $O(1)$.

```

class GameState {
private:
    // — Turn & fase —————
    int currTurn;
    int maxTurn;
    GamePhase phase;

    // — Pemain —————
    std::vector<Player*> listPlayer; // urutan giliran
    int currPlayerIdx; // indeks pemain aktif
    bool hasExtraTurn; // flag double dadu
};

```

- e. Pada TransactionLogger, Log hanya ditambah (addLog) dan dibaca dari belakang (getLastLogs). vector sangat efisien untuk pola append-only dan akses sekuensial dari belakang.

```
class TransactionLogger {  
private:  
    std::vector<LogEntry> logs;  
  
public:
```

- f. Pada CardDeck, Shuffle membutuhkan akses random by-index. Ini tidak bisa dilakukan dengan queue atau stack. vector + std::shuffle adalah kombinasi standar untuk implementasi deck kartu.

```
template <class T>  
class CardDeck {  
private:  
    std::vector<T*> drawPile;  
    std::vector<T*> discardPile;
```

2. std::map

- a. Pada StreetProperty, Key adalah level bangunan (0–5), lookup rentPrice[buildingCount] sangat natural. map memberikan lookup $O(\log n)$ dengan syntax bersih. Alternatif array[6] bisa dipakai, tapi map lebih ekspresif dan aman terhadap key yang tidak valid.

```
class StreetProperty : public Property
{
private:
    int houseUpgCost;
    int hotelUpgCost;
    map<int, int> rentPrice;
    int buildingCount;
```

- b. Pada RailroadProperty dan UtilityProperty, Sama seperti key adalah jumlah properti yang dimiliki (1–4), lookup by-key lebih intuitif daripada array. Data ini dibaca dari railroad.txt dan utility.txt yang formatnya memang key-value.

```
class RailroadProperty : public Property
{
private:
    map<int, int> rentFactor;
```

```
class UtilityProperty : public Property
{
private:
    map<int, int> rentPrice;
```

- c. Pada IScreen, Lookup by string key (colorGroup, code) sangat natural dengan map. Alternatif if-else chain seperti di getGroupColor() lebih verbose dan error-prone saat ada color group baru.

```
// — Textures —
std::map<std::string, Texture2D> tileTextures;
```

3. std::queue

Pada GUIManager, Command Pattern membutuhkan semantik FIFO yang ketat. command dieksekusi tepat sesuai urutan user menekan tombol. queue adalah satu-satunya container STL yang menjamin FIFO dengan interface push/front/pop yang jelas. vector bisa dipakai tapi butuh erase(begin()) yang $O(n)$.

```
class GUIManager {
private:
    Window window;
    IScreen* currentScreen = nullptr;
    GameMaster* gameMaster = nullptr;
    queue<Command*> commandQueue;
```

4. Std::unique_ptr

- a. Pada PropertyFactory, PropertyFactory adalah satu-satunya tempat yang membuat semua objek Property. Setelah dibuat, ownership harus dipindahkan ke main.cpp tanpa ada yang boleh double-delete.

```
class PropertyFactory
{
public:
    static vector<unique_ptr<Property>> createProperties(
        const vector<PropertyData> &propertyDataList,
        const map<int, int> &railroadRentMap,
        const map<int, int> &utilityFactorMap);
};
```

12.6. Konsep OOP lain

1. Abstract Base Class (ABC)
 - a. Pada Tile(), onLanded() wajib diimplementasi semua subclass

```
class Tile {  
protected:  
    int id;  
    string colorDisplay;  
    TileType type;  
    string name;  
    string code;  
    string picDisplay;  
public:  
    Tile(int id, string display, TileType type, string name, string code);  
    virtual void onLanded(Player& p, GameState& gs) = 0;  
    virtual ~Tile() = default;
```

Compiler mencegah instansiasi ABC secara langsung. Semua subclass dipaksa mengimplementasi kontrak yang sama, sehingga GameMaster bisa memanggil tile->onLanded() tanpa tahu tile jenis apa.

- b. Pada Property(), calculateRentPrice(), calculateSellPrice(), cetakAkta(), formattingTXT(), printList() wajib diimplementasi

```
class Property
{
    int getUsername() const;
    string getCode() const;
    string getName() const;
    string getColorGroup() const;
    int getPurchasePrice() const;
    int getMortgageValue() const;
    PropertyStatus getStatus() const;
    string getOwnerId() const;

    void setOwner(const string &newOwnerId);
    void clearOwner();
    void setStatus(PropertyStatus newStatus);

    virtual int calculateRentPrice(int diceRoll,
                                   int ownerSameColorCount,
                                   bool monopoly) const = 0;
    virtual int calculateSellPrice() const = 0;
    virtual string cetakAkta() const = 0;
    virtual string formattingTXT() const = 0;

    virtual string printList() const = 0;
};
```

- c. Pada method execute() di Card, method tersebut wajib diimplementasi

```
class Card
{
private:
    string description;

public:
    Card();
    Card(const string &description);
    virtual ~Card();
    virtual void execute(Player &p, GameState &gs) = 0;
    string getDescription();
};
```

- d. Pada command, execute(gamemaster& gm) wajib dieksekusi

```
class Command {
public:
    virtual ~Command() = default;
    virtual void execute(GameMaster& gm) = 0;
};
```

2. Multilevel Inheritance

Beberapa hierarchy memiliki lebih dari dua level. Misal, Tiga level: Tile → ActionTile → GoTile dan Tile → PropertyTile → StreetTile. Keuntungannya adalah ActionTile mengelompokkan petak-petak yang tidak punya properti, sementara PropertyTile mengelompokkan yang punya Property*. Logika bersama di level tengah tidak perlu diulang di setiap leaf class.

3. Composition

Objek dimiliki penuh oleh pemiliknya dan ikut hancur saat pemilik hancur:

- a. GameMaster memiliki GameState secara penuh (bukan pointer)

```
class GameMaster
{
private:
    GameState state;
```

- b. GUIManager memiliki Window secara penuh

```
class GUIManager {
private:
    Window window;
```

- c. GameState memiliki vector<Player*> secara penuh

```
class GameState {
private:
    // — Turn & fase —————
    int      currTurn;
    int      maxTurn;
    GamePhase phase;

    // — Pemain —————
    std::vector<Player*> listPlayer;    // urutan giliran
}
```

Keuntungannya adalah, lifetime otomatis terkelola. Saat GameMaster di-destroy, GameState ikut hancur tanpa perlu delete manual.

4. Aggregation

Objek digunakan tapi tidak dimiliki dan pemilik tidak bertanggung jawab menghapusnya. Contoh:

- a. PropertyTile menggunakan Property* tapi tidak memilikinya. Property* dimiliki oleh vector<unique_ptr<Property>> di main.cpp

```
class PropertyTile : public Tile {
protected:
    Property* prop;
}
```

- b. Player menggunakan Property* tapi tidak memilikinya


```
class Player
{
private:
    string username;
    int balance;
    int position; // indeks petak (0-39)
    PlayerStatus status;
    int jailTurns; // sudah berapa giliran di penjara

    vector<Property *> properties; // daftar properti yang dimiliki
```

- c. CardTile menggunakan CardDeck* tapi tidak memilikinya

```
class CardTile : public ActionTile {
private:
    CardDeck<Card>* card;
```

Keuntungannya adalah, Keuntungan: Satu Property bisa dirujuk oleh PropertyTile di board sekaligus oleh Player yang memilikinya, tanpa ada duplikasi data atau konflik ownership.

5. Dependency Injection

Objek tidak membuat dependensinya sendiri — dependensi di-pass dari luar:

- a. GUIManager di-inject ke GameScreen

```
gui.setGameMaster(gameMaster);
gameScreen->setGUIManager(&gui);
```

- b. GameMaster di-inject ke GUIManager

```
gui.setGameMaster(gameMaster);
gameScreen->setGUIManager(&gui);
```

Keuntungannya adalah, setiap komponen bisa diuji secara independen dengan mock object. GameScreen bisa jalan dalam mode mock (tanpa GameMaster) maupun mode real, karena GUIManager* bisa nullptr.

6. Interface Segregation via IScreen

```
class IScreen{
public:
    virtual ~IScreen() {};
    virtual void onEnter() {}
    virtual void onExit(){}
    virtual void update(float dt = 0) {};
    virtual void render(Window& window) {};
};
```

Keuntungan: GUIManager bisa melakukan transisi antar screen (menuScreen → gameScreen → winScreen) hanya dengan satu pointer IScreen*, tanpa perlu tahu implementasi masing-masing screen.

7. Back-Reference dengan Raw Pointer

- a. GameState menyimpan pointer ke GameMaster (back-reference)

```

class GameState {
    // — Turn & fase —
    int      currTurn;
    int      maxTurn;
    GamePhase phase;

    // — Pemain —
    std::vector<Player*> listPlayer;    // urutan
    int                  currPlayerIdx; // indeks
    bool                 hasExtraTurn;  // flag
    bool                 hasRolled;     // sudah
    bool                 hasUsedCard;   // sudah

    // — Entitas inti —
    Board*               gameBoard;
    Bank*                gameBank;
    Dice*                gameDice;
    AuctionManager*      auctionManager;
    CardDeck<Card>*      chanceCardDeck;
    CardDeck<Card>*      communityCardDeck;
    CardDeck<Card>*      skillCardDeck;
    TransactionLogger*   logger;
    GameMaster*          gameMaster;

```

Di-set oleh konstruktor GameMaster

```
GameMaster::GameMaster(GameState initialState)
: state(std::move(initialState))
{
    state.setGameMaster(this);
}
```

13. Bonus Yang dikerjakan

Hapuslah poin di bawah jika tidak dikerjakan. Jika dikerjakan, jelaskanlah pendekatan yang anda gunakan untuk menyelesaikan masalah tersebut berikut dengan screenshot cuplikan kode yang relevan untuk menjelaskan pendekatan anda!

13.1. Bonus yang diusulkan oleh spek

13.1.1. GUI

1. Library: Raylib

Raylib dipilih karena ringan, tidak butuh dependency eksternal, dan API-nya prosedural sehingga mudah diintegrasikan dengan arsitektur OOP yang sudah ada. Rendering dilakukan dalam game loop standar:

```

while (!WindowShouldClose()) {
    float dt = GetFrameTime();

    // — Transisi: Menu → Game —————
    if (menuScreen->isReadyToStart()) {
        auto setup = menuScreen->getSetup();
        menuScreen->resetReady();
        cleanupGame();

        // Load config
        ConfigLoader cfg("config");
        auto propData      = cfg.loadProperties();
        auto railroadRent  = cfg.loadRailroad();
        auto utilityFactor = cfg.loadUtility();
        auto specialCfg    = cfg.loadSpecial();
        auto miscCfg       = cfg.loadMisc();
        auto actData       = cfg.loadActions();
        auto taxData       = cfg.loadTax();
    }
}

```

2. Screen Management via IScreen + GUIManager

Setiap layar (menu, game, win) mengimplementasi interface IScreen. GUIManager menyimpan satu IScreen* aktif dan melakukan transisi antar layar:

```
class IScreen{
public:
    virtual ~IScreen() {};
    virtual void onEnter() {}
    virtual void onExit(){}
    virtual void update(float dt = 0) {};
    virtual void render(Window& window) {};
};
```

```
class GUIManager {
private:
    Window window;
    IScreen* currentScreen = nullptr;
    GameMaster* gameMaster = nullptr;
    queue<Command*> commandQueue;
```

Transisi Layar di main.cpp:

```

// — Transisi: Game → Win —————
if (gameScreen->isGameOver()) {
    std::vector<PlayerResult> results;
    for (int i = 0; i < (int)players.size(); ++i) {
        PlayerResult r;
        r.username = players[i]->getUsername();
        r.money    = players[i]->getBalance();
        r.color    = gameScreen->playerColors[i];
        results.push_back(r);
    }
    winScreen->setResults(results, WinScenario::MAX_TURN);
    gameScreen->gameOver = false;
    gui.setGameMaster(nullptr);
    gui.setScreen(winScreen);
}

// — Transisi: Win → Menu —————
if (winScreen->goToMainMenu()) {
    winScreen->reset();
    cleanupGame();
    gui.setScreen(menuScreen);
}

```

3. Command Pattern untuk Decoupling GUI ↔ Game Logic


```

if (!disabled && hover && IsMouseButtonPressed(MOUSE_LEFT_BUTTON)) {
    switch (i) {
        case 0: handleLemparDadu(); break;
        case 1: triggerAturDaduDialog(); break;
        case 2: triggerSkillCardDialog(); break;
        case 3: triggerBangunDialog(); break;
        case 4: triggerGadaiDialog(); break;
        case 5: triggerTebusDialog(); break;
        case 6: triggerJualBangunanDialog(); break;
        case 7: handleSimpan(); break;
        case 8:
    }
}

```

4. Dual Mode: Mock dan Real

GameScreen bisa berjalan tanpa GameMaster (mode mock untuk development UI) maupun dengan GameMaster penuh (mode real)

```

bool GameScreen::isRealMode() const
{
    return guiManager != nullptr && guiManager->getGameMaster() != nullptr;
}

```

5. Sync Layer: syncFromGameMaster() dan syncDiceResult()

GameScreen menyimpan MockGameState sebagai "mirror" dari state real. Setiap frame, data di-sync dari GameMaster ke MockGameState agar semua fungsi render tidak perlu diubah:

```
void GameScreen::syncFromGameMaster()
{
    if (!isRealMode())
        return;

    GameMaster*      gm      = guiManager->getGameMaster();
    const GameState& gs      = gm->getState();
    Board*           board    = gs.getBoard();

    gameState.currentTurn    = gs.getCurrTurn();
    gameState.maxTurn        = gs.getMaxTurn();
    gameState.activePlayerIdx = gs.getCurrPlayerIdx();

    // — Sync Players —————
    const auto& realPlayers = gs.getPlayers();
    if (realPlayers.size() > 0) {
        for (int i = 0; i < realPlayers.size(); i++) {
            Player* p = realPlayers[i];
            if (p->isHuman()) {
                p->setPos(guiManager->getBoard());
            }
        }
    }
}
```

6. Layout Papan: Scaling dan Rotasi Tile

```

void GameScreen::drawBoard()
{
    // Fallback: isi playerVisuals dari gameState jika belum ada
    if (playerVisuals.empty() && !gameState.players.empty()) {
        playerVisuals.resize(gameState.players.size());
        for (int i = 0; i < (int)gameState.players.size(); i++) {
            float pos = (float)gameState.players[i].position;
            playerVisuals[i].currentTileIdx = pos;           hakamavic
            playerVisuals[i].targetTileIdx = pos;
        }
    }
}

```

7. Komponen UI Utama

Left Panel	Info 4 pemain: saldo, posisi, status, kartu
Right Panel	Turn info, tombol aksi dengan disable logic per GamePhase
Dice Area	40 tile dengan rotasi, ownership strip, festival ring
Papan	Animasi rolling 0.6 detik, pola titik standar, highlight double
Buy Dialog	Muncul otomatis saat phase == AWAITING_BUY, tombol BELI/SKIP
Auction Dialog	Sync dari AuctionManager, input bid, validasi range

Tax Dialog	Pilih PPH flat atau persentase, atau bayar PBM langsung
Festival Dialog	Pilih properti untuk dinaikkan sewa 2×/4×/8×
Card Dialog	Tampilkan kartu Kesempatan / Dana Umum yang ditarik
Jail Dialog	Pilih opsi keluar penjara: bayar denda, lempar double, kartu
Log Popup	Scrollable, filter N terakhir, color-coded per jenis aksi
Save Popup	Input nama file, konfirmasi overwrite, feedback hasil
Tile Popuo	Klik tile → info singkat petak
Akta Dialog	Info lengkap properti: tabel sewa, harga, status
Cetak Properti	Semua properti milik pemain aktif
Atur Dadu	Input manual dadu 1-6, validasi, exception handling
Gadai Dialog	Pilih properti untuk digadai, tampilkan nilai gadai
Tebus Dialog	Pilih properti tergadai untuk ditebus
Bangun Dialog	Pilih petak untuk bangun rumah/hotel, cek monopoli
Jual Bangunan	Pilih bangunan untuk dijual kembali ke bank
Skill Card	Tampilkan kartu kemampuan di tangan, pilih untuk digunakan

13.1.2. COM

1. Arsitektur: Inheritance dari Player

ComputerPlayer adalah subclass dari Player. Ia tidak mengubah interface yang sudah ada, hanya menambahkan kemampuan mengambil keputusan sendiri.

```
arinazk, yesterday | 1 author (arinazk)
class ComputerPlayer : public Player {
private:
    COMDifficulty difficulty;

    // — Utilitas internal —————
    // Hitung total biaya sewa maksimum yang bisa dikenakan properti ini
    int estimateRentValue(Property* prop, GameState& gs) const;

    // Hitung berapa properti dengan warna yang sama sudah dimiliki COM
    int countSameColorOwned(const std::string& colorGroup, GameState& gs) const;

    // Hitung total properti dalam satu color group di papan
    int countSameColorTotal(const std::string& colorGroup, GameState& gs) const;
```

Di main.cpp, COM dibuat seperti player biasa dan GameMaster tidak perlu tahu bedanya:

```
Player* p;
if (setup.isBot[i])
    p = new ComputerPlayer(name, miscCfg.initialBalance, setup.botDifficulty);
else
    p = new Player(name, miscCfg.initialBalance);
p->setPosition(0);
players.push_back(p);
```

2. Tiga level kesulitan

```
enum class COMDifficulty {  
    EASY,    // Keputusan acak / naif  
    MEDIUM, // Keputusan berbasis aturan sederhana  
    HARD     // Keputusan strategis (heuristic-based)  
};
```

3. Sistem evaluasi property (Hard)

evaluatePropertyScore() menghitung skor numerik sebelum memutuskan beli atau bid

```

double ComputerPlayer::evaluatePropertyScore(Property* prop,
                                             GameState& gs) const {
    if (!prop) return 0.0;

    double score = 0.0;
    int price = prop->getPurchasePrice();
    if (price <= 0) return 0.0;

    // Faktor 1: Efisiensi biaya-sewa (sewa / harga beli)
    int baseRent = estimateRentValue(prop, gs);
    score += (static_cast<double>(baseRent) / price) * 100.0;

    // Faktor 2: Potensi monopoli color group
    std::string color = prop->getColorGroup();
    int owned = countSameColorOwned(color, gs);
    int total = countSameColorTotal(color, gs);
    if (total > 0) {
        double monopolyProgress = static_cast<double>(owned + 1) / total;
        score += monopolyProgress * 50.0;

        // Bonus besar jika ini properti terakhir untuk monopoli
        if (owned + 1 == total)
            score += 80.0;
    }
}

```

Threshold keputusan: beli jika score > 40, bid agresif jika score > 120.

4. Alur Execute Turn

Satu giliran COM dijalankan secara otomatis dalam urutan yang mencerminkan aturan spesifikasi:

- a. Gunakan Skill Card (jika ada & belum dipakai)
- b. Cek status JAILED: Bayar denda → gerak normal, Coba double → jika gagal, incrementJailTurns(). giliran ke-4 → wajib bayar
- c. Lempar dadu normal. Double? → loop giliran tambahan (max sebelum triple → jail)
- d. Aksi pasca-gerak (MEDIUM & HARD saja). Coba bangun rumah/hotel atau Coba tebus properti tergadai

5. Keputusan Per Situasi

- a. Easy:
 - Beli properti: 50% peluang
 - Bid lelang: selalu pass atau bid minimum
 - Tidak pernah bangun
 - Tidak pernah gadai kecuali terpaksa
 - Pilih pajak flat selalu
 - Tidak pernah pakai skill card
- b. Medium:
 - Beli properti: jika cukup uang dan properti tidak terlalu mahal
 - Bid lelang: bid sampai 70% saldo atau batas yang masuk akal
 - Bangun jika punya monopoli dan ada uang cadangan
 - Gadai properti dengan sewa terendah saat perlu uang
 - Pilih pajak yang lebih hemat
 - Pakai shield card jika dalam bahaya
- c. Hard:
 - Beli properti: evaluasi skor strategis
 - Bid lelang: bid agresif untuk properti bernilai tinggi
 - Bangun sesegera mungkin setelah monopoli
 - Gadai properti sewa terendah, tebus yang menguntungkan
 - Pilih pajak berdasarkan kalkulasi aktual kekayaan

- Pilih festival untuk properti sewa tertinggi
- Gunakan skill card secara optimal

14. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
	NIM mahasiswa yang membuat implementasi, bisa lebih dari satu	NIM mahasiswa yang melakukan tes, bisa lebih dari satu
Modul Auction Manager	13524077	13524077
Modul Bank	13524077	13524077
Modul Dadu	13524077	13524077
Modul Tile	13524075	13524075
Modul Board	13524075	13524075
Modul Pajak	13524075, 13524047	13524075, 13524047
Modul Bankrupt	13524047	13524047
Modul Game Master	13524049, 13524075, 13524047, 13524077	13524049, 13524075, 13524047
Modul Game State	13524049, 13524047, 13524077	13524049
Modul GameSync	13524075, 13524077	13524075
Modul Player	13524049, 13524047	13524049
Modul Property	13524049	13524049
Modul Card	13524049	13524049

Modul Kartu Kesempatan	13524075, 13524049	13524075
Modul Kartu Dana Umum	13524075, 13524049, 13524047	13524075, 13524047
Modul Carddeck	13524049	13524049
Modul Cetak Akta	13524049	13524049
Modul Cetak Properti	13524049	13524049
Modul Gunakan Kartu Kemampuan	13524049	13524049
Modul Drop Kartu Kemampuan	13524049	13524049
Modul Exception	13524049, 13524079, 131524077	13524049, 13524079, 13524077
Modul COM	13524079	13524079
Modul TransactionLogger	13524079	13524079
Modul ConfigLoader	13524079	13524079
Modul WinScreen	13524079	13524079
Modul GameScreen	13524079, 13524077	13524079
Modul MainMenuScreen	13524079	13524079
Modul Festival	13524079, 13524075	,13524075,13524079
Modul GUIManager	13524079	13524079
Modul SaveLoadManager	13524079	13524079

Lampiran

Lampiran A. Form Asistensi

Pranala Dokumen Asistensi:  IF2010_TB1_Asistensi_PCH_Signed.pdf